RKNN3 Runtime C API Reference

Document ID: RK-YH-YF-425

Version: V1.2.0

Date: 2026-10-22

Confidentiality: □Top Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

Address: No.18, Software Park, Tongpan Road, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document is the Rockchip RKNN3 Runtime C API Reference Manual.

Target Audience

This document (this guide) is primarily intended for the following engineers:

- Technical Support Engineers
- Software Development Engineers

Revision History

Version	Author	Date	Description	Approved By
V0.1.0	HPC Team	2025-05-12	Initial version	Vincent
V0.2.0	HPC Team	2025-08-04	Updated API interface descriptions	Vincent
V0.3.0b0	HPC Team	2025-09-10	Added description for rknn3_set_internal_mem interface; added KV cache storage methods and data type definitions; improved LLM configuration struct content	Vincent
V0.4.0b0	HPC Team	2025-11-03	Updated rknn3_vocab_info and rknn3_llm_multimodal_tensor structures and related interface descriptions	Vincent
V0.5.0	HPC Team	2025-11-29	Added post-processing related query commands and custom operator plugin mechanism; adjusted output callback function interface; added YUV input related structures and interface descriptions	Vincent
V1.0.0	HPC Team	2025-12-22	Added performance and memory analysis interfaces	Vincent
V1.0.4	HPC Team	2026-05-09	Added streaming weight loading interface; added rknn3_create_mem_from_fd interface	Vincent
V1.1.0	HPC Team	2026-08-03	Added rknn3_set_weight_mem, rknn3_set_kvcache_group, rknn3_session_pause, rknn3_session_resume, rknn3_mem_sync_range and other interfaces; restructured chapters to match the reference manual format; added multimodal inference and external memory usage flows; synchronized with rknn3_api.h header file	Vincent
V1.2.0	HPC Team	2026-10-22	Added PagedAttention support; added structured output (grammar) support; added MatMul matrix multiplication interfaces	Vincent

Table of Contents

- RKNN3 Runtime C API Reference
  - Table of Contents
  - 1. Reference Manual Overview
    - 1.1 Applicable SDK and Runtime Versions
    - 1.2 Supported Platforms
    - 1.3 Header File and Runtime Library
    - 1.4 API Stability and Compatibility
  - 2. Compilation and Linking
    - 2.1 Linux
    - 2.2 Android
    - 2.3 Cross-Compilation
  - 3. C API Common Conventions
    - 3.1 Context, Model, and Session Lifecycle
    - 3.2 Memory Ownership
    - 3.3 Input and Output Data Validity
    - 3.4 Synchronous and Asynchronous Behavior
    - 3.5 Thread Safety and Callbacks
    - 3.6 Return Values and Error Handling
    - 3.7 Platform Differences
    - 3.8 API Deprecation Rules
  - 4. Call Flow
    - 4.1 General Model Inference
    - 4.2 LLM Session Inference
    - 4.3 Multimodal Large Model Inference
    - 4.4 External Memory Usage Flow
      - 4.4.1 Weight Memory
      - 4.4.2 Internal Memory
      - 4.4.3 KV Cache Memory
      - 4.4.4 Paged KV Cache Memory
    - 4.5 MatMul Matrix Multiplication
      - 4.5.1 Multiple Shape Registration and Use Cases
      - 4.5.2 Multiple Shape Switching Sequence
      - 4.5.3 Quantization Parameters for Multiple Shapes

5. RKNN3 C API Description

5.1. Device and Context Management

rknn3_init

rknn3_destroy

rknn3_find_devices

rknn3_set_input_name_alias

5.2. Model Loading and Initialization

rknn3_load_model_from_path

rknn3_load_model_from_data

rknn3_load_model_from_device

rknn3_model_init

rknn3_load_weight_chunk

5.3. Query Interface

rknn3_query

5.4. Inference Execution

rknn3_run

rknn3_set_shape

5.5. Memory Management

rknn3_create_mem_from_fd

rknn3_create_mem

rknn3_destroy_mem

rknn3_mem_sync

rknn3_mem_sync_range

rknn3_set_kvcache_pages

rknn3_set_kvcache_mem

rknn3_set_internal_mem

rknn3_set_weight_mem

rknn3_set_kvcache_group

5.6. Model Decryption

rknn3_set_decrypt_key_from_path

rknn3_set_decrypt_key_from_data

5.7. LLM Session

rknn3_session_init

rknn3_session_destroy

rknn3_session_run

rknn3_session_run_async

rknn3_session_stop
rknn3_session_pause
rknn3_session_resume
rknn3_session_query_state
5.8. LLM Configuration and Callbacks
rknn3_session_set_callback
rknn3_session_set_chat_template
rknn3_session_set_llm_param
rknn3_session_set_function_tools
LLMResultCallback
LLMSamplingCallback
LLMGetEmbedCallback
LLMTokenizerCallback
LLMOutputCallback
LLMInputCallback
5.9. KV Cache Management
rknn3_session_set_kvcache_policy
rknn3_session_clear_kvcache
rknn3_session_load_kvcache_from_path
rknn3_session_load_kvcache_from_data
rknn3_session_save_kvcache
5.10. LoRA Adapters
rknn3_lora_init
rknn3_lora_init_from_data
rknn3_lora_load
rknn3_lora_unload
rknn3_lora_enable
rknn3_lora_disable
rknn3_session_enable_lora
rknn3_session_disable_lora
5.11. Profiling and Debugging
rknn3_dump_features
rknn3_profile_ops
rknn3_profile_mem
5.12. Custom Operators
rknn3_register_custom_ops_plugins

rknn3_register_custom_op_func
5.13. Image Processing
rknn3_im_mem_create
rknn3_im_mem_destroy
rknn3_im_cvt_color
5.14. Auxiliary Functions
rknn3_get_type_string
rknn3_get_qnt_type_string
rknn3_get_layout_string
5.15. Structured Output (Grammar)
rknn3_init_grammar
rknn3_grammar_create
rknn3_grammar_destroy
rknn3_grammar_set
rknn3_grammar_clear
rknn3_grammar_apply_mask
rknn3_grammar_accept_token
rknn3_grammar_reset
rknn3_session_set_grammar
rknn3_session_clear_grammar
5.16. MatMul Matrix Multiplication
rknn3_matmul_init
rknn3_matmul_query_infos
rknn3_matmul_build
rknn3_matmul_set_shape
rknn3_matmul_transpose
rknn3_matmul_set_quant
rknn3_matmul_run
rknn3_matmul_destroy
Usage Example
6. Type Reference
6.1. Constants
6.1.1. Tensor Related Constants
6.1.2. Device Related Constants
6.2. Status Codes
6.3. Enumerations

6.3.1. Tensor and Query Enumerations

rknn3_query_cmd

rknn3_tensor_type

rknn3_tensor_qnt_type

rknn3_tensor_layout

6.3.2. Memory and Core Enumerations

rknn3_mem_alloc_flags

rknn3_mem_sync_mode

rknn3_mem_type

rknn3_core_mask

6.3.3. KV Cache Enumerations

rknn3_kvcache_policy

rknn3_kvcache_clear_policy

rknn3_kvcache_dtype

rknn3_kvcache_store_method

rknn3_attention_type

rknn3_page_cache_type

6.3.4. LLM Enumerations

rknn3_llm_input_type

LLMCallState

rknn3_llm_task_type

LLMOutputCallbackState

6.3.5. Image Processing Enumerations

rknn3_im_fmt

rknn3_im_proc_flag

6.3.6. Custom Operator Enumerations

rknn3_op_target_type

rknn3_op_plugin_type

6.3.7. Structured Output Enumerations

rknn3_grammar_type

6.4. General Structures

Tensor Related Structures

rknn3_input_output_num

rknn3_quant_info

rknn3_tensor_attr

rknn3_tensor_mem

rknn3_tensor
Model and Configuration Structures
rknn3_sdk_version
rknn3_core_mem_size
rknn3_custom_string
rknn3_config
rknn3_shape_info
rknn3_shape_config
rknn3_allocation_info
Device Related Structures
rknn3_init_extend
rknn3_node_mem_info
rknn3_dev_mem_info
rknn3_device
rknn3_devices
6.5. LLM and VLM Structures
rknn3_llm_config
rknn3_vocab_info
rknn3_llm_extend_param
rknn3_sampling_params
rknn3_llm_param
rknn3_lora
rknn3_llm_multimodal_tensor
rknn3_llm_tensor
rknn3_llm_input
rknn3_llm_infer_param
RKLLMResult
RKLLMRunState
6.6. KV Cache Structures
KV Cache Related Structures
rknn3_kvcache_policy_param
rknn3_attention_kvcache_lens
rknn3_kvcache_len_group_info
rknn3_attention_config
rknn3_cache_page_attr
rknn3_kvcache_page_info

rknn3_kvcache_page_binding

6.7. Custom Operator Structures

Custom Operator Structures

rknn3_custom_op_attr

rknn3_custom_op_context

rknn3_custom_op

6.8. Image Processing Structures

Image Processing Structures

rknn3_im_rect

rknn3_im_metadata

rknn3_im_mem

6.9. Callback Types

LLMResultCallback

LLMSamplingCallback

LLMGetEmbedCallback

LLMTokenizerCallback

LLMOutputCallback

LLMInputCallback

RKLLMCallback

6.10. Structured Output Structures

Structured Output Structures

rknn3_grammar_config

rknn3_grammar

rknn3_grammar_set_options

6.11. MatMul Types

rknn3_matmul_context

rknn3_matmul_type

rknn3_matmul_desc

rknn3_matmul_config

rknn3_matmul_tensor_attr

rknn3_matmul_info

rknn3_matmul_quant_type_t

rknn3_matmul_quantization_t

Storage conventions for scales and zero_points

W4A16/W6A16/W8A16: B grouped along K

W8A8 per-channel: one parameter pair per B output channel

W8A8 per-layer: one parameter pair per B batch

W16A16/W8A8O32: no scale/zp submission

7. RKNN3 C API Change Log

v0.3.0 -> v0.4.0 Session API Main Changes (2025-11-03)

v0.4.0b0 -> v0.5.0 Main Changes (2025-11-29)

v0.5.0 -> v1.0.0 Main Changes (2025-12-22)

v1.0.0 -> v1.0.4 Main Changes (2026-05-09)

v1.0.4 -> v1.1.0 Main Changes (2026-08-03)

v1.1.0 -> v1.2.0 Main Changes (2026-10-22)

1. Reference Manual Overview

The RKNN3 C API is the C language interface for the RKNN3 Runtime. Developers use C/C++ to develop applications and deploy model inference through the RKNN3 C API. This document describes the functions, data structures, status codes, and other definitions of the RKNN3 C API.

1.1 Applicable SDK and Runtime Versions

This document corresponds to RKNN3 Runtime release version V1.2.0 (see Revision History). The interfaces described in this document are based on version V1.2.0.

1.2 Supported Platforms

This document applies to the following hardware platforms:

Coprocessor Platforms (NPU accelerators, connected to the host SoC via PCIe/USB):

- RK1820
- RK1828

General SoC Platforms (integrated NPU, applications run directly on the SoC):

- RK3572

1.3 Header File and Runtime Library

The RKNN3 SDK supports two deployment architectures:

Coprocessor mode: The application is compiled and executed on the host SoC, with the coprocessor (RK1820/RK1828) serving only as an NPU acceleration unit. The host SoC connects to the coprocessor via a high-speed PCIe/USB interface, such as an RK3588 connected to an RK1820/RK1828. In this mode, the versions of librknn3_api, rknn3_transfer_proxy, and the coprocessor firmware must be consistent.

General SoC mode: The application is compiled and executed directly on the RK3572, with the NPU being an integrated unit within the SoC, requiring no coprocessor communication. In this mode, the versions of librknn3_api and rknn3_server must be consistent.

In both modes, developers need to include the `rknn3_api.h` header file and link with the `librknn3_api.so` runtime library when compiling an RKNN3 application. To use MatMul, also include `rknn3_matmul_api.h`.

The RKNN3 C API is defined in the `rknn3_api.h` header file. It is organized by function into basic inference interfaces for general deep learning model deployment (e.g., `rknn3_init`, `rknn3_run`) and a series of session interfaces specifically optimized for large language models (e.g., `rknn3_session_init`, `rknn3_session_run`). The latter provides functionalities such as KV cache management, LoRA support, and sampling strategy configuration.

The RKNN3 SDK involves three dynamic libraries at runtime, with the following relationships:

Library Name	Description
librknn3_api.so	API entry layer, linked by the application. Automatically loads the corresponding backend library based on the deployment mode at runtime

Library Name	Description
librknn3_api_rkcp.so	Coprocessor mode backend library. Communicates with rknn3_transfer_proxy via localsocket, then forwards to the coprocessor via PCIe/USB for inference execution
librknn3_api_native.so	General SoC mode backend library. Directly accesses the SoC's integrated NPU without coprocessor communication

Coprocessor mode call chain:

```
APP -> librknn3_api.so -> librknn3_api_rkcp.so -> rknn3_transfer_proxy -> PCIe/USB -
> Coprocessor(RK1820/RK1828)
```

General SoC mode call chain:

```
APP -> librknn3_api.so -> librknn3_api_native.so -> NPU(integrated in SoC)
```

The application only needs to link `librknn3_api.so`; at runtime, this library automatically detects the deployment mode and loads `librknn3_api_rkcp.so` or `librknn3_api_native.so`. During deployment, ensure that the corresponding backend library is located in the same directory as `librknn3_api.so` or in the system library search path. The application can also directly link librknn3_api_rkcp.so or librknn3_api_native.so depending on the actual scenario.

There are two ways to obtain the runtime version number:

- Run the `strings` command on the corresponding runtime library and filter the version string, e.g., `strings librknn3_api.so | grep "version:"` (for coprocessor mode use `librknn3_api_rkcp.so`, for general SoC mode use `librknn3_api_native.so`) to read the runtime version from the version string;
- Query the actual version at runtime via `RKNN3_QUERY_SDK_VERSION`. The result is written to the `api_version` and `drv_version` fields of the `rknn3_sdk_version` structure (see the `rknn3_query` interface in 5. Function Reference).

1.4 API Stability and Compatibility

- **Version consistency requirement:** The versions of `librknn3_api`, `rknn3_transfer_proxy` (coprocessor mode) and the coprocessor firmware, or `rknn3_server` (general SoC mode), must be consistent. If versions are incompatible, `RKNN3_ERR_INCOMPATIBLE_VERSION` is returned.
- **Platform limitations:** Some interfaces are only available on specific platforms. For example, `rknn3_set_weight_mem` only supports RK3572 in the current version.

2. Compilation and Linking

When compiling an RKNN3 application, developers need to select the appropriate cross-compilation toolchain based on the target platform's hardware architecture and system type, and ensure that the RKNN3 API library for the corresponding platform is correctly linked. The RKNN3 C API call flow is consistent across coprocessor mode and general SoC mode, with the only difference being the underlying communication method (coprocessor mode communicates with the coprocessor via PCIe/USB, while general SoC mode directly accesses the integrated NPU).

2.1 Linux

When compiling an RKNN3 application on Linux (including Linux on the host SoC), you need to include the `rknn3_api.h` header file and link with `librknn3_api.so`. The RKNN3 runtime library is a prebuilt artifact provided with the SDK; developers do not need to compile the runtime library themselves and only need to compile their own application using the corresponding cross-compilation toolchain. Compilation example:

```
aarch64-linux-gnu-gcc app.c -I/path/to/include -L/path/to/lib -lrknn3_api -o app
```

2.2 Android

When compiling an RKNN3 application on Android, use the Android NDK toolchain with the target architecture arm64-v8a. The RKNN3 runtime library is a prebuilt artifact provided with the SDK; developers do not need to compile the runtime library themselves and only need to compile their own application using the NDK toolchain. Include `rknn3_api.h` and link with `librknn3_api.so` when compiling. Compilation example:

```
aarch64-linux-android24-clang app.c -I/path/to/include -L/path/to/lib -lrknn3_api -o
app
```

2.3 Cross-Compilation

In coprocessor mode, the application is compiled and executed on the host SoC, requiring the cross-compilation toolchain corresponding to the host SoC. For instructions on downloading and installing the cross-compiler for the host SoC, please refer to the <`Rockchip_RK182X_Quick_Start_RKNN3_SDK`> document. In general SoC mode, the application is compiled and executed directly on the RK3572, using the toolchain corresponding to RK3572.

Regardless of the mode, when compiling you must:

1. Include the directory containing `rknn3_api.h` in the header file search path;
2. Include the directory containing `librknn3_api.so` in the library search path, and link with `-lrknn3_api`;
3. Select the cross-compilation toolchain for the corresponding platform based on the deployment architecture.

3. C API Common Conventions

This chapter describes common conventions for using the RKNN3 C API, including object lifecycle, memory ownership, synchronous/asynchronous behavior, thread safety, return values and error handling, platform differences, and deprecation rules. For specific constraints of each function, see 5. Function Reference.

3.1 Context, Model, and Session Lifecycle

The RKNN3 API is organized around three core objects: runtime context (context), model (model), and session (session). Their typical lifecycle is as follows:

1. **Context initialization:** Call `rknn3_init` to create a context; this must be called before loading and running a model. Call `rknn3_destroy` to release when no longer needed; each context is destroyed only once.
2. **Model loading and initialization:** Load the model via `rknn3_load_model_from_path` / `rknn3_load_model_from_data` (model files on the device-side filesystem can be loaded via `rknn3_load_model_from_device`), then call `rknn3_model_init` to complete model initialization configuration.
3. **Streaming weight loading (optional):** When the weight parameter is NULL during model loading, streaming weight loading mode is entered. After `rknn3_model_init`, weight data must be uploaded in segments via `rknn3_load_weight_chunk`; when the cumulative uploaded bytes reach the total weight size, the streaming state is automatically completed.
4. **Session initialization (LLM scenarios):** Call `rknn3_session_init` to create a session; call `rknn3_session_destroy` to destroy the session when no longer needed.
5. **Resource release order:** First destroy the session (`rknn3_session_destroy`), then destroy the context (`rknn3_destroy`).

Additionally, some interfaces have requirements on calling timing:

- Model decryption key setting (`rknn3_set_decrypt_key_from_*`) must be called before `rknn3_load_model_from_*`.
- `rknn3_set_kvcache_mem` must be called after `rknn3_model_init`.
- User-allocated internal memory (`rknn3_set_internal_mem`) and weight memory (`rknn3_set_weight_mem`) require setting `config.user_mem_internal` / `config.user_mem_weight` to 1 before `rknn3_model_init`, then calling the corresponding setting interface after `rknn3_model_init`.
- LoRA initialization (`rknn3_lora_init` / `rknn3_lora_init_from_data`) must be called after `rknn3_model_init` and before the session uses LoRA.

3.2 Memory Ownership

- **API-allocated memory:** Tensor memory allocated via `rknn3_create_mem` is owned by the caller and must be released via `rknn3_destroy_mem` when no longer needed; avoid double-freeing.

- **User-allocated memory:** User-allocated memory accepted by `rknn3_set_kvcache_mem`, `rknn3_set_internal_mem`, and `rknn3_set_weight_mem` remains owned by the caller; after use, the caller must release it via `rknn3_destroy_mem`.
- **Paged KV cache memory:** Each page of a paged KV cache is an independent `rknn3_tensor_mem` object. The caller creates it with `rknn3_create_mem` and destroys it with `rknn3_destroy_mem`. `rknn3_set_kvcache_pages` does not copy page contents or take ownership of page memory; bound pages must remain valid until they are unbound. Before destroying a page, unbind its page-table entry by submitting a binding with `n_pages = 0`.
- **External memory wrapping:** `rknn3_create_mem_from_fd` associates already-allocated memory (represented by a file descriptor) with the RKNN3 framework to achieve zero-copy; the wrapped `rknn3_tensor_mem` object still needs to be released via `rknn3_destroy_mem`.
- **Memory in callbacks:** Tensor data pointers passed in callback functions (e.g., custom operator attribute data, output tensors) are typically not owned by the caller and are only valid within the current callback scope; the caller should not cache or free them.

3.3 Input and Output Data Validity

- The `inputs` / `outputs` arrays of the inference interface (`rknn3_run`) and the `rknn3_tensor` and memory they point to must remain valid during the call, and the input/output tensor memory must be allocated by the user in advance.
- After the CPU writes to input memory and before the NPU accesses it, call `rknn3_mem_sync` (`RKNN3_MEMORY_SYNC_TO_DEVICE`) to synchronize CPU cache data to memory; after the NPU writes to output memory and before the CPU reads it, call `rknn3_mem_sync` (`RKNN3_MEMORY_SYNC_FROM_DEVICE`) to invalidate the CPU cache and re-read from memory. Failure to synchronize may cause abnormal inference results.
- Tensor pointers obtained in callbacks (e.g., `output_tensors` of `LLMOutputCallback`, `data` of `rknn3_custom_op_attr`) are only valid within the callback scope and should not be used after the callback returns.

3.4 Synchronous and Asynchronous Behavior

- **General model synchronous inference:** `rknn3_run` is a synchronous interface that blocks until completion during inference.
- **LLM session inference:** The inference results of `rknn3_session_run` / `rknn3_session_run_async` are obtained asynchronously via callback functions (see 5.8 LLM Configuration and Callbacks).
- **Memory synchronization:** `rknn3_mem_sync` / `rknn3_mem_sync_range` are used for data consistency synchronization between the CPU and the device; `rknn3_mem_sync_range` only synchronizes the specified `[offset, offset+length)` range, suitable for partial update scenarios.

3.5 Thread Safety and Callbacks

- Concurrent use of the same `rknn3_context` or `rknn3_session` requires the caller to ensure thread safety; avoid multiple threads operating on the same handle simultaneously.

- LLM inference results are returned asynchronously via callbacks, which are triggered during inference; callbacks should not perform long-blocking operations or recursively initiate inference on the same session, so as not to affect the inference flow.
- Temporary data passed in callbacks (tensor pointers, attribute data, etc.) is only valid within the callback scope; if it needs to be retained, the caller should copy it.

3.6 Return Values and Error Handling

- Most interfaces return `int` status codes: **0 indicates success**, **negative values indicate errors**. A few interfaces return pointers (e.g., `rknn3_create_mem`, `rknn3_create_mem_from_fd`, `rknn3_session_init`); they return `NULL` on failure.
- For the complete status code definitions, see 6.2 Status Codes.
- The caller should check the return value of each call; when handling errors, use the status code to locate the problem (e.g., `RKNN3_ERR_INCOMPATIBLE_VERSION` indicates component version mismatch, `RKNN3_ERR_WEIGHT_LOAD_NOT_PREPARED` indicates streaming weight loading is not prepared, etc.).

3.7 Platform Differences

- **Deployment mode differences:** Coprocessor mode communicates with the coprocessor via PCIe/USB, requiring `librknn3_api_rkcp.so` + `rknn3_transfer_proxy` + coprocessor firmware; general SoC mode directly accesses the integrated NPU, requiring `librknn3_api_native.so` + `rknn3_server`. See 1.3 Header File and Runtime Library.
- **Interface availability differences:** Some interfaces are only supported on specific platforms. For example, `rknn3_set_weight_mem` only supports RK3572 in the current version.
- **Call flow consistency:** The C API call flow is consistent across coprocessor mode and general SoC mode; the only difference is the underlying communication method.

3.8 API Deprecation Rules

- For the deprecation and change history of interfaces, see 7. RKNN3 C API Change Log.
- All component versions must be consistent (see 1.4 API Stability and Compatibility); when upgrading, `librknn3_api`, `rknn3_transfer_proxy` / `rknn3_server`, and the firmware must be updated together.

4. Call Flow

The RKNN3 C API call flow is consistent across coprocessor mode and general SoC mode, with the only difference being the underlying communication method (coprocessor mode communicates with the coprocessor via PCIe/USB, while general SoC mode directly accesses the integrated NPU). The following describes the inference flows for general model synchronous inference, general model asynchronous inference, LLM Session inference, multimodal large model inference, and external memory usage.

4.1 General Model Inference

The inference flow for non-large language models such as CNNs/ViTs using the basic API mainly includes the following steps. First, the user needs to call the rknn3_init interface to initialize the runtime context, and load the model using rknn3_load_model_from_path or rknn3_load_model_from_data. After the model is loaded, rknn3_model_init must be called for model initialization and configuration. Subsequently, the user can query the attribute information of input and output tensors through the rknn3_query interface and allocate memory for inputs and outputs. When performing inference, the rknn3_run interface is called to complete synchronous inference. Finally, rknn3_destroy is used to release the context resources. The inference flow for the basic API is shown in Figure 4-1.

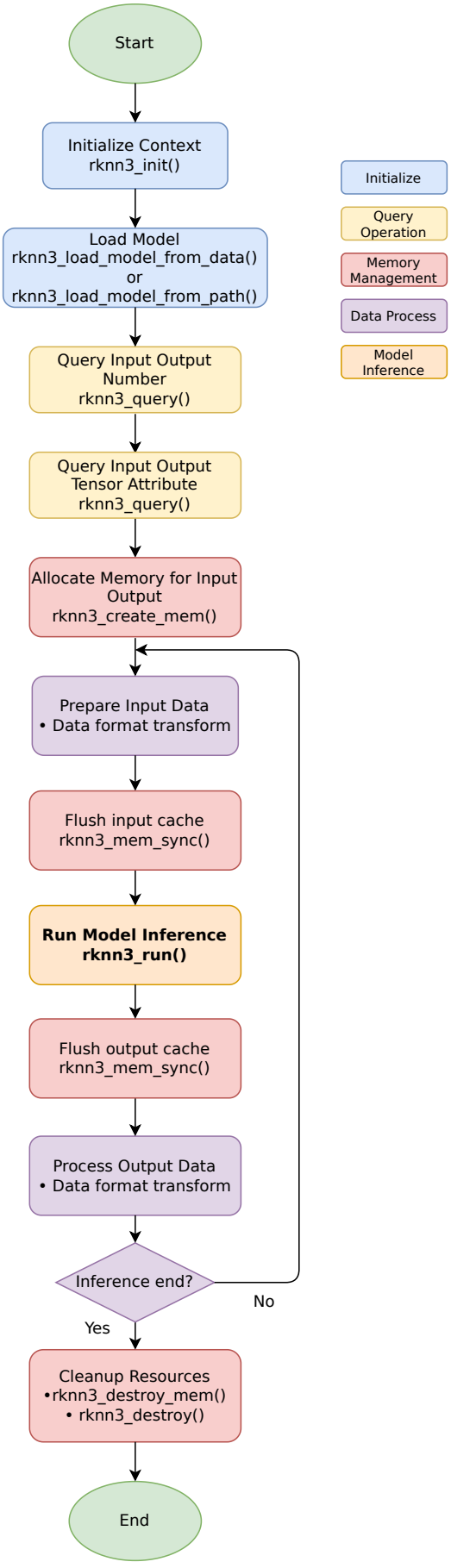


Figure 4-1 Basic API Inference Flow

4.2 LLM Session Inference

The RKNN3 large language model inference flow typically includes the following main steps. First, the user needs to get a list of available devices by calling rknn3_find_devices, initialize the context with rknn3_init, load the model file using rknn3_load_model_from_path, and complete the model initialization configuration with rknn3_model_init. Next, the user needs to prepare resources such as the tokenizer and embeddings, and set up the rknn3_llm_param structure to configure session parameters. Then, call rknn3_session_init to initialize the session and set callback functions for the inference process via rknn3_session_set_callback. Before inference, a chat template can be set or the KV cache can be cleared as needed.

During the inference phase, the user constructs an rknn3_llm_input structure and calls the rknn3_session_run interface for inference. The inference results are obtained asynchronously through callback functions. After inference is complete, performance data can be collected or multi-turn dialogues can be conducted. Finally, the rknn3_session_destroy and rknn3_destroy interfaces must be called to release session and context resources, completing the entire inference flow. The large language model inference flow is shown in Figure 4-2.

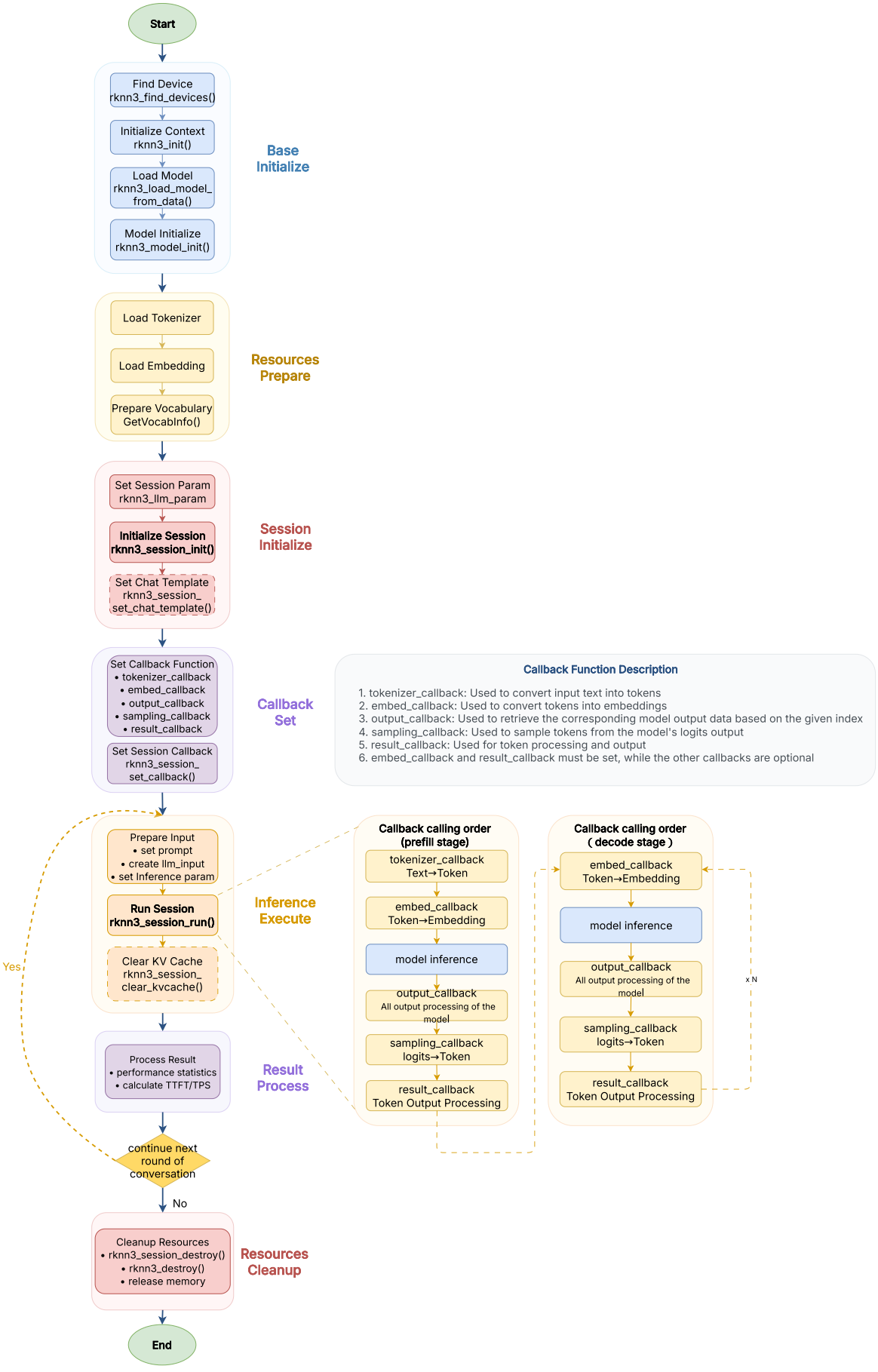


Figure 4-2 Large Language Model Inference Flow

4.3 Multimodal Large Model Inference

Multimodal large model inference (supporting speech, image, text, video, and other modality inputs) is based on the LLM Session flow. The key difference from pure-text LLM inference is that, in addition to the LLM itself, independent modality encoder models (e.g., vision encoder, audio encoder) are needed to convert non-text modalities such as images and audio into embedding vectors, which are then fed into the LLM together with the text prompt to complete inference. The overall flow of multimodal large model inference is shown in Figure 4-3.

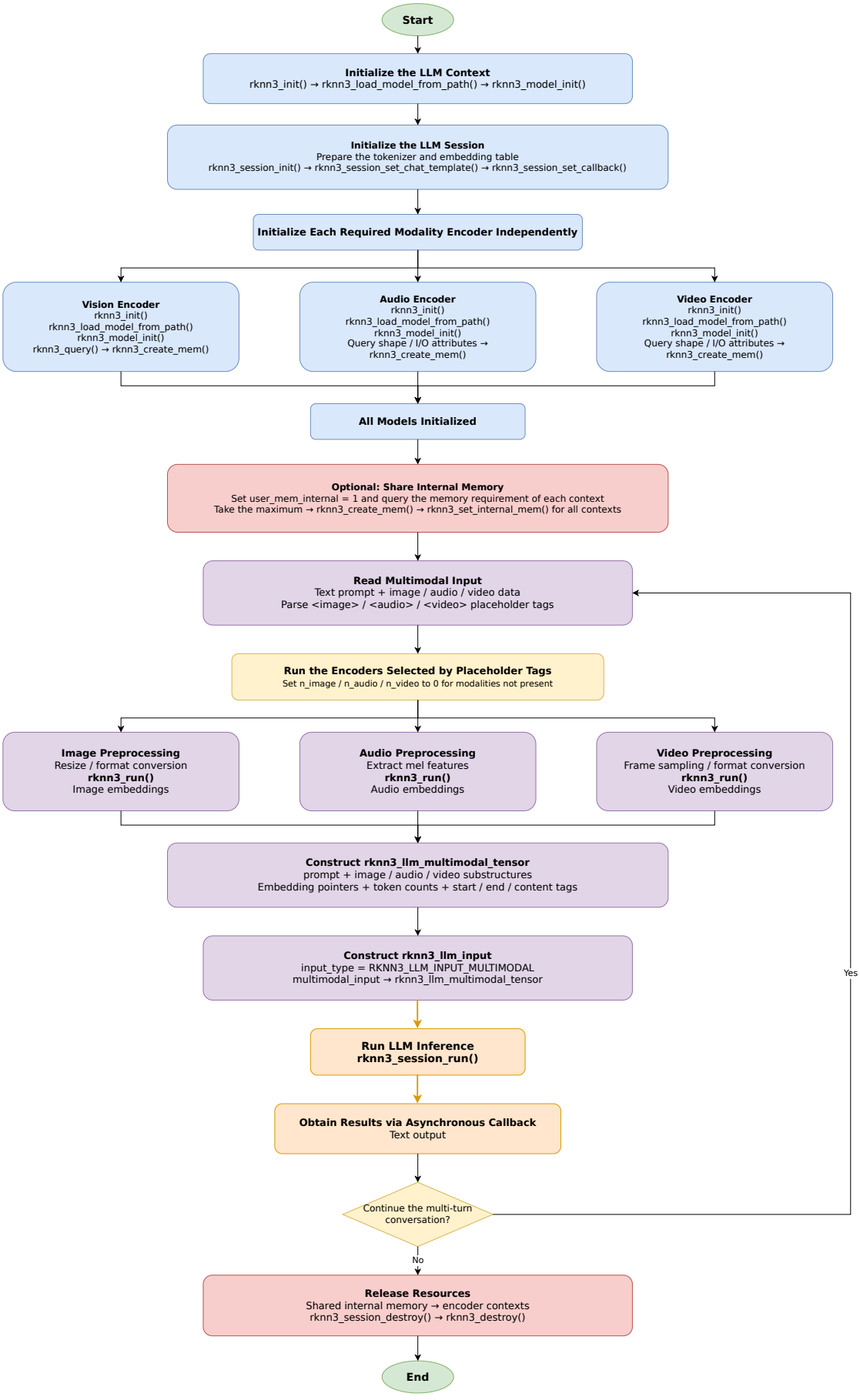


Figure 4-3 Multimodal Large Model Inference Flow

The main steps are described as follows:

1. **Model initialization:** Multimodal inference involves multiple RKNN3 contexts—one context per modality encoder (e.g., vision encoder, audio encoder) and one for the LLM. The initialization flow for each context is the same: `rknn3_init` creates the context, `rknn3_load_model_from_path` loads the model, and `rknn3_model_init` completes model initialization. For modality encoders, query input/output attributes via `RKNN3_QUERY_INPUT_ATTR` / `RKNN3_QUERY_OUTPUT_ATTR` and allocate memory with `rknn3_create_mem`. For the LLM, call `rknn3_session_init` to create a session, and set the chat template via `rknn3_session_set_chat_template` and callbacks via `rknn3_session_set_callback`. Additionally, auxiliary resources such as the tokenizer and embedding table must be prepared for tokenization and token embedding lookup in callbacks.
2. **Internal memory sharing:** To save memory, the contexts of modality encoders and the LLM can share the same internal memory block. During initialization, set `config.user_mem_internal` to 1 for each context. After model initialization, query the required internal memory size for each core of each context via `RKNN3_QUERY_CORE_NUMBER` and `RKNN3_QUERY_CORE_MEM_SIZE`, allocate memory with the maximum size via `rknn3_create_mem`, and set it to all contexts via `rknn3_set_internal_mem`.
3. **Modality embedding generation:** Based on the modality placeholder tags in the input prompt, run the corresponding encoders to generate embeddings. If the prompt contains `<image>`, preprocess the input image (resize, format conversion, etc.) and feed it to the vision encoder for inference (`rknn3_run`) to obtain image embeddings. If it contains `<audio>`, preprocess the input audio (e.g., extract mel features) and feed it to the audio encoder for inference to obtain audio embeddings. Video modality is similar. For modalities not present, set their count (`n_image` / `n_audio` / `n_video`) to 0 to skip.
4. **Construct multimodal input and infer:** Construct `rknn3_llm_multimodal_tensor`, set the `prompt` text, and fill each modality's embeddings and corresponding tags into the `image` / `audio` / `video` substructures (embed pointers, token counts, counts, start/end/content placeholder tags). The tags must be consistent with the model's convention, e.g., images use `<|vision_bos|>` / `<|vision_eos|>` / `<|IMAGE|>`, audio uses `<|audio_bos|>` / `<|audio_eos|>` / `<|AUDIO|>`. Use `<image>` / `<audio>` / `<video>` placeholder tags in the prompt to indicate the insertion positions of modality data; multiple modality data can be input in any order and combination. Then construct `rknn3_llm_input`, set `input_type` to `RKNN3_LLM_INPUT_MULTIMODAL`, point `multimodal_input` to the above structure, and call `rknn3_session_run` for inference. Results are obtained asynchronously via callback functions.
5. **Resource release:** After inference is complete, first release the shared internal memory, then release each modality encoder context and the LLM session and context resources.

The field definitions of the multimodal input structure `rknn3_llm_multimodal_tensor` are in 6.5. LLM and VLM Structures.

4.4 External Memory Usage Flow

When the user wishes to allocate and manage weight memory, internal memory, or KV cache memory themselves, the following flow can be used. The three types of external memory have

different query methods and setting timings: weight memory and internal memory require enabling the corresponding configuration flag before `rknn3_model_init`, and the size is queried via `RKNN3_QUERY_CORE_MEM_SIZE` after initialization; KV cache memory is queried via `RKNN3_QUERY_ALLOCATION_INFO` after `rknn3_model_init`. The overall usage flow for the three types of external memory is shown in Figure 4-4.

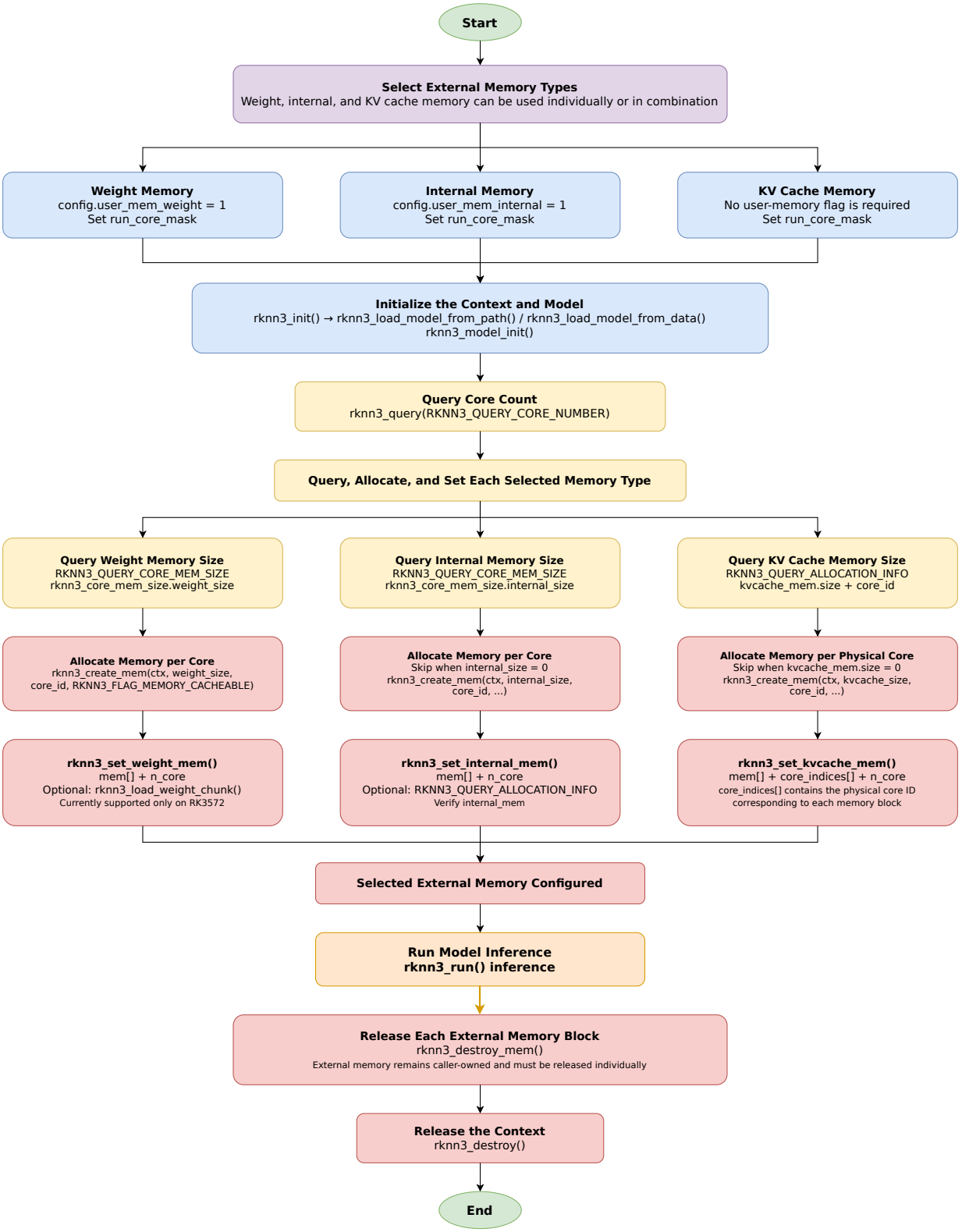


Figure 4-4 External Memory Usage Flow

> **Note:** The `user_mem_weight` / `user_mem_internal` configuration flags must be set before `rknn3_model_init`; the query, allocation, and setting of weight memory and internal memory must be completed after `rknn3_model_init` and before `rknn3_run`. KV cache memory must be set after `rknn3_model_init`. The three types of external memory can be used individually or in combination.
>
> **Important:** If the KV cache memory size (`kvcache_mem.size`) queried via `RKNN3_QUERY_ALLOCATION_INFO` is non-zero, user-allocated KV cache external memory must be set via `rknn3_set_kvcache_mem` before calling `rknn3_run` for inference; otherwise, inference will fail due to missing KV cache memory. If using the `rknn3_session_run` interface for LLM inference, there is no need to set KV cache external memory, as KV cache memory is managed internally by the session.

The specific steps for each memory type are described below:

4.4.1 Weight Memory

Weight memory is used to store model weight data. When using user-managed weight memory, the `user_mem_weight` flag must be enabled before model initialization, and the required weight memory size for each core is queried and allocated after initialization. Steps:

1. **Initialize context and load model:** Call `rknn3_init` to create a context, and load the model via `rknn3_load_model_from_path` or `rknn3_load_model_from_data`. When using user weight memory, the weight parameter of `rknn3_load_model_from_data` can be NULL (weights are subsequently uploaded via `rknn3_load_weight_chunk` in streaming mode) or the complete weight data can be passed in.
2. **Set configuration and initialize model:** Construct `rknn3_config`, set `config.user_mem_weight` to 1 and set `run_core_mask`, then call `rknn3_model_init` to complete model initialization.
3. **Query core count:** Query the number of cores via `RKNN3_QUERY_CORE_NUMBER` for subsequent per-core memory allocation.
4. **Query weight memory size:** Query the weight memory size for each core via `RKNN3_QUERY_CORE_MEM_SIZE` (the `weight_size` field of the `rknn3_core_mem_size` structure).
5. **Allocate and set weight memory:** For each core, call `rknn3_create_mem(ctx, weight_size, core_id, RKNN3_FLAG_MEMORY_CACHEABLE)` to allocate weight memory, collect into an array, and call `rknn3_set_weight_mem(ctx, mem[], n_core)` to set (currently only supports RK3572).
6. **Streaming weight loading (optional):** If the weight parameter was NULL in step 1, upload weight data in chunks via `rknn3_load_weight_chunk(ctx, offset, data, size)` until all weights are uploaded before inference.
7. **Release resources:** After inference is complete, release each weight memory via `rknn3_destroy_mem`, then call `rknn3_destroy` to release the context.

4.4.2 Internal Memory

Internal memory is used to store intermediate tensors and other data during model inference. When using user-managed internal memory, the `user_mem_internal` flag must also be enabled before model initialization, and the required internal memory size for each core is queried and allocated after initialization. Steps:

1. **Initialize context and load model:** Call `rknn3_init` to create a context, and load the model via `rknn3_load_model_from_path` or `rknn3_load_model_from_data`.
2. **Set configuration and initialize model:** Construct `rknn3_config`, set `config.user_mem_internal` to 1 and set `run_core_mask`, then call `rknn3_model_init` to complete model initialization.
3. **Query core count:** Query the number of cores via `RKNN3_QUERY_CORE_NUMBER`.
4. **Query internal memory size:** Query the internal memory size for each core via `RKNN3_QUERY_CORE_MEM_SIZE` (the `internal_size` field of the `rknn3_core_mem_size` structure). If `internal_size` is 0, no allocation is needed for that core.
5. **Allocate and set internal memory:** For each core, call `rknn3_create_mem(ctx, internal_size, core_id, RKNN3_FLAG_MEMORY_CACHEABLE)` to allocate internal memory, collect into an array, and call `rknn3_set_internal_mem(ctx, mem[], n_core)` to set.
6. **Verify (optional):** Query memory allocation info via `RKNN3_QUERY_ALLOCATION_INFO` (the `internal_mem` field of the `rknn3_allocation_info` structure) to confirm that internal memory has been set correctly.
7. **Release resources:** After inference is complete, release each internal memory via `rknn3_destroy_mem`, then call `rknn3_destroy` to release the context.

> **Tip:** In multimodal scenarios, multiple contexts (e.g., vision encoder, audio encoder, LLM) can share the same internal memory block to save memory. The approach is to enable `user_mem_internal` for each context and query the required memory size for each core, allocate a single shared memory block with the maximum size, and set it to all contexts via `rknn3_set_internal_mem` (see 4.3 Multimodal Large Model Inference).

4.4.3 KV Cache Memory

KV cache memory is used to store Key/Value caches during large language model inference. Unlike weight memory and internal memory, KV cache memory does not require enabling a configuration flag before model initialization; instead, the required size for each core is queried via `RKNN3_QUERY_ALLOCATION_INFO` after `rknn3_model_init` and then allocated and set. Steps:

1. **Initialize context and load model:** Call `rknn3_init` to create a context, load the model via `rknn3_load_model_from_path` or `rknn3_load_model_from_data`, set `run_core_mask`, and call `rknn3_model_init` to complete model initialization.
2. **Query core count:** Query the number of cores via `RKNN3_QUERY_CORE_NUMBER`.
3. **Query KV cache memory size:** Query the memory allocation info for each core via `RKNN3_QUERY_ALLOCATION_INFO` (the `rknn3_allocation_info` structure), where the `kvcache_mem.size` field is the required KV cache memory size for that core and the `core_id` field is the physical ID of that core. If `kvcache_mem.size` is 0, no allocation is needed for that core.

4. **Allocate KV cache memory:** For cores that need allocation, call `rknn3_create_mem(ctx, kvcache_size, core_id, RKNN3_FLAG_MEMORY_CACHEABLE)` to allocate KV cache memory, and record the corresponding `core_id` for each memory.
5. **Set KV cache memory:** Call `rknn3_set_kvcache_mem(ctx, mem[], core_indices[], n_core)` to set, where `core_indices[]` is the array of physical core IDs corresponding to each memory, and `n_core` is the number of memories. Note that this interface requires a core ID array, which differs from the weight/internal memory interfaces (which take a core count).
6. **Release resources:** After inference is complete, release each KV cache memory via `rknn3_destroy_mem`, then call `rknn3_destroy` to release the context.

4.4.4 Paged KV Cache Memory

Paged KV cache is used by LLM models that declare page cache types. Unlike a traditional contiguous KV cache, the runtime receives an array of individually allocated `rknn3_tensor_mem` pages and builds page-table bindings by request and cache type. The complete flow is:

1. **Complete model initialization:** Call `rknn3_init`, a model loading interface, and `rknn3_model_init` in order. Paged KV cache is only valid for paged models. For a non-paged model, `rknn3_kvcache_page_info.n_cache_types` is 0; use the `rknn3_set_kvcache_mem` flow in 4.4.3 KV Cache Memory instead.
2. **Query page properties:** For each participating physical core, set `rknn3_kvcache_page_info.core_id` to the core ID and call `rknn3_query(ctx, RKNN3_QUERY_KVCACHE_PAGE_INFO, &info, sizeof(info))`. `info.caches[]` reports each cache type's `cache_type`, page size in bytes (`page_size`), and compiled maximum page capacity (`max_page_num`). `max_request_num` reports the maximum number of concurrent requests for which page-table entries may be prepared.
3. **Query the paged configuration and calculate the page count:** Query `max_ctx_len`, `tokens_per_page`, and related settings through `RKNN3_QUERY_LLM_CONFIG` and `RKNN3_QUERY_ATTENTION_CONFIG`. For `RKNN3_PAGE_CACHE_FULL_ATTENTION`, allocate `ceil(max_ctx_len / tokens_per_page)` pages, not exceeding `max_page_num`; allocate the complete `max_page_num` capacity for other cache types.
4. **Create each page:** For every core, request, and cache type, call `rknn3_create_mem(ctx, page_size, core_id, RKNN3_FLAG_MEMORY_CACHEABLE)` to create an independent page. If the CPU clears or writes page contents, call `rknn3_mem_sync(ctx, page, RKNN3_MEMORY_SYNC_TO_DEVICE)`.
5. **Set the paged KV cache:** Create one `rknn3_kvcache_page_binding` for each `(core_id, cache_type, request_index)`, set `n_pages` to the page count, set `pages` to the array of `rknn3_tensor_mem*`, and call `rknn3_set_kvcache_pages(ctx, bindings, n_bindings)`. The context must be idle while this function is called. Only listed bindings are replaced; unlisted bindings remain unchanged.
6. **Unbind and destroy:** When inference is complete or the page table must be rebuilt, submit a binding with `n_pages = 0` and `pages = NULL` for each bound key to `rknn3_set_kvcache_pages`. After unbinding succeeds, call `rknn3_destroy_mem` for every page. Do not destroy page memory while it is still referenced by a page table.

Example: The following example shows creation, binding, unbinding, and destruction for one page-cache binding. In a real application, repeat this for every core, request, and cache type, and retain the page objects for the lifetime of the binding.

```
// page_attr comes from RKNN3_QUERY_KVCACHE_PAGE_INFO. Calculate page_count
// from max_ctx_len and tokens_per_page, and do not exceed max_page_num.
uint32_t page_count = (max_ctx_len + tokens_per_page - 1) / tokens_per_page;
rknn3_tensor_mem* pages[page_count];
for (uint32_t i = 0; i < page_count; ++i) {
    pages[i] = rknn3_create_mem(ctx, page_attr->page_size, core_id,
                                RKNN3_FLAG_MEMORY_CACHEABLE);
    if (!pages[i]) {
        // Free pages already created; they have not been bound yet.
        for (uint32_t j = 0; j < i; ++j)
            rknn3_destroy_mem(ctx, pages[j]);
        return RKNN3_ERR_OUT_OF_MEMORY;
    }
    memset(pages[i]->virt_addr, 0, pages[i]->size);
    rknn3_mem_sync(ctx, pages[i], RKNN3_MEMORY_SYNC_TO_DEVICE);
}

rknn3_kvcache_page_binding binding = {
    core_id, page_attr->cache_type, request_index,
    page_count, (const rknn3_tensor_mem* const*)pages, {0}
};
int ret = rknn3_set_kvcache_pages(ctx, &binding, 1);
if (ret != RKNN3_SUCCESS) {
    for (uint32_t i = 0; i < page_count; ++i)
        rknn3_destroy_mem(ctx, pages[i]);
    return ret;
}

// After inference: unbind first, then destroy page memory.
rknn3_kvcache_page_binding unbind = {
    core_id, page_attr->cache_type, request_index, 0, NULL, {0}
};
ret = rknn3_set_kvcache_pages(ctx, &unbind, 1);
if (ret == RKNN3_SUCCESS) {
    for (uint32_t i = 0; i < page_count; ++i)
        rknn3_destroy_mem(ctx, pages[i]);
}
```

> **Note:** After a successful binding, page memory remains owned by the caller, but the caller must keep it alive until unbinding. `n_pages = 0` unbinds one specified key; one call may submit multiple bindings. If validation fails, old bindings are preserved. If the page-table write fails, every key in that batch is unbound; the caller must handle the lifetime of pages in that batch according to the return value. Paged models must not use `rknn3_set_kvcache_mem` at the same time.

4.5 MatMul Matrix Multiplication

The MatMul interfaces execute matrix multiplication directly, without loading an RKNN model or creating a Session. Mathematically, A is an M×K matrix, B is a K×N matrix, and the output C is an M×N matrix. See 6.11 MatMul Types for the contiguous logical storage order.

1. Call `rknn3_init` to create the parent context.
2. Populate one or more `rknn3_matmul_desc` structures and call `rknn3_matmul_init` to register the shapes.
3. Call `rknn3_matmul_query_infos` to query the size, layout, alignment, core, and quantization requirements of A/B/C for each shape. Allocate the corresponding NPU memory with `rknn3_create_mem`.
4. Call `rknn3_matmul_build` to build the handle, then `rknn3_matmul_set_shape` to select a shape.
5. Call `rknn3_matmul_transpose` to convert logical A/B to the queried NPU layouts, then call `rknn3_mem_sync(..., RKNN3_MEMORY_SYNC_TO_DEVICE)` for each input.
6. For types requiring quantization parameters, submit them with `rknn3_matmul_set_quant`. This step and B conversion may occur in either order; both must be completed before execution.
7. Call `rknn3_matmul_run(matmul, a_mem, b_mem, c_mem)` and check the return value.
8. After success, call `rknn3_mem_sync(..., RKNN3_MEMORY_SYNC_FROM_DEVICE)` before reading C on the CPU. If contiguous logical output is needed, call `rknn3_matmul_transpose` to convert C back to NCHW.
9. For multiple shapes, follow the rules below to register and switch shapes and prepare quantization parameters and A/B/C. A/B/C must be supplied for every execution.
10. Successfully call `rknn3_matmul_destroy` first, then release the caller-allocated A/B/C with `rknn3_destroy_mem`, and finally destroy the parent context.

The application must serialize shape selection, conversion, quantization, execution, and output conversion on the same handle. `run` is synchronous; success indicates that this NPU execution has completed. On failure, C must not be treated as a valid result. If `RKNN3_ERR_MATMUL_COMPLETION_UNKNOWN` is returned, retain the handle, parent context, and A/B/C. Do not modify, reuse, or release these buffers. Release the associated resources only after the MatMul handle is successfully destroyed.

4.5.1 Multiple Shape Registration and Use Cases

Multiple shapes can be registered in one MatMul handle. Use `rknn3_matmul_set_shape` to select the shape for the current execution. Pass the `descs` array at initialization; each queried `infos[i]` corresponds to `descs[i]`, and `shape_id` is the array index. Initialize, query, and build once, then switch among the registered shapes as needed.

Only M and A_batch may differ within one handle. The type, core_mask, B_batch, K, N, group_size, B_layout, and queried B size and layout must match. Multiple shapes use ordinary weight B: set `desc.B_layout=NCHW`; the queried NPU B layout is O1I1HWI2O2. NC1HWC2 feature B is supported only for single-shape W16A16.

A typical use case shares a fixed B while A/C vary with the shape. For example, keep K/N unchanged and register different M values. Convert and synchronize B once for all shapes. After each switch, prepare A for the active shape and receive the result using the corresponding C attributes. A/C buffers may be reused; switching shapes does not require new allocations every time.

4.5.2 Multiple Shape Switching Sequence

Switching shapes does not require initialization or building again. Follow this sequence:

1. Finish the current execution and any required C synchronization, output conversion, or result saving, then call `rknn3_matmul_set_shape(matmul, shape_id)`.
2. Use `infos[shape_id].A/B/C` as the NPU attributes for this execution. A/C buffers may be allocated in advance for the maximum requirements across all shapes, but must satisfy the active shape's capacity, layout, strides, alignment, and core requirements. Conversion must still use the active shape's attributes.
3. Prepare A for the new shape. Reusing a sufficiently large buffer does not make the previous shape's A data valid for the new shape; ensure that the data is organized for the new shape and layout.
4. Reuse the fixed B that has already been converted and synchronized to the device. No further B conversion or synchronization is needed.
5. Ensure that the required quantization values have been submitted for the active shape, then execute `run`. After success, process C according to the active shape.

After selecting the first shape, convert and synchronize B to the device once. Then switch and execute as follows:

```
set_shape(shape_id) -> prepare A/C using infos[shape_id]
-> submit required quantization values for the type on first use of this shape
-> convert and synchronize A -> run(A, fixed B, C)
-> synchronize C to the CPU -> convert or save C for the active shape as needed
-> switch to the next shape
```

A/B/C must be supplied to every `run`; B must not be NULL. Synchronize C before reading it on the CPU. Reusing the same C buffer overwrites previous results, so finish any required reading, conversion, or saving first. If an input is already prepared in the queried NPU layout, its logical conversion may be omitted; CPU writes still require cache synchronization.

`set_shape` can select only shapes registered at initialization. Create another MatMul handle for an unregistered shape, another computation type, or different K/N settings.

4.5.3 Quantization Parameters for Multiple Shapes

Prepare quantization parameters according to the computation type when switching shapes. Do not call `set_quant` unconditionally before every execution.

Computation Type	Initial Submission	Handling When Switching Shapes
W16A16 / W8A8O32	Do not call `set_quant`.	Do not call `set_quant`.
W4A16 / W6A16 / W8A16	After building and selecting a shape, submit B quantization values once.	The entire handle reuses the submitted B values; no submission is needed for other registered shapes.

Computation Type	Initial Submission	Handling When Switching Shapes
W8A8 per-channel / per-layer	Submit A/B/C quantization values separately for each shape before its first execution.	No further submission is needed for a configured shape. Submit values before first using an unconfigured shape. B values must match across shapes; A/C values may differ.

You may select each shape and submit its quantization values before starting the execution loop, or submit them on first use of that shape. Switching back to a configured shape reuses its existing values.

The current interface does not allow overwriting successfully submitted quantization values. Repeated submission returns `RKNN3_ERR_MATMUL_INVALID_STATE`. Create another MatMul handle if different scales or zero points are required.

5. RKNN3 C API Description

API Functions

The RKNN3 C APIs are categorized by function as follows:

Category	Description
5.1. Device and Context Management	Device discovery, runtime context initialization and destruction.
5.2. Model Loading and Initialization	Model file loading, model initialization, and streaming weight loading.
5.3. Query Interface	Query model attributes, SDK version, memory allocation, etc.
5.4. Inference Execution	Synchronous/asynchronous inference execution and dynamic shape setting.
5.5. Memory Management	Tensor memory allocation, deallocation, synchronization, and user-defined memory setting.
5.6. Model Decryption	Model decryption key setting, must be called before model loading.
5.7. LLM Session	LLM session lifecycle management: creation, inference, pause/resume, destruction.
5.8. LLM Configuration and Callbacks	Parameter configuration and callback function type definitions for LLM sessions.
5.9. KV Cache Management	KV cache policy setting, clearing, loading, and saving.
5.10. LoRA Adapters	LoRA initialization, loading, enabling, and disabling.
5.11. Profiling and Debugging	Per-layer intermediate result dump, operator performance profiling, memory usage analysis.
5.12. Custom Operators	Custom operator plugin registration.
5.13. Image Processing	Image memory creation, destruction, and color space conversion.
5.14. Auxiliary Functions	String conversion for tensor types, quantization types, and layout types.
5.15. Structured Output (Grammar)	JSON Schema/EBNF/regex and other structured output constraints based on xgrammar.
5.16. MatMul Matrix Multiplication	Matrix shape registration, query, building, conversion, quantization, and execution.

5.1. Device and Context Management

Device discovery, runtime context initialization and destruction.

rknn3_init

This interface is used to initialize the RKNN3 runtime context and must be called before loading and running a model. context is an output parameter, and init_extend can be used to specify extended information such as device ID. After initialization, rknn3_destroy must be called to release resources when they are no longer needed. If initializing multiple times, ensure that resources are properly released to avoid memory leaks.

API	rknn3_init
Function	Initializes a new RKNN3 runtime context for running RKNN3 models on Rockchip NPU hardware.
Parameters	rknn3_context* context: Pointer to the RKNN3 context handle to be initialized.
	rknn3_init_extend* init_extend: Pointer to device-specific initialization information.
Return	int Status code: see 6.2. Status Codes.
Note	After use, please call rknn3_destroy(context) to release the context resources.

Example:

```
rknn3_context ctx = 0;
rknn3_init_extend extend = {0};
// extend.device_id = "xxx";  // Specify for multi-device, can be set to NULL for
single device
int ret = rknn3_init(&ctx, &extend);
if (ret < 0) { /* Error handling */ }
```

rknn3_destroy

This interface is used to destroy the RKNN3 runtime context and release related resources. After calling, the context handle becomes invalid and cannot be used for other operations. Destroying the same context multiple times leads to undefined behavior; ensure each context is destroyed only once.

API	rknn3_destroy
Function	Destroys the RKNN3 runtime context and releases resources.
Parameters	rknn3_context context: The RKNN3 context handle to destroy.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_destroy(ctx);
ctx = 0;
```

rknn3_find_devices

This interface is used to get the list of available RKNN3 devices on the current system. pdevs is a pointer to the device information structure. Ensure pdevs is valid before calling. A return value of 0 indicates no devices or not implemented.

API	rknn3_find_devices
Function	Gets the list of available RKNN3 devices.
Parameters	rknn3_devices* pdevs: Pointer to the structure that will receive the device list.
Return	int Status code: see 6.2. Status Codes.

API	rknn3_find_devices
Note	This function fills the provided rknn3_devices structure with information about all available RKNN3 devices on the system.

Example:

```
rknn3_devices devs;
int ret = rknn3_find_devices(&devs);
if (ret == 0 && devs.n_devices > 0) {
    printf("found %u devices, device[0]: id=%s\n", devs.n_devices,
devs.devices[0].id);
}
```

rknn3_set_input_name_alias

This interface is used to set an alias for the model input name to adapt to the Session's built-in input names.

API	rknn3_set_input_name_alias
Function	Sets an alias mapping from the model input name to the Session input name.
Parameters	rknn3_context context: RKNN3 context handle obtained via rknn3_init.
	const char* src_input_name: The actual input name in the model (e.g., "input_embedding").
	const char* dst_input_name: The Session's built-in input name (e.g., "input_embeds").
Return	int Status code: see 6.2. Status Codes.
Note	When the model input name does not match the Session's built-in input name, call this interface to set the mapping. It can be called repeatedly to update the mapping. For example: the model input name is "input_embedding", while the Session's built-in name is "input_embeds"; call this interface to set the alias.

Example:

```
// Map the model input name to the session's expected input name
rknn3_set_input_name_alias(ctx, "input_embedding", "input_embeds");
```

5.2. Model Loading and Initialization

Model file loading (host-side path/memory/device-side filesystem), model initialization, and streaming weight loading.

rknn3_load_model_from_path

This interface is used to load an RKNN3 model from the specified model file path and weight file path into an initialized context. model_path and weight_path are the paths to the model and weight files respectively; weight_path can be NULL. When weight_path is NULL, streaming weight loading mode is entered, and the weight data must be uploaded in segments via the rknn3_load_weight_chunk interface after rknn3_model_init. Ensure the context is initialized and

the paths are valid before loading. On failure, an error code is returned; on success, model initialization and inference can proceed.

API	rknn3_load_model_from_path
Function	Loads an RKNN3 model from file paths into the specified context.
Parameters	rknn3_context context: RKNN3 context handle.
	const char* model_path: Path to the RKNN3 model file.
	const char* weight_path: Path to the RKNN3 weight file; pass NULL to enter streaming weight loading mode.
Return	int Status code: see 6.2. Status Codes.
Note	When weight_path is NULL, streaming weight loading mode is entered; rknn3_load_weight_chunk must be called after rknn3_model_init to upload weight data in segments.

Example:

```
// Normal mode: specify weight file path
rknn3_load_model_from_path(ctx, "/path/to/model.rknn", "/path/to/model.weight");

// Streaming weight loading mode: pass NULL for weight_path
rknn3_load_model_from_path(ctx, "/path/to/model.rknn", NULL);
```

rknn3_load_model_from_data

This interface is used to load an RKNN3 model from memory data, suitable for scenarios where the model and weights are already in memory. model_data and weight_data are memory pointers for the model and weights respectively; model_size and weight_size are the corresponding data sizes. When weight_data is NULL or weight_size is 0, streaming weight loading mode is entered, and the weight data must be uploaded in segments via the rknn3_load_weight_chunk interface after rknn3_model_init. Ensure the context is initialized and the data pointers and sizes are valid before calling. On failure, an error code is returned; on success, model initialization and inference can proceed.

API	rknn3_load_model_from_data
Function	Loads an RKNN3 model from memory data.
Parameters	rknn3_context context: RKNN3 context handle.
	const void* model_data: Pointer to the model data in memory.
	uint64_t model_size: Size of the model data in bytes.
	const void* weight_data: Pointer to the weight data in memory; pass NULL to enter streaming weight loading mode.
	uint64_t weight_size: Size of the weight data in bytes; pass 0 to enter streaming weight loading mode.
Return	int Status code: see 6.2. Status Codes.

API	rknn3_load_model_from_data
Note	When weight_data is NULL or weight_size is 0, streaming weight loading mode is entered; rknn3_load_weight_chunk must be called after rknn3_model_init to upload weight data in segments.

Example:

```
// Load model and weights from memory
rknn3_load_model_from_data(ctx, model_data, model_size, weight_data, weight_size);

// Streaming mode: pass NULL for weight_data and 0 for weight_size
rknn3_load_model_from_data(ctx, model_data, model_size, NULL, 0);
```

rknn3_load_model_from_device

This interface is used to load an RKNN3 model from the device-side filesystem (emmc). Unlike `rknn3_load_model_from_path`, `model_path` and `weight_path` point to files on the device's (e.g., coprocessor side) filesystem, not the host's files. After the call, the host sends the paths to the device, and the device side reads and parses the model files locally.

API	rknn3_load_model_from_device
Function	Loads an RKNN3 model from the device-side filesystem (emmc) into the specified context.
Parameters	rknn3_context context: RKNN3 context handle.
	const char* model_path: Path to the model file on the device-side filesystem.
	const char* weight_path: Path to the weight file on the device-side filesystem; pass NULL to enter streaming weight loading mode.
Return	int Status code: see 6.2. Status Codes.
Note	`model_path` / `weight_path` refer to paths on the device-side filesystem; passing NULL for weight_path enters streaming weight loading mode.

Example:

```
// Load model and weights from the device-side filesystem
rknn3_load_model_from_device(ctx, "/data/model.rknn", "/data/model.weight");

// Streaming weight loading mode: pass NULL for weight_path
rknn3_load_model_from_device(ctx, "/data/model.rknn", NULL);
```

rknn3_model_init

This interface is used to initialize the runtime configuration of a loaded model, including priority, timeout, core mask, etc. config is a pointer to the configuration parameters. Ensure the model is loaded and config is valid before calling. On failure, an error code is returned; on success, the model is ready for inference.

In streaming weight loading mode (i.e., when the weight parameter is NULL during model loading), rknn3_model_init only completes model configuration; weight data is not synchronized

at this stage. After the function returns, the caller must upload weight data in segments via the rknn3_load_weight_chunk interface.

API	rknn3_model_init
Function	Initializes the RKNN3 model.
Parameters	rknn3_context context: RKNN3 context handle.
	rknn3_config* config: Model configuration parameters.
Return	int Status code: see 6.2. Status Codes.
Note	In streaming weight loading mode, this interface does not transfer weight data; weights must be uploaded in segments via rknn3_load_weight_chunk after calling.

Example:

```
rknn3_config config = {0};
config.run_core_mask = RKNN3_NPU_CORE_0;
config.run_timeout = 0;  // 0 means no timeout
int ret = rknn3_model_init(ctx, &config);
```

rknn3_load_weight_chunk

This interface is used to upload weight data in segments to the NPU device in streaming weight loading mode. The caller can write weight data segment by segment in any chunk size without maintaining a complete weight memory copy on the host side. The interface internally copies data to the correct addresses based on the weight memory allocation information of each NPU core.

Prerequisite: Must be called after rknn3_model_init returns successfully (i.e., streaming weight loading is ready). When the cumulative uploaded bytes reach the total weight size, the streaming state is automatically completed, and inference can proceed normally.

API	rknn3_load_weight_chunk
Function	Uploads a chunk of weight data to the NPU device in streaming weight loading mode.
Parameters	rknn3_context context: RKNN3 context handle.
	uint64_t weight_offset: Global byte offset of the weight data within the weight file.
	const void* data: Pointer to the weight data chunk.
	uint64_t size: Size of the data chunk in bytes.
Return	int Status code: see 6.2. Status Codes.
Note	Must be called after rknn3_model_init; weight_offset is the global offset relative to the beginning of the weight file; when the cumulative uploaded bytes reach the total weight size, the streaming state is automatically completed.

Example:

```
// Streaming weight loading: read weight file in a loop and upload in segments
FILE* fp = fopen("model.weight", "rb");
uint64_t offset = 0;
char buf[CHUNK_SIZE];
size_t n;
while ((n = fread(buf, 1, sizeof(buf), fp)) > 0) {
    rknn3_load_weight_chunk(ctx, offset, buf, n);
    offset += n;
}
fclose(fp);
```

5.3. Query Interface

Query model attributes, SDK version, memory allocation, etc.

rknn3_query

This interface is used to query various information about the model and runtime, such as input/output attributes, SDK version, etc. cmd specifies the query type; see rknn3_query_cmd for details. info is a pointer to the result buffer, and size is the buffer size. Ensure context and info are valid before calling, and size matches the corresponding structure size for the query type.

API	rknn3_query
Function	Queries RKNN3 information or status.
Parameters	rknn3_context context: The context of the RKNN3 model.
	rknn3_query_cmd cmd: The query command type.
	void* info: Pointer to the buffer for storing query results.
	uint64_t size: Size of the info buffer in bytes.
Return	int Status code: see 6.2. Status Codes.
Note	This function is used to query various information about the RKNN3 model and runtime, such as SDK version, device information, model information, etc. The specific information returned depends on the specified query command.

Example:

```
// Query the number of inputs and outputs
rknn3_input_output_num io_num;
rknn3_query(ctx, RKNN3_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));

// Query the attribute of input tensor 0
rknn3_tensor_attr attr;

memset(&attr, 0x00, sizeof(attr));
attr.index = 0;
rknn3_query(ctx, RKNN3_QUERY_INPUT_ATTR, &attr, sizeof(attr));
```

5.4. Inference Execution

Synchronous/asynchronous inference execution and dynamic shape setting.

rknn3_run

This interface is used to execute synchronous inference. inputs and outputs are the input and output tensor arrays respectively; n_inputs and n_outputs are the counts. Ensure context, inputs, and outputs are all properly allocated and configured before calling. The inference blocks until completion, and the return value is a status code. The memory for input and output tensors must be allocated by the user in advance.

API	**rknn3_run**
Function	Executes RKNN3 model inference.
Parameters	rknn3_context context: RKNN3 context handle obtained from rknn3_init.
	const rknn3_tensor inputs[]: Input tensor array containing input data.
	uint32_t n_inputs: Number of input tensors.
	rknn3_tensor outputs[]: Output tensor array for storing inference results.
	uint32_t n_outputs: Number of output tensors.
Return	int Status code: see 6.2. Status Codes.
Note	This function performs inference using the specified RKNN3 model. It receives input data through the inputs array and writes results to the outputs array. Input and output tensors must be properly allocated and configured before calling this function.

Example:

```
// Complete inference flow: query attributes -> allocate memory -> fill inputs ->
sync -> inference -> sync -> read outputs

// 1. Query the number of inputs and outputs
rknn3_input_output_num io_num;
rknn3_query(ctx, RKNN3_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));

// 2. Query each input/output tensor attribute, allocate memory by aligned_size and
core_id
rknn3_tensor inputs[io_num.n_input];
rknn3_tensor outputs[io_num.n_output];
for (uint32_t i = 0; i < io_num.n_input; i++) {
    inputs[i].attr = malloc(sizeof(rknn3_tensor_attr));
    inputs[i].attr->index = i;
    rknn3_query(ctx, RKNN3_QUERY_INPUT_ATTR, inputs[i].attr,
sizeof(rknn3_tensor_attr));
    inputs[i].mem = rknn3_create_mem(ctx, inputs[i].attr->aligned_size,
                                     inputs[i].attr->core_id,
RKNN3_FLAG_MEMORY_CACHEABLE);
}
// outputs similarly ...

// 3. Fill input data, sync to device
memcpy(inputs[0].mem->virt_addr, input_data, inputs[0].attr->aligned_size);
rknn3_mem_sync(ctx, inputs[0].mem, RKNN3_MEMORY_SYNC_TO_DEVICE);

// 4. Execute inference
rknn3_run(ctx, inputs, io_num.n_input, outputs, io_num.n_output);

// 5. Sync output to CPU, read results
rknn3_mem_sync(ctx, outputs[0].mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
// Read inference results from outputs[0].mem->virt_addr ...

// 6. Free memory
for (uint32_t i = 0; i < io_num.n_input; i++) {
    rknn3_destroy_mem(ctx, inputs[i].mem);
    free(inputs[i].attr);
}
// outputs similarly ...
```

rknn3_set_shape

This interface is used to set the current shape for a dynamic input model. shape_id is the predefined shape ID in the model. Ensure the context is valid and shape_id is legitimate before calling. Only dynamic input models need to call this; static models do not require shape setting.

API	**rknn3_set_shape**
Function	Sets the model shape for dynamic input.
Parameters	rknn3_context context: RKNN3 model context handle.
	int32_t shape_id: The ID of the shape to set.
Return	int Status code: see 6.2. Status Codes.

Example:

```
// Dynamic shape model: set the current shape ID
rknn3_set_shape(ctx, shape_id);
```

5.5. Memory Management

Tensor memory allocation, deallocation, synchronization, and user-defined memory setting.

rknn3_create_mem_from_fd

This interface allows the user to associate already-allocated memory (represented by a file descriptor) with the RKNN3 inference framework, creating a tensor memory object from the file descriptor for efficient memory reuse and zero-copy data transfer. Ensure the context is valid and the relevant parameters (such as fd, size, etc.) are correct before calling. The allocated memory must be released via rknn3_destroy_mem when no longer needed to avoid memory leaks.

API	**rknn3_create_mem_from_fd**
Function	Creates a tensor memory object from a file descriptor.
Parameters	rknn3_context context: RKNN3 context handle.
	int32_t fd: File descriptor of the memory.
	void* virt_addr: Virtual address of the memory.
	uint64_t size: Size of the memory in bytes.
	uint64_t offset: Offset from the start of the memory referenced by fd.
Return	rknn3_tensor_mem* Pointer to the created tensor memory object, or NULL if creation fails.

Example:

```
// Create tensor memory from an existing fd (zero-copy scenario)
rknn3_tensor_mem* mem = rknn3_create_mem_from_fd(ctx, fd, virt_addr, size, offset);
```

rknn3_create_mem

This interface is used to allocate or register memory for a tensor. Ensure the context is valid before calling, and the core_id of input and output tensors should be set to the core_id member of the rknn3_tensor_attr structure obtained from the rknn3_query interface. The allocated memory must be released via rknn3_destroy_mem when no longer needed to avoid memory leaks.

API	**rknn3_create_mem**
Function	Creates a memory buffer for an RKNN3 tensor.
Parameters	rknn3_context context: RKNN3 context handle.
	uint64_t size: Size of the memory to allocate in bytes.
	int32_t core_id: Target NPU core ID for memory allocation.
	rknn3_mem_alloc_flags flags: Memory allocation flags that control allocation behavior.
Return	rknn3_tensor_mem* Pointer to the allocated tensor memory, or NULL if allocation fails.

API	rknn3_create_mem
Note	This function allocates memory that can be used for RKNN3 tensor operations. Memory is allocated on the specified core with the given flags.

Example:

```
// Allocate 1MB cacheable memory on NPU core 0
rknn3_tensor_mem* mem = rknn3_create_mem(ctx, 1024 * 1024, 0,
RKNN3_FLAG_MEMORY_CACHEABLE);
```

rknn3_destroy_mem

This interface is used to release tensor memory allocated or registered through the RKNN3 API. Ensure context and mem are valid before calling. Double-freeing the same memory leads to undefined behavior.

API	rknn3_destroy_mem
Function	Destroys memory allocated for an RKNN3 tensor.
Parameters	rknn3_context context: RKNN3 context handle.
	rknn3_tensor_mem* mem: Pointer to the tensor memory structure to destroy.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_destroy_mem(ctx, mem);
mem = NULL;
```

rknn3_mem_sync

This interface is used to synchronize memory data between the CPU and the device. mode specifies the synchronization direction, commonly used before and after inference to ensure data consistency. Ensure context and mem are valid and mode is set correctly before calling. Failure to synchronize may cause abnormal inference results.

API	rknn3_mem_sync
Function	Synchronizes memory data between the CPU and the device.
Parameters	rknn3_context context: The context of the RKNN3 model.
	rknn3_tensor_mem* mem: Memory handle of the tensor.

API	rknn3_mem_sync
	rknn3_mem_sync_mode mode: The mode for flushing CPU cache and DDR data. **RKNN3_MEMORY_SYNC_TO_DEVICE**: Synchronizes CPU cache data to DDR. Typically used after CPU writes to memory and before the NPU accesses the same memory, to write back cache data to DDR. **RKNN3_MEMORY_SYNC_FROM_DEVICE**: Synchronizes DDR data to CPU cache. Typically used after the NPU writes to memory, to invalidate the cache so that the next CPU access to the same memory reads data from DDR. In FROM_DEVICE mode, data is transferred in blocks; the block size can be configured via the environment variable `MEM_SYNC_CHUNK_SIZE` (unit: bytes, default 2MiB). **RKNN3_MEMORY_SYNC_BIDIRECTIONAL**: Synchronizes CPU cache data to DDR and also causes the CPU to re-read data from DDR.
Return	int Status code: see 6.2. Status Codes.

Example:

```
// After CPU writes, sync to device (before NPU reads)
rknn3_mem_sync(ctx, input_mem, RKNN3_MEMORY_SYNC_TO_DEVICE);

// After NPU writes, sync to CPU (before CPU reads)
rknn3_mem_sync(ctx, output_mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
```

rknn3_mem_sync_range

This interface is used to synchronize a specified range of memory data between the CPU and the device. Unlike `rknn3_mem_sync` which synchronizes the entire memory, this interface only synchronizes data in the `[offset, offset+length)` range, suitable for scenarios where only partial memory needs to be updated, reducing synchronization overhead. `offset` is the byte offset relative to `mem->virt_addr`, and `length` is the number of bytes to synchronize; `offset + length` must not exceed `mem->size`.

API	rknn3_mem_sync_range
Function	Synchronizes a specified range of memory data between the CPU and the device.
Parameters	rknn3_context context: The context of the RKNN3 model.
	rknn3_tensor_mem* mem: Memory handle of the tensor.
	rknn3_mem_sync_mode mode: Sync mode, same values as the mode parameter in rknn3_mem_sync.
	uint64_t offset: Starting byte offset relative to `mem->virt_addr`.
	uint64_t length: Number of bytes to synchronize (`offset + length` must not exceed `mem->size`).
Return	int Status code: 0 for success, negative value for error; see 6.2. Status Codes.
Note	Returns an error code when `offset + length` exceeds `mem->size`, `length` is 0, or `offset` is greater than or equal to `mem->size`.

Example:

```
// Sync only the [offset, offset+length) range to device
rknn3_mem_sync_range(ctx, mem, RKNN3_MEMORY_SYNC_TO_DEVICE, 1024, 4096);

// Sync only 4096 bytes from device to CPU
rknn3_mem_sync_range(ctx, mem, RKNN3_MEMORY_SYNC_FROM_DEVICE, 0, 4096);
```

rknn3_set_kvcache_pages

Sets page-table bindings for a paged model. Each binding consists of a physical core, cache type, request index, and an array of page memory objects. Only the bindings listed in the call are changed; unlisted bindings remain unchanged. Before calling, query page sizes and cache types with `RKNN3_QUERY_KVCACHE_PAGE_INFO`, then allocate one `page_size`-byte memory object per page with `rknn3_create_mem`.

API	**rknn3_set_kvcache_pages**
Function	Sets or updates page-table bindings for a paged KV cache.
Parameters	rknn3_context context: RKNN3 context handle.
	const rknn3_kvcache_page_binding* bindings: Array of page-table bindings.
	uint32_t n_bindings: Number of bindings; must be greater than 0.
Return	int Status code: see 6.2. Status Codes.
Note	Only valid for paged KV cache models; the context must be idle. `n_pages = 0` with `pages = NULL` unbinds one `(core_id, cache_type, request_index)` key. The API does not take ownership of page memory; bound pages must remain valid until unbound.

Example:

```
// First query page properties for the target core; page_attr is info.caches[type].
rknn3_kvcache_page_info info = {0};
info.core_id = core_id;
int ret = rknn3_query(ctx, RKNN3_QUERY_KVCACHE_PAGE_INFO,
                      &info, sizeof(info));
if (ret != RKNN3_SUCCESS || type >= info.n_cache_types)
    return RKNN3_ERR_ARGUMENT_INVALID;
rknn3_cache_page_attr* page_attr = &info.caches[type];

// Calculate page_count from the model configuration and do not exceed
// page_attr->max_page_num.
uint32_t page_count = (max_ctx_len + tokens_per_page - 1) / tokens_per_page;
rknn3_tensor_mem* page_mems[page_count];
for (uint32_t i = 0; i < page_count; ++i) {
    page_mems[i] = rknn3_create_mem(ctx, page_attr->page_size, core_id,
                                    RKNN3_FLAG_MEMORY_CACHEABLE);
}

rknn3_kvcache_page_binding binding = {
    core_id, page_attr->cache_type, request_index,
    page_count, (const rknn3_tensor_mem* const*)page_mems, {0}
};
ret = rknn3_set_kvcache_pages(ctx, &binding, 1);

// When this request no longer uses the paged KV cache, unbind first,
// then destroy the page memory.
rknn3_kvcache_page_binding unbind = {
    core_id, page_attr->cache_type, request_index, 0, NULL, {0}
};
ret = rknn3_set_kvcache_pages(ctx, &unbind, 1);
if (ret == RKNN3_SUCCESS) {
    for (uint32_t i = 0; i < page_count; ++i)
        rknn3_destroy_mem(ctx, page_mems[i]);
}
```

> **Error handling:** If validation fails, old bindings are preserved. If the page-table write fails, keys in the batch are unbound. Keep all page objects alive and destroy them only after confirming that the corresponding pages are no longer referenced by a page table. Do not call `rknn3_set_kvcache_mem` for a paged model.

rknn3_set_kvcache_mem

This interface is used to set KV cache memory for the specified NPU core. mem is the KV cache memory array, npu_core_indices is the core index array, and n_core is the number of cores. Ensure context, mem, and npu_core_indices are valid and n_core is greater than 0 before calling. mem and npu_core_indices must correspond one-to-one.

API	**rknn3_set_kvcache_mem**
Function	Sets KV cache memory.
Parameters	rknn3_context context: RKNN3 model context handle.
	rknn3_tensor_mem* mem[]: KV cache memory related information.
	int* npu_core_indices: NPU core indices.

API	rknn3_set_kvcache_mem
	int n_core: Number of NPU cores to set kvcache for.
Return	int Status code: see 6.2. Status Codes.

Example:

```
// KV cache memory source: query each core's required size via
RKNN3_QUERY_ALLOCATION_INFO,
// allocate with rknn3_create_mem, then set with rknn3_set_kvcache_mem.
// Prerequisite: called after rknn3_model_init.

// 1. Query the number of cores
uint32_t core_num = 0;
rknn3_query(ctx, RKNN3_QUERY_CORE_NUMBER, &core_num, sizeof(core_num));

// 2. Query each core's memory allocation info (including kvcache size)
rknn3_allocation_info alloc_infos[core_num];
rknn3_query(ctx, RKNN3_QUERY_ALLOCATION_INFO, alloc_infos,
            sizeof(rknn3_allocation_info) * core_num);

// 3. Allocate memory based on each core's kvcache_mem.size
rknn3_tensor_mem* kvcache_mems[core_num];
int core_indices[core_num];
int n = 0;
for (uint32_t i = 0; i < core_num; i++) {
    uint64_t size = alloc_infos[i].kvcache_mem.size;
    if (size == 0) continue;
    kvcache_mems[n] = rknn3_create_mem(ctx, size, alloc_infos[i].core_id,
                                       RKNN3_FLAG_MEMORY_CACHEABLE);
    core_indices[n] = alloc_infos[i].core_id;
    n++;
}

// 4. Set KV cache memory
rknn3_set_kvcache_mem(ctx, kvcache_mems, core_indices, n);

// Free after use
for (int i = 0; i < n; i++)
    rknn3_destroy_mem(ctx, kvcache_mems[i]);
```

rknn3_set_internal_mem

Sets user-allocated internal tensor memory for multiple cores. Each mem's core_id field specifies the target core.

API	rknn3_set_internal_mem
Function	Sets user-allocated internal tensor memory for multiple cores.
Parameters	rknn3_context context: RKNN3 context.
	rknn3_tensor_mem* mem[]: Array of user-allocated memory objects.
	uint32_t n_core: Number of cores.
Return	int Status code: see 6.2. Status Codes.

Example:

```
// Internal memory source: query each core's required size via
RKNN3_QUERY_CORE_MEM_SIZE,
// allocate with rknn3_create_mem, then set with rknn3_set_internal_mem.
// Note: config.user_mem_internal must be set to 1 before rknn3_model_init.

// 1. Mark the use of user-allocated internal memory during model initialization
rknn3_config config = {0};
config.user_mem_internal = 1;
rknn3_model_init(ctx, &config);

// 2. Query the number of cores
uint32_t core_num = 0;
rknn3_query(ctx, RKNN3_QUERY_CORE_NUMBER, &core_num, sizeof(core_num));

// 3. Query each core's internal memory size
rknn3_core_mem_size core_mem_sizes[core_num];
rknn3_query(ctx, RKNN3_QUERY_CORE_MEM_SIZE, core_mem_sizes,
            sizeof(rknn3_core_mem_size) * core_num);

// 4. Allocate memory based on each core's internal_size
rknn3_tensor_mem* internal_mems[core_num];
for (uint32_t i = 0; i < core_num; i++) {
    if (core_mem_sizes[i].internal_size > 0) {
        internal_mems[i] = rknn3_create_mem(ctx, core_mem_sizes[i].internal_size,
                                            core_mem_sizes[i].core_id,
                                            RKNN3_FLAG_MEMORY_CACHEABLE);
    } else {
        internal_mems[i] = NULL;
    }
}

// 5. Set user-allocated internal memory
rknn3_set_internal_mem(ctx, internal_mems, core_num);

// Free after use
for (uint32_t i = 0; i < core_num; i++)
    if (internal_mems[i]) rknn3_destroy_mem(ctx, internal_mems[i]);
```

rknn3_set_weight_mem

Sets user-allocated weight memory for multiple cores. Each mem's core_id field specifies the target core. Note: The current version only supports RK3572.

API	**rknn3_set_weight_mem**
Function	Sets user-allocated weight tensor memory for multiple cores.
Parameters	rknn3_context context: RKNN3 context handle.
	rknn3_tensor_mem* mem[]: Array of user-allocated memory objects; each mem's core_id field specifies the target core.
	uint32_t n_core: Number of cores.
Return	int Status code: see 6.2. Status Codes.
Note	The current version only supports RK3572.

Example:

```
// Weight memory source: query each core's weight size via
RKNN3_QUERY_CORE_MEM_SIZE,
// allocate with rknn3_create_mem, then set with rknn3_set_weight_mem.
// Note: config.user_mem_weight must be set to 1 before rknn3_model_init.
//       The current version only supports RK3572.

// 1. Mark the use of user-allocated weight memory during model initialization
rknn3_config config = {0};
config.user_mem_weight = 1;
rknn3_model_init(ctx, &config);

// 2. Query the number of cores and each core's weight memory size
uint32_t core_num = 0;
rknn3_query(ctx, RKNN3_QUERY_CORE_NUMBER, &core_num, sizeof(core_num));
rknn3_core_mem_size core_mem_sizes[core_num];
rknn3_query(ctx, RKNN3_QUERY_CORE_MEM_SIZE, core_mem_sizes,
            sizeof(rknn3_core_mem_size) * core_num);

// 3. Allocate memory based on each core's weight_size
rknn3_tensor_mem* weight_mems[core_num];
for (uint32_t i = 0; i < core_num; i++) {
    weight_mems[i] = rknn3_create_mem(ctx, core_mem_sizes[i].weight_size,
                                      core_mem_sizes[i].core_id,
                                      RKNN3_FLAG_MEMORY_CACHEABLE);
}

// 4. Set user-allocated weight memory
rknn3_set_weight_mem(ctx, weight_mems, core_num);

// Free after use
for (uint32_t i = 0; i < core_num; i++)
    rknn3_destroy_mem(ctx, weight_mems[i]);
```

rknn3_set_kvcache_group

Sets the active KV cache group for the current context. Each group corresponds to an entry in the `kvcache_buffer_len` list configured when converting the model with RKNN3 Toolkit. Each group corresponds to a different KV cache buffer size, i.e., a different maximum context length (max context len). Switching groups is equivalent to switching max context len.

The number of groups (`n_groups`) and the KV cache size of each group (`kvcache_sizes`) for each core can be queried via `RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO` . `group_id` must be in the range `[0, n_groups)` . After calling, the internal KV cache buffer address is updated to the corresponding group's address, and the current context's execution ID is switched.

API	**rknn3_set_kvcache_group**
Function	Sets the active KV cache group for the current context, equivalent to switching max context len.
Parameters	rknn3_context context: RKNN3 context handle.
	int32_t group_id: The group ID to activate (must be in [0, n_groups) range).
Return	int Status code: see 6.2. Status Codes.

API	rknn3_set_kvcache_group
Note	Groups correspond one-to-one with the `kvcache_buffer_len` list set during Toolkit conversion; if using externally allocated KV cache memory, ensure the memory size meets the target group's requirements after switching groups, and reallocate and call `rknn3_set_kvcache_mem` if necessary.

Example:

```
// After querying group info, switch to group 1 (equivalent to switching max context
len)
rknn3_kvcache_len_group_info info[core_number];
rknn3_query(ctx, RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO, info,
            sizeof(rknn3_kvcache_len_group_info) * core_number);
rknn3_set_kvcache_group(ctx, 1);  // Switch to group_id=1
```

5.6. Model Decryption

The RKNN SDK supports encrypting RKNN models offline and decrypting them at runtime. Model decryption key setting must be called before model loading.

> Model encryption is only one part of model protection and cannot completely eliminate the risk of the model being cracked. Users are advised to prevent third parties from directly accessing the model and implement overall protection at the system level.

rknn3_set_decrypt_key_from_path

This interface is used to load an RSA-encrypted user key envelope from a file path and associate it with the context. To protect the intellectual property of user models, the RKNPU3 runtime provides a model encryption and decryption mechanism. The encryption scheme uses AES-256-CTR symmetric encryption for encrypting the model file body, and RSA-OAEP asymmetric encryption for protecting the AES key. Ensure the context is initialized and key_path points to a valid key envelope file before calling.

API	rknn3_set_decrypt_key_from_path
Function	Loads an RSA-encrypted user key envelope from a file path and associates it with the context.
Parameters	rknn3_context context: RKNN3 runtime context.
	const char* key_path: Path to the key envelope file.
Return	int Status code: see 6.2. Status Codes.
Note	This interface must be called before `rknn3_load_model_from_path` / `rknn3_load_model_from_data`.

Example:

```
// Must be called before rknn3_load_model_from_path
rknn3_set_decrypt_key_from_path(ctx, "/path/to/key_envelope.bin");
rknn3_load_model_from_path(ctx, "/path/to/model.rknn", "/path/to/model.weight");
```

rknn3_set_decrypt_key_from_data

This interface is used to load an RSA-encrypted user key envelope from memory and associate it with the context. Used for scenarios where model data comes from memory transfer. Ensure `context` is initialized, `key_data` points to valid key envelope memory, and `key_size` is the byte length of the data before calling.

API	rknn3_set_decrypt_key_from_data
Function	Loads an RSA-encrypted user key envelope from memory and associates it with the context.
Parameters	rknn3_context context: RKNN3 runtime context.
	const void* key_data: Memory address of the key envelope.
	uint64_t key_size: Length of the key envelope data in bytes.
Return	int Status code: see 6.2. Status Codes.
Note	This interface must be called before `rknn3_load_model_from_path` / `rknn3_load_model_from_data`.

Example:

```
// Load key envelope from memory, must be called before loading the model
rknn3_set_decrypt_key_from_data(ctx, key_data, key_size);
```

5.7. LLM Session

LLM session lifecycle management: creation, inference, pause/resume, destruction.

rknn3_session_init

This interface is used to initialize a new RKNN3 session, supporting advanced features such as LLM. context is the context, params is the session parameters, and n_params is the number of parameters. Ensure context and params are valid before calling. On success, a session handle is returned; on failure, NULL is returned. The session must be released via rknn3_session_destroy.

API	rknn3_session_init
Function	Initializes a new RKNN3 session with the specified parameters.
Parameters	rknn3_context context: RKNN3 context pointer for the session.
	rknn3_llm_param* params: Pointer to the rknn3_llm_param structure containing session configuration parameters.
	int n_params: Number of parameters.
Return	rknn3_session* Session handle on success, NULL on initialization failure.

Example:

```
rknn3_llm_param params[1] = {0};
params[0].max_context_len = 512;
params[0].sampling_param.top_k = 40;
params[0].sampling_param.top_p = 0.9;
// ... configure vocab_info etc. ...
rknn3_session* session = rknn3_session_init(ctx, params, 1);
```

rknn3_session_destroy

This interface is used to destroy an RKNN3 session and release related resources. session is the session pointer. After calling, the session pointer becomes invalid and cannot be used for other operations. Ensure each session is destroyed only once to avoid resource leaks or double-free.

API	rknn3_session_destroy
Function	Destroys an RKNN3 session and releases related resources.
Parameters	rknn3_session* session: The RKNN3 session pointer to destroy.
Return	int Status code: see 6.2. Status Codes.
Note	After calling this function, the session pointer becomes invalid and should not be used again.

Example:

```
rknn3_session_destroy(session);
session = NULL;
```

rknn3_session_run

Runs inference with the provided inputs and parameters. Ensure session, inputs, and param are valid before calling.

API	rknn3_session_run
Function	Runs inference with the provided inputs and parameters.
Parameters	rknn3_session* session: RKNN3 session handle pointer.
	rknn3_llm_input inputs[]: Input tensor array containing input data.
	uint32_t n_inputs: Number of input tensors provided.
	rknn3_llm_infer_param* param: Pointer to inference parameter configuration.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_llm_input input = {0};
input.input_type = RKNN3_LLM_INPUT_PROMPT;
input.llm_input.prompt = "Hello, please introduce yourself.";

rknn3_llm_infer_param param = {0};
param.keep_history = 1;
param.max_new_tokens = 128;

rknn3_session_run(session, &input, 1, &param);
```

rknn3_session_run_async

Asynchronously runs inference for a large language model session. Ensure session, inputs, and param are valid before calling. Asynchronous inference requires callbacks or wait mechanisms to obtain results.

API	rknn3_session_run_async
Function	Asynchronously runs inference for a large language model session.
Parameters	rknn3_session* session: RKNN3 session handle pointer.
	rknn3_llm_input inputs[]: Input tensor array for the model.
	uint32_t n_inputs: Number of input tensors.
	rknn3_llm_infer_param* param: Pointer to inference parameter configuration.
Return	int Status code: see 6.2. Status Codes.
Note	This function performs asynchronous inference for the given LLM session. It allows non-blocking execution of the model with the provided inputs and parameters.

Example:

```
// Asynchronous inference, results obtained via callback
rknn3_session_run_async(session, &input, 1, &param);
// Results are returned asynchronously in result_callback
```

rknn3_session_stop

Terminates the current session inference. After calling, only the current inference is terminated; new inference can be initiated subsequently. Suitable for scenarios that need to interrupt long-running inference.

API	rknn3_session_stop
Function	Terminates the current session inference.
Parameters	rknn3_session* session: RKNN3 session pointer.
Return	int Status code: see 6.2. Status Codes.

Example:

```
// Interrupt current inference (new inference can be initiated afterwards)
rknn3_session_stop(session);
```

rknn3_session_pause

Pauses the execution of an RKNN3 session. After calling, the current inference is paused and can be resumed via `rknn3_session_resume`.

API	rknn3_session_pause
Function	Pauses the execution of an RKNN3 session.
Parameters	rknn3_session* session: RKNN3 session pointer to pause.
Return	int Status code: 0 for success, negative value for error; see 6.2. Status Codes.

Example:

```
rknn3_session_pause(session);
// Inference paused, can be resumed via rknn3_session_resume
```

rknn3_session_resume

Resumes the execution of a paused RKNN3 session.

API	rknn3_session_resume
Function	Resumes the execution of an RKNN3 session.
Parameters	rknn3_session* session: RKNN3 session pointer to resume.
Return	int Status code: 0 for success, negative value for error; see 6.2. Status Codes.

Example:

```
rknn3_session_resume(session);
// Resumes inference from where it was paused
```

rknn3_session_query_state

This interface is used to query the current running state of the session, including the number of generated tokens, KV policy, etc. session is the session pointer, and state is a pointer to the state structure. Ensure session and state are valid before calling. Can be used to monitor inference progress and session state.

API	rknn3_session_query_state
Function	Queries the current state of an RKNN3 session.
Parameters	rknn3_session* session: RKNN3 session pointer to query.
	RKLLMRunState* state: Pointer to store the queried running state.
Return	int Status code: see 6.2. Status Codes.

Example:

```
RKLLMRunState state;
rknn3_session_query_state(session, &state);
printf("total=%llu, prefill=%llu, decode=%llu\n",
       state.n_total_tokens, state.n_prefill_tokens, state.n_decode_tokens);
```

5.8. LLM Configuration and Callbacks

Parameter configuration and callback function type definitions for LLM sessions.

rknn3_session_set_callback

This interface is used to set callback functions for a session, supporting custom result handling, sampling, tokenization, embedding, output tensor callbacks, etc. session is the session pointer, and callback is a pointer to the callback structure. Ensure session and callback are valid before calling. Callback functions are triggered during inference to allow users to customize the inference flow.

API	**rknn3_session_set_callback**
Function	Sets callback functions for an RKNN3 session.
Parameters	rknn3_session* session: RKNN3 session instance pointer.
	RKLLMCallback* callback: Pointer to the RKLLMCallback structure containing callback functions.
Return	int Status code: see 6.2. Status Codes.

Example:

```
RKLLMCallback callback = {0};
callback.result_callback = my_result_callback;
callback.result_userdata = tokenizer;
callback.sampling_callback = my_sampling_callback;
callback.sampling_userdata = &embed_info;
rknn3_session_set_callback(session, &callback);
```

rknn3_session_set_chat_template

This interface is used to set the LLM chat template, including system prompt, prefix, and suffix. session is the session handle; system_prompt, prompt_prefix, and prompt_postfix are the template contents. Ensure the session is valid and the string contents meet the model's requirements before calling. Custom templates help adjust model behavior and conversation style.

API	**rknn3_session_set_chat_template**
Function	Sets the LLM chat template, including system prompt, prefix, and suffix.
Parameters	rknn3_session* session: RKNN3 session handle.
	const char* system_prompt: System prompt defining the language model's context or behavior.

API	rknn3_session_set_chat_template
	const char* prompt_prefix: Prefix added before user input in the conversation.
	const char* prompt_postfix: Postfix added after user input in the conversation.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_session_set_chat_template(session,
    "You are a helpful assistant.",  // system_prompt
    "User: ",                         // prompt_prefix
    "\nAssistant: "                  // prompt_postfix
);
```

rknn3_session_set_llm_param

This interface is used to set LLM-related parameters (such as sampling parameters, vocabulary info, etc.) for an RKNN3 session. Ensure session and params pointers, and the number of parameters are valid.

API	rknn3_session_set_llm_param
Function	Configures LLM-related parameters (such as sampling parameters, vocabulary info, etc.), supporting updating LLM-related parameters after Session initialization.
Parameters	rknn3_session* session: RKNN3 session handle.
	rknn3_llm_param* params: Pointer to LLM parameter array (one LLM parameter per logits output).
	int n_params: Number of LLM parameters (equals the number of logits outputs).
Return	int Status code: see 6.2. Status Codes.
Note	params must be set in output order; if the model has multiple logits outputs, they must be configured accordingly.

Example:

```
// Update sampling parameters after session initialization
rknn3_llm_param params[1] = {0};
params[0].sampling_param.temperature = 0.8;
params[0].sampling_param.top_k = 50;
rknn3_session_set_llm_param(session, params, 1);
```

rknn3_session_set_function_tools

This interface is used to set function tool descriptions (function tools) for a session, enabling the model to call tools during conversations. Ensure session and tools string are valid before calling.

API	rknn3_session_set_function_tools
Function	Sets function tool descriptions for an LLM session.
Parameters	rknn3_session* session: RKNN3 session handle.

API	rknn3_session_set_function_tools
	const char* tools: Function tool description string.
Return	int Status code: see 6.2. Status Codes.

Example:

```
const char* tools = "[{\"name\":\"get_weather\",\"description\":\"Query
weather\"}]";
rknn3_session_set_function_tools(session, tools);
```

LLMResultCallback

LLM result callback function type.

Function Pointer Type	LLMResultCallback
Function	Callback function for handling LLM generation results.
Parameters	void* userdata: User-defined data pointer.
	RKLLMResult* result: Pointer to the LLM result.
	LLMCallState state: LLM call state (e.g., finished, error, etc.).
Return	int Status code: see 6.2. Status Codes.

Example:

```
int my_result_callback(void* userdata, RKLLMResult* result, LLMCallState state) {
    if (state == RKLLM_RUN_NORMAL) {
        for (int i = 0; i < result->num_tokens; i++)
            printf("token[%d] = %d\n", i, result->token_ids[i]);
    } else if (state == RKLLM_RUN_FINISH) {
        printf("generation finished\n");
    }
    return 0;
}
```

LLMSamplingCallback

LLM sampling callback function type.

Function Pointer Type	LLMSamplingCallback
Function	Callback function for handling sampling logits.
Parameters	void* userdata: User-defined data pointer.
	float16* logits: Pointer to the logits array.
	char* logits_name: Name of the logits.
Return	int: Returns the selected token ID (>=0) on success, negative value for error code; see 6.2. Status Codes.

Example:

```
int my_sampling_callback(void* userdata, float16* logits, char* logits_name) {
    // Custom sampling: simple argmax
    int max_id = 0;
    for (int i = 1; i < vocab_size; i++) {
        if (fp16_to_fp32(logits[i]) > fp16_to_fp32(logits[max_id]))
            max_id = i;
    }
    return max_id;  // Return the selected token ID
}
```

LLMGetEmbedCallback

Callback function type for getting LLM embedding vectors.

Function Pointer Type	LLMGetEmbedCallback
Function	Callback function for getting LLM embedding vectors.
Parameters	void* userdata: User-defined data pointer.
	int32_t* tokens: Token ID array.
	uint64_t num_tokens: Number of tokens in the tokens array.
	void* embed: Buffer pointer for storing embedding output.
	uint64_t len: Length of the embedding buffer in bytes.
Return	int Status code: see 6.2. Status Codes.

Example:

```
int my_embed_callback(void* userdata, int32_t* tokens, uint64_t num_tokens,
                      void* embed, uint64_t len) {
    // Copy embedding results to the embed buffer
    memcpy(embed, my_embed_data, len);
    return 0;
}
```

LLMTokenizerCallback

LLM tokenizer callback function type.

Function Pointer Type	LLMTokenizerCallback
Function	Callback function for handling text tokenization.
Parameters	void* userdata: User-defined data pointer.
	const char* text: Input text string pointer.
	int32_t text_len: Text length.
	int32_t* tokens: Token ID array pointer.
	int32_t n_tokens_max: Maximum number of tokens to generate.

Function Pointer Type	LLMTokenizerCallback
Return	int: Returns the number of generated tokens (>=0) on success, negative value for error code; see 6.2. Status Codes.

Example:

```
int my_tokenizer_callback(void* userdata, const char* text, int32_t text_len,
                          int32_t* tokens, int32_t n_tokens_max) {
    // Call custom tokenizer to convert text to token ID array
    int n = my_tokenize(text, text_len, tokens, n_tokens_max);
    return n;  // Return the number of generated tokens
}
```

LLMOutputCallback

Callback function type for getting output tensors.

Function Pointer Type	LLMOutputCallback
Function	Callback function for getting output tensors.
Parameters	void* userdata: User-defined data pointer.
	rknn3_tensor* output_tensors: Output tensor array pointer.
	uint32_t n_output_tensors: Number of output tensors.
	LLMOutputCallbackState state: Output callback state.
Return	int: Returns 0 on success, non-zero on error; see 6.2. Status Codes.

Example:

```
int my_output_callback(void* userdata, rknn3_tensor* output_tensors,
                       uint32_t n_output_tensors, LLMOutputCallbackState state) {
    if (state == RKLLM_OUTPUT_CALLBACK_DECODE_FINISHED) {
        for (uint32_t i = 0; i < n_output_tensors; i++) {
            float16* data = (float16*)output_tensors[i].mem->virt_addr;
            // Process output tensor data ...
        }
    }
    return 0;
}
```

LLMInputCallback

Callback function type for providing custom input tensors, suitable for scenarios that require custom processing of inputs for each layer (e.g., gemma4's per_layer_inputs). When called, the framework passes the current inference's input tensor array and context parameters (`LLMInputCallbackParam`) to the callback, and the user fills in the corresponding tensor data in the callback.

LLMInputCallbackParam parameter structure:

Field	Type	Description
tokens	int32_t*	Pointer to the current input token ID array
num_tokens	int32_t	Number of tokens
shape_id	int32_t	Shape ID of the current input tensor
pos	int32_t	Position of the current input within the entire context
mrope_pos	int32_t	MRoPE position
mrope_start	int32_t	Starting position of MRoPE within the entire context
system_prompt_seqlen	int32_t	Sequence length of the system prompt
valid_system_prompt_seqlen	int32_t	Valid sequence length of the system prompt
max_position_embeddings	int32_t	Maximum position embeddings supported by the model
reserved	uint8_t[128]	Reserved field for future expansion

Function Pointer Type	LLMInputCallback
Function	Callback function for providing custom input tensor data to the inference framework.
Parameters	void* userdata: User-defined data pointer.
	rknn3_tensor* input_tensors: Input tensor array pointer, allocated by the framework; the user fills in data in the callback.
	uint32_t n_input_tensors: Number of input tensors.
	LLMInputCallbackParam param: Context parameters for the current inference step, including token info, position info, etc.
Return	int: Returns 0 on success, non-zero on error; see 6.2. Status Codes.

Example:

```
int my_input_callback(void* userdata, rknn3_tensor* input_tensors,
                      uint32_t n_input_tensors, LLMInputCallbackParam param) {
    // Fill custom input tensors based on param.pos and other info
    for (uint32_t i = 0; i < n_input_tensors; i++) {
        // input_tensors[i].mem->virt_addr = ...;
    }
    return 0;
}
```

5.9. KV Cache Management

KV cache policy setting, clearing, loading, and saving.

rknn3_session_set_kvcache_policy

Sets the KV cache policy for an RKNN3 session. Applicable to models that require a caching mechanism, such as Transformers. Ensure the session is valid and the relevant parameters are correct before calling. KV cache operations must be used in coordination with the inference flow to avoid cache inconsistency.

API	rknn3_session_set_kvcache_policy
Function	Sets the KV cache policy for an RKNN3 session.
Parameters	rknn3_session* session: RKNN3 session handle.
	rknn3_kvcache_policy policy: The KV cache policy to set.
	rknn3_kvcache_policy_param* param: Parameters specifying the KV cache policy.
Return	int Status code: see 6.2. Status Codes.

Example:

```
// Use checkpoint policy
rknn3_kvcache_policy_param param = {0};
param.save_checkpoint.checkpoint_start_pos = 0;
param.save_checkpoint.checkpoint_interval = 128;
param.save_checkpoint.max_checkpoint_count = 10;
rknn3_session_set_kvcache_policy(session, RKNN3_KVCACHE_POLICY_SAVE_CHECKPOINT,
&param);
```

rknn3_session_clear_kvcache

Clears the KV cache of an RKNN3 session according to the specified policy.

API	rknn3_session_clear_kvcache
Function	Clears the KV cache of an RKNN3 session according to the specified policy.
Parameters	rknn3_session* session: RKNN3 session handle pointer.
	rknn3_kvcache_clear_policy clear: The policy for clearing KV cache, defining how to clear the cache.
	rknn3_kvcache_clear_policy_param* param: Parameters for the KV cache clear policy. Required when clear is RKNN3_KVCACHE_KEEP_N_TOKENS.
Return	int Status code: see 6.2. Status Codes.

Example:

```
// Clear KV cache but keep system prompt
rknn3_session_clear_kvcache(session, RKNN3_KVCACHE_KEEP_SYSTEM_PROMPT, nullptr);

// Clear KV cache but keep n_tokens
clear_param.keep_n_tokens.n_tokens = 512;
rknn3_session_clear_kvcache(session, RKNN3_KVCACHE_KEEP_N_TOKENS, &clear_param);
```

rknn3_session_load_kvcache_from_path

Loads KV cache from the specified file path.

API	rknn3_session_load_kvcache_from_path
Function	Loads a KV cache file from the specified path into an RKNN3 session.
Parameters	rknn3_session* session: RKNN3 session pointer.

API	rknn3_session_load_kvcache_from_path
	const char* kvcache_path: Path to the KV cache file.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_session_load_kvcache_from_path(session, "/path/to/kvcache.bin");
```

rknn3_session_load_kvcache_from_data

Loads KV cache from memory data.

API	rknn3_session_load_kvcache_from_data
Function	Loads KV cache from memory data into an RKNN3 session.
Parameters	rknn3_session* session: RKNN3 session pointer.
	const void* kvcache_data: KV cache binary data.
	int64_t size: KV cache data length in bytes.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_session_load_kvcache_from_data(session, kvcache_data, kvcache_size);
```

rknn3_session_save_kvcache

Saves the KV cache to the specified path.

API	rknn3_session_save_kvcache
Function	Saves the KV cache to the specified path.
Parameters	rknn3_session* session: RKNN3 session pointer.
	const char* kvcache_path: Path to save the KV cache.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_session_save_kvcache(session, "/path/to/kvcache.bin");
```

5.10. LoRA Adapters

LoRA initialization, loading, enabling, and disabling.

rknn3_lora_init

Registers a LoRA weight file with an RKNN3 context and loads metadata. This interface must be called after `rknn3_model_init` and before the session uses LoRA. This function may be called repeatedly to register multiple weight files; adapter names must be unique across all registered files. After calling, loaded LoRA information can be queried via `RKNN3_QUERY_LORA_NUM` and `RKNN3_QUERY_LORA_INFO`, and then the specified adapter can be loaded via `rknn3_lora_load`.

API	rknn3_lora_init
Function	Registers a LoRA weight file with an RKNN3 context and loads metadata. May be called repeatedly to register multiple weight files; adapter names must be unique across all registered files.
Parameters	rknn3_context context: RKNN3 context handle.
	const char* lora_weight_path: LoRA weight file path.
Return	int Status code: see 6.2. Status Codes.

Example:

```
// Load LoRA metadata from file (must be called after model_init)
rknn3_lora_init(ctx, "/path/to/lora_weights.bin");

// Query loaded LoRA info
rknn3_lora loras[RKNN3_MAX_LORA_NUM];
rknn3_query(ctx, RKNN3_QUERY_LORA_INFO, loras, sizeof(loras));
```

rknn3_lora_init_from_data

Registers an in-memory LoRA weight source with an RKNN3 context and loads metadata. Suitable for scenarios where LoRA weights are already in memory. This function may be called repeatedly to register multiple weight buffers; each buffer must remain valid until the RKNN3 context is released. Usage is the same as `rknn3_lora_init`.

API	rknn3_lora_init_from_data
Function	Registers an in-memory LoRA weight source with an RKNN3 context and loads metadata. May be called repeatedly to register multiple weight buffers; each buffer must remain valid until the RKNN3 context is released.
Parameters	rknn3_context context: RKNN3 context handle.
	const void* weight_data: Pointer to the LoRA weight data buffer.
	uint64_t weight_size: Buffer size in bytes.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_lora_init_from_data(ctx, lora_data, lora_size);
```

rknn3_lora_load

Loads the specified LoRA into the RKNN3 context. Before calling, LoRA initialization must be completed via `rknn3_lora_init` or `rknn3_lora_init_from_data`, and a valid `rknn3_lora`

descriptor must be obtained via `RKNN3_QUERY_LORA_INFO`.

API	rknn3_lora_load
Function	Loads a LoRA into the RKNN3 context.
Parameters	rknn3_context context: RKNN3 context handle.
	rknn3_lora* lora: Pointer to the LoRA descriptor to load.
Return	int Status code: see 6.2. Status Codes.

Example:

```
// Load the specified LoRA into the context (requires lora_init and querying lora
descriptor first)
rknn3_lora_load(ctx, &loras[0]);
```

rknn3_lora_unload

Unloads the specified LoRA from the RKNN3 context, releasing its occupied resources. After unloading, the adapter is no longer available for any session.

API	rknn3_lora_unload
Function	Unloads a LoRA from the RKNN3 context.
Parameters	rknn3_context context: RKNN3 context handle.
	rknn3_lora* lora: Pointer to the LoRA descriptor to unload.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_lora_unload(ctx, &loras[0]);
```

rknn3_lora_enable

Enables a loaded LoRA at the context level, making it effective for all inference under that context. If you need to independently control LoRA for a specific session, use `rknn3_session_enable_lora`.

Effect rule: This interface only updates the LoRA configuration once at the time of the call and does not repeatedly update it during subsequent inference. If a session has enabled a different LoRA via `rknn3_session_enable_lora`, that session will override this interface's configuration each time `rknn3_session_run` is called. Therefore, it is not recommended to use both interfaces to control LoRA simultaneously.

API	rknn3_lora_enable
Function	Enables a loaded LoRA at the RKNN3 context level.
Parameters	rknn3_context context: RKNN3 context handle.
	rknn3_lora* lora: Pointer to the LoRA descriptor to enable.
Return	int Status code: see 6.2. Status Codes.

API	rknn3_lora_enable
Note	Different adapters mapped to the same pattern slot cannot be enabled together in the context-level execution scope.

Example:

```
// Context-level enable (affects all inference)
rknn3_lora_enable(ctx, &loras[0]);
```

rknn3_lora_disable

Disables an enabled LoRA at the context level. After disabling, the adapter no longer takes effect for inference, but remains loaded and can be re-enabled via `rknn3_lora_enable`.

Effect rule: Same as `rknn3_lora_enable`. It is recommended not to mix context-level and session-level interfaces to control LoRA to avoid uncertain effect ordering.

API	rknn3_lora_disable
Function	Disables an enabled LoRA at the RKNN3 context level.
Parameters	rknn3_context context: RKNN3 context handle.
	rknn3_lora* lora: Pointer to the LoRA descriptor to disable.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_lora_disable(ctx, &loras[0]);
```

rknn3_session_enable_lora

Enables LoRA for the specified session. LoRA weights are loaded at the context level via `rknn3_lora_load`, and different sessions share the same weights; this interface only controls whether the session applies the loaded LoRA and does not reload weights. Ensure session and lora are valid and lora has been loaded via `rknn3_lora_load` before calling.

Effect rule: The LoRA configuration enabled via this interface is repeatedly updated each time `rknn3_session_run` is called. Therefore, if `rknn3_lora_enable` is also used for context-level enabling of different LoRAs, `rknn3_session_run` will override the context-level configuration with the session-level configuration. It is recommended not to mix both interfaces for LoRA control to avoid uncertain effect ordering.

API	rknn3_session_enable_lora
Function	Enables a LoRA loaded in the context for the specified RKNN3 session. LoRA weights are managed by the context and can be shared across multiple sessions.
Parameters	rknn3_session* session: RKNN3 session pointer.
	rknn3_lora* lora: Pointer to the LoRA to enable.
Return	int Status code: see 6.2. Status Codes.

API	rknn3_session_enable_lora
Note	Different adapters mapped to the same pattern slot cannot be enabled together in one session. Different sessions may reuse that slot; the runtime reapplies each session's LoRA binding before inference.

Example:

```
// Enable loaded LoRA (only affects the current session)
rknn3_session_enable_lora(session, &lora);
```

rknn3_session_disable_lora

Disables LoRA for the specified session. After disabling, the session's inference no longer applies LoRA, but the weights remain in the context and other sessions are unaffected. Can be re-enabled at any time via `rknn3_session_enable_lora`. Ensure session and lora are valid before calling.

Effect rule: Same as `rknn3_session_enable_lora`. It is recommended not to mix context-level and session-level interfaces to control LoRA to avoid uncertain effect ordering.

API	rknn3_session_disable_lora
Function	Disables LoRA for the specified RKNN3 session. This operation only affects the current session; LoRA weights remain in the context and do not affect other sessions.
Parameters	rknn3_session* session: RKNN3 session pointer.
	rknn3_lora* lora: Pointer to the LoRA to disable.
Return	int Status code: see 6.2. Status Codes.

Example:

```
// Disable LoRA for the current session (weights remain in the context)
rknn3_session_disable_lora(session, &lora);
```

5.11. Profiling and Debugging

Per-layer intermediate result dump, operator performance profiling, memory usage analysis.

rknn3_dump_features

Executes the model layer by layer and dumps each layer's output tensor in npy format to the specified directory. If no output tensor is provided, it is automatically allocated.

API	rknn3_dump_features
Function	Runs the model layer by layer and saves intermediate tensor data.
Parameters	rknn3_context context: RKNN3 context.
	const rknn3_tensor inputs[]: Input tensor array.
	uint32_t n_inputs: Number of input tensors.

API	rknn3_dump_features
	rknn3_tensor outputs[]: Optional output tensor array; when NULL or count is 0, internally allocated.
	uint32_t n_outputs: Number of output tensors; can be set to 0 to enable internal output allocation.
	const char* dump_dir: Directory path for saving npy files.
	uint32_t core_mask: NPU core mask to dump; when set to 0, uses the `run_core_mask` configured in `rknn3_model_init`.
	const char *dump_selector: Dump mode string. NULL, empty string,* "", or "all" means dump all operators on the selected cores.
Return	int Status code: see 6.2. Status Codes.
Note	The dump directory must be writable; only cores with bits set in `core_mask` perform feature dump, and the selected core mask must be a subset of the model's running core mask.

Example:

```
// Dump intermediate outputs of all operators to the specified directory (output
tensors auto-allocated)
rknn3_dump_features(ctx, inputs, n_inputs, NULL, 0, "/tmp/dump", 0, "all");
```

rknn3_profile_ops

Executes the model layer by layer and prints operator performance information, including time, cycles, and bandwidth. If no output tensor is provided, it is automatically allocated.

API	rknn3_profile_ops
Function	Runs the model layer by layer and prints operator performance information.
Parameters	rknn3_context context: RKNN3 context handle.
	const rknn3_tensor inputs[]: Input tensor array.
	uint32_t n_inputs: Number of input tensors.
	rknn3_tensor outputs[]: Output tensor array, can be NULL.
	uint32_t n_outputs: Number of output tensors; can be 0 to enable internal output allocation.
	uint32_t log_level: Log level (0: summary only; 1: operator details + summary; 2: operator details (with time/cycles/bandwidth) + summary).
Return	int Status code: see 6.2. Status Codes.
Note	The cores used for per-layer profiling come from the run_core_mask set in `rknn3_model_init`.

Example:

```
// Print operator-level performance info (log_level=2 includes
time/cycles/bandwidth)
rknn3_profile_ops(ctx, inputs, n_inputs, NULL, 0, 2);
```

rknn3_profile_mem

This interface is used to print memory usage information for each NPU core; must be called after rknn3_model_init.

API	rknn3_profile_mem
Function	Prints memory usage information for each NPU core.
Parameters	rknn3_context context: RKNN3 context handle.
Return	int Status code: see 6.2. Status Codes.
Note	Recommended to call after rknn3_model_init to ensure memory has been allocated.

Example:

```
// Call after model_init to print each core's memory usage
rknn3_profile_mem(ctx);
```

5.12. Custom Operators

Custom operator plugin registration.

rknn3_register_custom_ops_plugins

Registers custom operator plugins (currently supports post-processing plugins). This interface loads the plugin and registers it to the specified context.

API	rknn3_register_custom_ops_plugins
Function	Registers custom operator plugins.
Parameters	rknn3_context ctx: RKNN3 context handle.
	const char* plugin_path: Path to the plugin shared library.
	int64_t size: Reserved field, can be set to 0.
Return	int Status code: see 6.2. Status Codes.

Example:

```
// Register post-processing plugin shared library
rknn3_register_custom_ops_plugins(ctx, "/path/to/postprocess_plugin.so", 0);
```

rknn3_register_custom_op_func

Custom operator registration function pointer type, used to register custom operators within the plugin library.

Function Pointer Type	rknn3_register_custom_op_func
Function	Returns a custom operator descriptor structure pointer based on the operator index.
Parameters	int op_index: Operator index (starting from 0).

Function Pointer Type	rknn3_register_custom_op_func
Return	rknn3_custom_op*: Pointer to the corresponding custom operator descriptor; returns NULL if the index does not exist.

Example:

```
// Exported function within the plugin library, returns custom operator descriptor
by index
rknn3_custom_op* my_register_func(int op_index) {
    if (op_index == 0) return &my_postprocess_op;
    return NULL;
}
```

5.13. Image Processing

Image memory creation, destruction, and color space conversion.

rknn3_im_mem_create

This interface is used to create an RKNN3 image memory object `rknn3_im_mem`, typically used for image pre-processing/post-processing flows before and after model inference.

API	rknn3_im_mem_create
Function	Allocates an RKNN3 image memory object on the device.
Parameters	rknn3_context context: RKNN3 context handle.
	int32_t width: Image width in pixels.
	int32_t height: Image height in pixels.
	rknn3_im_fmt format: Image format; see rknn3_im_fmt.
	int32_t size: Image memory size in bytes; when 0, automatically calculated from width, height, and format.
	int32_t core_id: Target NPU core ID for allocation.
	rknn3_mem_alloc_flags flags: Memory allocation flags.
	rknn3_im_mem* im_mem: Output parameter, receives the created `rknn3_im_mem` object pointer.
Return	int Status code: see 6.2. Status Codes.
Note	After successful creation, the object must be released with `rknn3_im_mem_destroy`.

Example:

```
rknn3_im_mem im_mem;
// Allocate 1920x1080 RGB888 image memory
rknn3_im_mem_create(ctx, 1920, 1080, RKNN3_IM_FMT_RGB888, 0, 0,
                    RKNN3_FLAG_MEMORY_CACHEABLE, &im_mem);
```

rknn3_im_mem_destroy

This interface is used to destroy an RKNN3 image memory object created via `rknn3_im_mem_create` and release its occupied device memory.

API	rknn3_im_mem_destroy
Function	Destroys an RKNN3 image memory object and releases device memory.
Parameters	rknn3_context context: RKNN3 context handle.
	rknn3_im_mem* im_mem: Pointer to the `rknn3_im_mem` object to destroy.
Return	int Status code: see 6.2. Status Codes.
Note	After calling, `im_mem` must not be used again.

Example:

```
rknn3_im_mem_destroy(ctx, &im_mem);
```

rknn3_im_cvt_color

This interface is used to perform image color format conversion on RKNN3 image memory, converting an input image from one RGB/YUV format to another before model inference.

API	rknn3_im_cvt_color
Function	Performs color space conversion between RKNN3 image memory objects.
Parameters	rknn3_context context: RKNN3 context handle.
	rknn3_im_mem* src: Source image memory object.
	rknn3_im_mem* dst: Destination image memory object.
Return	int Status code: see 6.2. Status Codes.
Note	Both `src` and `dst` must be created by `rknn3_im_mem_create`.

Example:

```
// Convert RGB888 to BGR888 (both src and dst must be created with im_mem_create
first)
rknn3_im_cvt_color(ctx, &src_im_mem, &dst_im_mem);
```

5.14. Auxiliary Functions

String conversion for tensor types, quantization types, and layout types.

rknn3_get_type_string

Gets the string representation of a tensor type.

API	rknn3_get_type_string
Function	Gets the string representation of a tensor type.
Parameters	rknn3_tensor_type type: The type of the tensor.
Return	const char* The type string.

Example:

```
const char* type_str = rknn3_get_type_string(RKNN3_TENSOR_FLOAT32);
// type_str = "FP32"
```

rknn3_get_qnt_type_string

Gets the string representation of a quantization type.

API	rknn3_get_qnt_type_string
Function	Gets the string representation of a quantization type.
Parameters	rknn3_tensor_qnt_type type: The quantization type of the tensor.
Return	const char* The quantization type string.

Example:

```
const char* qnt_str = rknn3_get_qnt_type_string(RKNN3_TENSOR_PER_CHANNEL_SYMMETRIC);
// qnt_str = "PER_CHANNEL_SYMMETRIC"
```

rknn3_get_layout_string

Gets the string representation of a layout type.

API	rknn3_get_layout_string
Function	Gets the string representation of a layout type.
Parameters	rknn3_tensor_layout layout: The layout type of the tensor.
Return	const char* The layout type string.

Example:

```
const char* layout_str = rknn3_get_layout_string(RKNN3_TENSOR_NHWC);
// layout_str = "NHWC"
```

5.15. Structured Output (Grammar)

LLM structured output constraints (JSON Schema / EBNF / regex / built-in JSON, etc.) based on xgrammar, used to constrain the model to generate text that conforms to a specific grammar rule. The usage flow is as follows:

1. **Initialize the grammar engine (context level):** Call `rknn3_init_grammar` with the encoded vocabulary (`token_id -> byte string`) and stop token IDs to build the XGrammar TokenizerInfo and GrammarCompiler in one step, shared by all sessions under this context.
2. **Create a grammar handle:** Call `rknn3_grammar_create` to create a `rknn3_grammar` handle bound to the context. Each session should hold its own handle to keep constraints independent.

3. **Set the grammar rule:** Call `rknn3_grammar_set` to set the rule string and type (JSON Schema / EBNF / etc.). The grammar can also be set directly on a session via `rknn3_session_set_grammar`.
4. **Sampling constraint:** In a custom sampling callback, call `rknn3_grammar_apply_mask` before sampling to set the logits of illegal tokens to -inf; after sampling and accepting a token, call `rknn3_grammar_accept_token` to advance the matcher state.
5. **Clear and release:** Use `rknn3_grammar_clear` / `rknn3_session_clear_grammar` to remove the constraint and restore unconstrained generation, `rknn3_grammar_reset` to reset the matcher state, and finally `rknn3_grammar_destroy` to release the handle.

> **Note:** `rknn3_grammar_apply_mask` must be called **before** sampling the next token; `rknn3_grammar_accept_token` must be called **after** sampling and accepting a token.
>
> **Note:** If the grammar is set on a session via `rknn3_session_set_grammar`, `rknn3_session_run` automatically applies the mask and accepts tokens during the decode stage, so there is no need to manually call `rknn3_grammar_apply_mask` / `rknn3_grammar_accept_token`.

rknn3_init_grammar

Initializes the grammar engine at the context (model) level. Builds the XGrammar TokenizerInfo and GrammarCompiler from the `encoded_vocab` in config. The caller is responsible for building `encoded_vocab` (e.g., by iterating token IDs via the tokenizer's TokenToPiece function) before calling this API.

API	rknn3_init_grammar
Function	Initializes the xgrammar-based grammar engine at the context (model) level.
Parameters	rknn3_context context: RKNN3 context handle.
	const rknn3_grammar_config* config: Pointer to the config carrying the encoded vocabulary, vocabulary size, and stop token IDs.
Return	int Status code: see 6.2. Status Codes.
Note	One-time, context-level operation shared by all sessions; must be called before creating/using grammar handles.

Example:

```
rknn3_grammar_config config = {0};
config.vocab_size = vocab_size;
config.encoded_vocab = encoded_vocab;       // serialized token_id -> byte string
data
config.encoded_vocab_len = encoded_vocab_len;
config.n_stop_tokens = 1;
config.stop_tokens = &eos_id;
rknn3_init_grammar(ctx, &config);
```

rknn3_grammar_create

Creates a grammar handle bound to the context. The handle shares the context-level TokenizerInfo / GrammarCompiler and holds its own grammar rule and per-request GrammarMatcher state. Each session should own its own handle to keep constraints independent.

API	rknn3_grammar_create
Function	Creates a grammar handle bound to the context.
Parameters	rknn3_context context: RKNN3 context handle (must have called `rknn3_init_grammar`).
Return	rknn3_grammar* Non-NULL grammar handle on success, NULL on failure.

Example:

```
rknn3_grammar* grammar = rknn3_grammar_create(ctx);
if (!grammar) { /* Error handling */ }
```

rknn3_grammar_destroy

Destroys a grammar handle and releases per-handle resources. Does not affect the context-level vocabulary/compiler; safe to call after the owning session is destroyed.

API	rknn3_grammar_destroy
Function	Destroys a grammar handle and releases related resources.
Parameters	rknn3_context context: RKNN3 context handle.
	rknn3_grammar* grammar: The grammar handle to destroy (may be NULL, no-op).
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_grammar_destroy(ctx, grammar);
grammar = NULL;
```

rknn3_grammar_set

Sets the grammar rule on a handle; takes effect on the next `rknn3_session_run`. For the grammar string and type, see rknn3_grammar_type. Passing NULL or an empty string together with `RKNN3_GRAMMAR_BUILTIN_JSON` enables any valid JSON constraint; for other types, an empty/NULL string disables the constraint. `options` is the compile options (see rknn3_grammar_set_options); each set recompiles the grammar.

API	rknn3_grammar_set
Function	Sets the grammar rule on a grammar handle.
Parameters	rknn3_grammar* grammar: Grammar handle.
	const char* grammar_str: Grammar rule string (JSON Schema / EBNF / regex, etc.).
	rknn3_grammar_type grammar_type: Type of the grammar string.
	const rknn3_grammar_set_options* options: Compile options; pass NULL to use all defaults.

API	rknn3_grammar_set
Return	int Status code: see 6.2. Status Codes.

Example:

```
// Constrain output to a valid JSON object
const char* schema = "{\"type\":\"object\",\"properties\":{\"name\":
{\"type\":\"string\"}},\"required\":[\"name\"]}";
rknn3_grammar_set(grammar, schema, RKNN3_GRAMMAR_JSON_SCHEMA, NULL);

// Any valid JSON
rknn3_grammar_set(grammar, NULL, RKNN3_GRAMMAR_BUILTIN_JSON, NULL);
```

rknn3_grammar_clear

Clears the grammar rule on a handle and restores unconstrained generation. The handle is retained so a subsequent `rknn3_grammar_set` can re-arm it.

API	rknn3_grammar_clear
Function	Clears the grammar rule on a grammar handle.
Parameters	rknn3_grammar* grammar: Grammar handle.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_grammar_clear(grammar);  // Restore unconstrained generation
```

rknn3_grammar_apply_mask

Applies the grammar bitmask to logits (sets illegal token logits to -inf); must be called before sampling the next token. Internally fills the bitmask via GrammarMatcher::FillNextTokenBitmask and applies it with ApplyTokenBitmaskInplaceCPU. `output_idx` is for multi-output scenarios; pass 0 for single output.

API	rknn3_grammar_apply_mask
Function	Applies the grammar bitmask to logits, setting illegal token logits to -inf.
Parameters	rknn3_grammar* grammar: Grammar handle.
	float16* logits: Pointer to the float16 logits array (modified in place).
	int32_t vocab_size: Vocabulary size (length of the logits array).
	int32_t output_idx: Output index for multi-output scenarios (single output: pass 0).
Return	int Status code: see 6.2. Status Codes.

Example:

```
// In a custom sampling callback: apply the mask first, then sample
rknn3_grammar_apply_mask(grammar, logits, vocab_size, 0);
int token_id = my_sampling(logits, vocab_size);
```

rknn3_grammar_accept_token

Notifies the grammar handle that a token was accepted, advancing the matcher state. Must be called after sampling and accepting a token. Returns a negative value if the token is rejected by the grammar (mismatch between mask and sampler).

API	rknn3_grammar_accept_token
Function	Notifies the grammar handle of an accepted token, advancing the matcher state.
Parameters	rknn3_grammar* grammar: Grammar handle.
	int32_t token_id: The token ID that was sampled and accepted.
	int32_t output_idx: Output index for multi-output scenarios (single output: pass 0).
Return	int Status code: 0 means the token is legal; negative value means the token was rejected by the grammar; see 6.2. Status Codes.

Example:

```
rknn3_grammar_accept_token(grammar, token_id, 0);
```

rknn3_grammar_reset

Resets the grammar matcher state (e.g., when clearing conversation history). The handle is retained and can continue to be used.

API	rknn3_grammar_reset
Function	Resets the grammar matcher state.
Parameters	rknn3_grammar* grammar: Grammar handle.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_grammar_reset(grammar);
```

rknn3_session_set_grammar

Convenience interface: sets the grammar on a session (internally creates/uses a grammar handle). Requires `rknn3_init_grammar` to have been called on the context. The session internally owns a `rknn3_grammar` handle; `rknn3_session_run` automatically applies the mask and accepts tokens during decode.

API	rknn3_session_set_grammar
Function	Sets the grammar constraint on a session.

API	rknn3_session_set_grammar
Parameters	rknn3_session* session: RKNN3 session handle.
	const char* grammar_str: Grammar rule string.
	rknn3_grammar_type grammar_type: Type of the grammar string.
	const rknn3_grammar_set_options* options: Compile options; pass NULL to use all defaults.
Return	int Status code: see 6.2. Status Codes.
Note	`rknn3_init_grammar` must have been called on the context.

Example:

```
rknn3_session_set_grammar(session, schema, RKNN3_GRAMMAR_JSON_SCHEMA, NULL);
```

rknn3_session_clear_grammar

Convenience interface: clears the grammar constraint on a session, restoring unconstrained generation.

API	rknn3_session_clear_grammar
Function	Clears the grammar constraint on a session.
Parameters	rknn3_session* session: RKNN3 session handle.
Return	int Status code: see 6.2. Status Codes.

Example:

```
rknn3_session_clear_grammar(session);  // Restore unconstrained generation
```

5.16. MatMul Matrix Multiplication

These interfaces are declared in `rknn3_matmul_api.h`. All return an `int` status code: `RKNN3_SUCCESS` on success, or the corresponding error code on failure. See 6.2 Status Codes and 4.5 MatMul Matrix Multiplication for the calling sequence.

rknn3_matmul_init

Create a MatMul handle and register executable shapes.

```
int rknn3_matmul_init(rknn3_context context,
                      const rknn3_matmul_desc* descs,
                      uint32_t n_descs,
                      rknn3_matmul_context* matmul);
```

Parameter	Description
context	Initialized parent context; must remain valid until the MatMul handle is destroyed.
descs	Input shape descriptor array. Zero-initialize each structure before setting its fields.

Parameter	Description
n_descs	Number of registered shapes; must be greater than zero.
matmul	Output handle address; zero is an invalid handle.

`descs[i]` corresponds to shape index i. One handle uses one computation type and one NPU core; see `rknn3_matmul_desc` for multiple shape constraints. Memory requirements can be queried after initialization. Build the handle and select a shape before execution. Initialization may fail for shape/type combinations unsupported by the target platform; check the return value.

rknn3_matmul_query_infos

Query A/B/C attributes and quantization requirements for all registered shapes.

```
int rknn3_matmul_query_infos(rknn3_matmul_context matmul,
                             rknn3_matmul_info* infos,
                             uint32_t n_descs);
```

Parameter	Description
matmul	Successfully initialized handle.
infos	Caller-provided output array; `infos[i]` corresponds to `descs[i]`.
n_descs	Output array capacity; at least the number of registered shapes.

Querying is available after initialization, before `build`. Insufficient capacity returns an error without writing any entries; excess entries remain unchanged. This interface does not select the active shape. Allocate NPU buffers according to the query and use the queried attributes directly for conversion. Do not derive NPU padding or strides from M/K/N.

rknn3_matmul_build

Build the runtime resources used by the handle.

```
int rknn3_matmul_build(rknn3_matmul_context matmul,
                       const rknn3_matmul_config* config);
```

`matmul` must be successfully initialized. Pass `NULL` for defaults, or a completely zero-initialized configuration structure. Building does not allocate caller-owned A/B/C or select a shape. Repeating a successful build returns `RKNN3_ERR_MATMUL_INVALID_STATE`.

rknn3_matmul_set_shape

Select a registered shape for subsequent conversion, quantization, and execution.

```
int rknn3_matmul_set_shape(rknn3_matmul_context matmul, uint32_t shape_id);
```

`matmul` must be built. `shape_id` is the zero-based registration index. An invalid index preserves the previous selection. This interface only selects the shape; it does not convert or

bind B. Successfully submitted quantization values are retained and need not be resubmitted when switching back to that shape.

rknn3_matmul_transpose

Convert data between contiguous logical and NPU layouts. Here, "transpose" means storage layout conversion; this interface does not provide arbitrary matrix transposition or data type conversion.

```
int rknn3_matmul_transpose(rknn3_matmul_context matmul,
                           const rknn3_matmul_tensor_attr* src_attr,
                           const rknn3_tensor_mem* src,
                           const rknn3_matmul_tensor_attr* dst_attr,
                           rknn3_tensor_mem* dst);
```

Parameter	Description
matmul	Built handle with an active shape.
src_attr / dst_attr	Source and destination attributes. Populate the logical endpoint; use the active shape's queried attributes for the NPU endpoint.
src / dst	Source and destination memory; both require CPU mappings. All arguments are required, including destination arguments for B conversion.

Supported layout directions:

Use	Source Layout	Destination Layout
A or feature B input	NCHW	NC1HWC2
Ordinary B input	NCHW	O1I1HWI2O2
C output	NC1HWC2	NCHW

Both endpoints must have the same canonical shape and `dtype`, matching the active shape. Zero-initialize logical attributes and set `n_dims=4`, `shape`, `dtype`, `layout=NCHW`, and `size` to the logical data size in bytes. `n_strides=0` indicates contiguous storage; arbitrary logical strides are unsupported.

Logical FP16 uses `uint16_t` half-precision bit patterns; a float array cannot be passed directly. INT8/INT32 use `int8_t` / `int32_t`. For W4A16/W6A16, each logical B value occupies one `int8_t`, while `dtype` remains INT4/INT6. Values must be within [-8,7] / [-32,31], respectively. Logical B size is `B_batch*K*N` bytes; this interface performs packing.

The effective CPU address is `virt_addr + offset`, and available capacity is `size - offset`. NPU buffers must also meet the queried alignment and core requirements. Source and destination must not overlap. This interface only converts data; it does not execute NPU tasks, synchronize caches, or retain B in the handle. B must still be supplied at execution.

rknn3_matmul_set_quant

Submit quantization values for the active shape.

```
int rknn3_matmul_set_quant(rknn3_matmul_context matmul,
                           const rknn3_matmul_quantization_t* quant_A,
                           const rknn3_matmul_quantization_t* quant_B,
                           const rknn3_matmul_quantization_t* quant_C);
```

`matmul` must be built and have an active shape. Parameter fields must match the queried `info.A/B/C.quant_info` requirements. See 6.11 MatMul Types for parameter counts and data types.

- W8A8 per-channel/per-layer: all three A/B/C arguments are required. Submit successfully once per shape. B values must match across shapes; A/C may differ.
- W4A16/W6A16/W8A16: supply only B, using `rknn3_matmul_set_quant(matmul, NULL, &quant_B, NULL)`. One successful submission suffices for the entire handle.
- W16A16/W8A8O32: do not call this interface.

This interface copies the quantization arrays. After success, the caller may release the original arrays. W4A16/W6A16/W8A16 also allocate, pack, and synchronize the shared device Param without depending on B weight data; subsequent runs reuse Param. W8A8 retains quantization values for each shape and packs and synchronizes that shape's Param using the current B weights on every run, allocating Param on the first run. Resubmitting W8A8 parameters for the same shape, or group quantization parameters for the same handle, returns `RKNN3_ERR_MATMUL_INVALID_STATE`.

rknn3_matmul_run

Synchronously execute matrix multiplication for the active shape.

```
int rknn3_matmul_run(rknn3_matmul_context matmul,
                     rknn3_tensor_mem* A,
                     rknn3_tensor_mem* B,
                     rknn3_tensor_mem* C);
```

`matmul` must be built, have an active shape, and have the required quantization values submitted. A/B/C are caller-owned NPU buffers. All must be non-NULL and meet the active shape's queried size, layout, strides, alignment, and core requirements. Ordinary packed B also requires a CPU mapping.

Each execution uses the B supplied to that call; the caller may update or replace B before the next execution. Ordinary B uses the queried weight layout; single-shape W16A16 feature B uses NC1HWC2. Convert and synchronize A/B before execution. Synchronize and read C only after success. Do not modify inputs or access output concurrently during execution.

Missing required quantization values returns `RKNN3_ERR_MATMUL_INVALID_STATE`. Success indicates completed NPU execution; failure does not guarantee valid C. See 4.5 MatMul Matrix Multiplication for resource retention when completion is unknown.

rknn3_matmul_destroy

Release the MatMul handle and its runtime resources.

```
int rknn3_matmul_destroy(rknn3_matmul_context matmul);
```

This interface does not release caller-provided A/B/C. After success, the handle is invalid and must not be reused or destroyed again. If `RKNN3_ERR_MATMUL_COMPLETION_UNKNOWN` is returned, the handle and its resources are retained. Until the MatMul handle is successfully destroyed, do not destroy the parent context or modify, reuse, or release A/B/C.

Usage Example

The following snippet registers and builds a W8A8O32 shape. This type does not require quantization parameter submission.

```
#include "rknn3_api.h"
#include "rknn3_matmul_api.h"
#include <stddef.h>

/* context was created by rknn3_init; the caller must destroy the output handle. */
int prepare_matmul(rknn3_context context,
                   rknn3_matmul_context* matmul,
                   rknn3_matmul_info* info)
{
    rknn3_matmul_desc desc = {0};
    desc.M = 16;
    desc.K = 64;
    desc.N = 64;
    desc.A_batch = desc.B_batch = 1;
    desc.type = RKNN3_MATMUL_W8A8O32;
    desc.A_layout = desc.B_layout = desc.C_layout = RKNN3_TENSOR_NCHW;
    desc.core_mask = RKNN3_NPU_CORE_0;

    int ret = rknn3_matmul_init(context, &desc, 1, matmul);
    if (ret != RKNN3_SUCCESS) return ret;
    ret = rknn3_matmul_query_infos(*matmul, info, 1);
    if (ret != RKNN3_SUCCESS) return ret;
    ret = rknn3_matmul_build(*matmul, NULL);
    if (ret != RKNN3_SUCCESS) return ret;
    return rknn3_matmul_set_shape(*matmul, 0);
}
```

Set the output handle to zero before calling. Even if querying or building fails, once initialization has succeeded the caller must check the result of `rknn3_matmul_destroy` and release the handle.

This example uses logical arrays `int8_t a[64*16]`, `int8_t b[64*64]`, and `int32_t c[64*16]`, indexed as `a[k*16+m]`, `b[k*64+n]`, and `c[n*16+m]`. The A/C order differs from conventional row-major M×K and M×N arrays; organize input and output accordingly.

6. Type Reference

6.1. Constants

6.1.1. Tensor Related Constants

Constant Name	Value	Description
RKNN3_MAX_DIMS	16	Maximum number of tensor dimensions
RKNN3_MAX_STRIDE_DIMS	17	Maximum number of tensor stride dimensions
RKNN3_MAX_NAME_LEN	256	Maximum length of a tensor name
RKNN3_MAX_DYNAMIC_SHAPE_NUM	512	Maximum number of dynamic shapes per input
RKNN3_MAX_LORA_NUM	32	Maximum number of LoRA adapters
RKNN3_MAX_SPECIAL_BOS_ID_NUM	64	Maximum number of special beginning-of-sequence (BOS) token IDs
RKNN3_MAX_SPECIAL_EOS_ID_NUM	64	Maximum number of special end-of-sequence (EOS) token IDs
RKNN3_MAX_KVCACHE_LEN_GROUPS	16	Maximum number of KV cache length groups
RKNN3_MAX_ATTENTION_TYPE_NUM	3	Maximum number of attention types
RKNN3_MAX_CACHE_PAGE_TYPES	16	Maximum number of paged KV cache types supported by one core
RKNN3_GRAMMAR_SET_OPTIONS_INIT	`{ -1, -1, -1, -1, -1, NULL, NULL, NULL }`	Default initialization macro for the rknn3_grammar_set_options structure; -1/NULL values mean "use the xgrammar default"

6.1.2. Device Related Constants

Constant Name	Value	Description
RKNN3_MAX_DEVS	64	Maximum number of devices
RKNN3_MAX_DEV_LEN	64	Maximum length of device ID/type
RKNN3_MAX_NPU_NODE_NUM	128	Maximum number of NPU nodes

6.2. Status Codes

Status Code	Value	Description
RKNN3_SUCCESS	0	Success
RKNN3_ERR_FAIL	-1	Execution failure
RKNN3_ERR_ARGUMENT_INVALID	-2	Invalid parameter
RKNN3_ERR_MODEL_INVALID	-3	Invalid model

Status Code	Value	Description
RKNN3_ERR_CTX_INVALID	-4	Invalid context
RKNN3_ERR_RUN_TASK_FAILED	-5	Task execution failure
RKNN3_ERR_OUT_OF_MEMORY	-6	Out of memory
RKNN3_ERR_TIMEOUT	-7	Execution timeout
RKNN3_ERR_INPUT_INVALID	-8	Invalid input
RKNN3_ERR_OUTPUT_INVALID	-9	Invalid output
RKNN3_ERR_DEVICE_UNAVAILABLE	-10	Device unavailable
RKNN3_ERR_DEVICE_UNMATCH	-11	Device mismatch
RKNN3_ERR_TARGET_PLATFORM_UNMATCH	-12	Target platform mismatch
RKNN3_ERR_COMMUNICATION	-13	Communication error
RKNN3_ERR_MEM_SYNC_FAILED	-14	Memory synchronization failure
RKNN3_ERR_KEY_INVALID	-15	Invalid decryption key or RSA decryption failure
RKNN3_ERR_DECRYPT_FAILED	-16	Model decryption failed (key mismatch or data corruption)
RKNN3_ERR_WEIGHT_LOAD_NOT_PREPARED	-17	Streaming weight loading not prepared; please call rknn3_model_init first
RKNN3_ERR_INCOMPATIBLE_VERSION	-18	Component version incompatible: librknn3_api, rknn3_transfer_proxy, and coprocessor firmware (or rknn3_server for non-coprocessor mode) must have consistent versions
RKNN3_ERR_MATMUL_INVALID_STATE	-19	Invalid MatMul call state, such as an unbuilt handle, no active shape, missing quantization values, or repeated submission.
RKNN3_ERR_MATMUL_COMPLETION_UNKNOWN	-20	MatMul task completion cannot be confirmed. Retain the handle, parent context, and A/B/C. Do not modify, reuse, or release the associated buffers until the MatMul handle is successfully destroyed.
RKNN3_WARN_NPU_CORE_UNUSED	-100	NPU core unused; warning only, does not affect model execution

6.3. Enumerations

The RKNN3 enumeration types are categorized by function as follows:

Category	Description
6.3.1. Tensor and Query Enumerations	Contains 4 enumerations
6.3.2. Memory and Core Enumerations	Contains 4 enumerations
6.3.3. KV Cache Enumerations	Contains 6 enumerations

Category	Description
6.3.4. LLM Enumerations	Contains 4 enumerations
6.3.5. Image Processing Enumerations	Contains 2 enumerations
6.3.6. Custom Operator Enumerations	Contains 2 enumerations
6.3.7. Structured Output Enumerations	Contains 1 enumeration

6.3.1. Tensor and Query Enumerations

RKNN3_QUERY_CMD

Query command enumeration, used for the rknn3_query function.

Enum Value	Description
RKNN3_QUERY_IN_OUT_NUM	Query the number of input and output tensors
RKNN3_QUERY_INPUT_ATTR	Query the attributes of input tensors
RKNN3_QUERY_OUTPUT_ATTR	Query the attributes of output tensors
RKNN3_QUERY_SDK_VERSION	Query SDK and driver version
RKNN3_QUERY_CORE_MEM_SIZE	Query weight and internal memory size for each core
RKNN3_QUERY_CUSTOM_STRING	Query custom string
RKNN3_QUERY_ORIGIN_INPUT_ATTR	Query the attributes of original input tensors
RKNN3_QUERY_ORIGIN_OUTPUT_ATTR	Query the attributes of original output tensors
RKNN3_QUERY_DEVICE_MEM_INFO	Query RKNN3 memory information attributes
RKNN3_QUERY_CORE_NUMBER	Query the number of cores
RKNN3_QUERY_ALLOCATION_INFO	Query allocation information
RKNN3_QUERY_DYNAMIC_SHAPE_CONFIG	Query the full dynamic shape configuration
RKNN3_QUERY_DYNAMIC_SHAPE_INFO	Query all supported shape combinations
RKNN3_QUERY_LLM_CONFIG	Query LLM-specific configuration, such as chat template
RKNN3_QUERY_POSTPROCESS_IN_OUT_NUM	Query the number of post-processing inputs and outputs; only valid when post-processing is enabled
RKNN3_QUERY_POSTPROCESS_OUTPUT_ATTR	Query post-processing output tensor attributes; only valid when post-processing is enabled
RKNN3_QUERY_POSTPROCESS_DYNAMIC_SHAPE_INFO	Query post-processing dynamic shape information; only valid when post-processing is enabled
RKNN3_QUERY_LORA_NUM	Query the number of initialized LoRAs; must be called after `rknn3_lora_init` / `rknn3_lora_init_from_data`; result is uint32_t
RKNN3_QUERY_LORA_INFO	Query LoRA information list; result is written to a `rknn3_lora` array (size `RKNN3_MAX_LORA_NUM`)

Enum Value	Description
RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO	Query KV cache length group information (number of groups, KV cache size per group per core)
RKNN3_QUERY_KVCACHE_PAGE_INFO	Query paged KV cache information for the specified physical core (maximum concurrent requests, cache types, page size, and maximum page count)
RKNN3_QUERY_ATTENTION_CONFIG	Query all `rknn3_attention_config` entries, including paged-cache configuration such as tokens per page
RKNN3_QUERY_CMD_MAX	Maximum value of the query command enumeration

RKNN3_TENSOR_TYPE

Tensor data type enumeration.

Enum Value	Description
RKNN3_TENSOR_FLOAT32	float32 type
RKNN3_TENSOR_FLOAT16	float16 type
RKNN3_TENSOR_INT8	int8 type
RKNN3_TENSOR_UINT8	uint8 type
RKNN3_TENSOR_INT16	int16 type
RKNN3_TENSOR_UINT16	uint16 type
RKNN3_TENSOR_INT32	int32 type
RKNN3_TENSOR_UINT32	uint32 type
RKNN3_TENSOR_INT64	int64 type
RKNN3_TENSOR_UINT64	uint64 type
RKNN3_TENSOR_BOOL	bool type
RKNN3_TENSOR_INT4	int4 type
RKNN3_TENSOR_FLOAT8E4M3FN	float8e4m3fn type
RKNN3_TENSOR_BFLOAT16	bfloat16 type
RKNN3_TENSOR_FLOAT8E8M0	float8e8m0 type
RKNN3_TENSOR_FLOAT4E2M1	float4e2m1 (fp4) type
RKNN3_TENSOR_INT6	int6 type
RKNN3_TENSOR_TYPE_MAX	Maximum value of the tensor data type enumeration

RKNN3_TENSOR_QNT_TYPE

Quantization type enumeration.

Enum Value	Description
RKNN3_TENSOR_QNT_NONE	No quantization
RKNN3_TENSOR_PER_LAYER_SYMMETRIC	Per-layer symmetric quantization
RKNN3_TENSOR_PER_LAYER_ASYMMETRIC	Per-layer asymmetric quantization
RKNN3_TENSOR_PER_CHANNEL_SYMMETRIC	Per-channel symmetric quantization
RKNN3_TENSOR_PER_CHANNEL_ASYMMETRIC	Per-channel asymmetric quantization
RKNN3_TENSOR_PER_GROUP_SYMMETRIC	Per-group symmetric quantization
RKNN3_TENSOR_PER_GROUP_ASYMMETRIC	Per-group asymmetric quantization
RKNN3_TENSOR_QNT_MAX	Maximum value of the quantization type enumeration

RKNN3_TENSOR_LAYOUT

Tensor data layout enumeration.

Enum Value	Description
RKNN3_TENSOR_UNDEFINED	Undefined
RKNN3_TENSOR_NCHW	Data layout is NCHW
RKNN3_TENSOR_NHWC	Data layout is NHWC
RKNN3_TENSOR_NC1HWC2	Data layout is NC1HWC2
RKNN3_TENSOR_CHWN	Reserved value, not currently used
RKNN3_TENSOR_HWIO	Reserved value, not currently used
RKNN3_TENSOR_OIHW	Reserved value, not currently used
RKNN3_TENSOR_O1I1HWI2O2	Reserved value, not currently used
RKNN3_TENSOR_LAYOUT_MAX	Maximum value of the tensor data layout enumeration

6.3.2. Memory and Core Enumerations

RKNN3_MEM_ALLOC_FLAGS

Memory allocation flags for creating RKNN3 tensor memory.

Enum Value	Description
RKNN3_FLAG_MEMORY_FLAGS_DEFAULT	Same as RKNN3_FLAG_MEMORY_CACHEABLE
RKNN3_FLAG_MEMORY_CACHEABLE	Create cacheable memory
RKNN3_FLAG_MEMORY_NON_CACHEABLE	Create non-cacheable memory

RKNN3_MEM_SYNC_MODE

Memory synchronization mode for the rknn3_mem_sync function.

Enum Value	Description
RKNN3_MEMORY_SYNC_TO_DEVICE	Synchronizes CPU cache data to DDR. Typically used after CPU writes to memory and before the NPU accesses the same memory, to write back cache data to DDR.
RKNN3_MEMORY_SYNC_FROM_DEVICE	Synchronizes DDR data to CPU cache. Typically used after the NPU writes to memory, to invalidate the cache so that the next CPU access to the same memory reads data from DDR.
RKNN3_MEMORY_SYNC_BIDIRECTIONAL	Synchronizes CPU cache data to DDR and also causes the CPU to re-read data from DDR.

RKNN3_MEM_TYPE

Memory type enumeration.

Enum Value	Description
RKNN3_MEMORY_TYPE_NPU_DRAM	NPU DRAM memory
RKNN3_MEMORY_TYPE_EXT_DDR	External DDR memory

RKNN3_CORE_MASK

Mode for running on target NPU cores.

Enum Value	Description
RKNN3_NPU_CORE_AUTO	Default; indicates automatic scheduling, automatically running on the currently idle NPU core
RKNN3_NPU_CORE_0	Run on NPU core 0
RKNN3_NPU_CORE_1	Run on NPU core 1
RKNN3_NPU_CORE_2	Run on NPU core 2
RKNN3_NPU_CORE_3	Run on NPU core 3
RKNN3_NPU_CORE_4	Run on NPU core 4
RKNN3_NPU_CORE_5	Run on NPU core 5
RKNN3_NPU_CORE_6	Run on NPU core 6
RKNN3_NPU_CORE_7	Run on NPU core 7
RKNN3_NPU_CORE_ALL	Indicates running on all NPU cores

6.3.3. KV Cache Enumerations

RKNN3_KVCACHE_POLICY

KV cache policy.

Enum Value	Description
RKNN3_KVCACHE_POLICY_DEFAULT	Default policy, equivalent to RKNN3_KVCACHE_POLICY_RECURRENT
RKNN3_KVCACHE_POLICY_RECURRENT	Use recurrent cache policy

Enum Value	Description
RKNN3_KVCACHE_POLICY_NORMAL	Use normal cache policy; KV cache is used only within max_context_len
RKNN3_KVCACHE_POLICY_SAVE_CHECKPOINT	Use checkpoint cache policy

RKNN3_KVCACHE_CLEAR_POLICY

Policy for clearing KV cache.

Enum Value	Description
RKNN3_KVCACHE_CLEAR_ALL	Clear all KV cache entries completely
RKNN3_KVCACHE_KEEP_SYSTEM_PROMPT	Clear KV cache but keep the system prompt
RKNN3_KVCACHE_KEEP_N_TOKENS	Clear KV cache but keep N tokens; the number of kept tokens is specified by `rknn3_kvcache_clear_policy_param.keep_n_tokens.n_tokens`

RKNN3_KVCACHE_DTYPE

KV cache data type. These types are internally defined data types that affect KV cache size and management; users do not need to parse them.

Enum Value	Description
RKNN3_KVCACHE_DTYPE_UNDEFINED	Undefined
RKNN3_KVCACHE_DTYPE_INT4_TO_F16	INT4 to FLOAT16
RKNN3_KVCACHE_DTYPE_INT4_TO_F8	INT4 to FLOAT8
RKNN3_KVCACHE_DTYPE_INT8_TO_F16	INT8 to FLOAT16
RKNN3_KVCACHE_DTYPE_FLOAT4_TO_F16	FLOAT4 to FLOAT16
RKNN3_KVCACHE_DTYPE_FLOAT4_TO_F8	FLOAT4 to FLOAT8
RKNN3_KVCACHE_DTYPE_FLOAT8_TO_F16	FLOAT8 to FLOAT16
RKNN3_KVCACHE_DTYPE_FLOAT8_TO_F8	FLOAT8 to FLOAT8
RKNN3_KVCACHE_DTYPE_FLOAT16	FLOAT16

RKNN3_KVCACHE_STORE_METHOD

KV cache storage method. These types are internally defined data types that affect KV cache size and management; users do not need to parse them.

Enum Value	Description
RKNN3_KVCACHE_STORE_METHOD_UNDEFINED	Undefined
RKNN3_KVCACHE_STORE_METHOD_NORMAL	Normal storage
RKNN3_KVCACHE_STORE_METHOD_GROUP_QUANT	Group quantization storage

RKNN3_ATTENTION_TYPE

LLM attention type enumeration, used for attention configuration.

Enum Value	Description
RKNN3_ATTENTION_TYPE_FULL_ATTENTION	Full attention
RKNN3_ATTENTION_TYPE_SLIDING_ATTENTION	Sliding attention
RKNN3_ATTENTION_TYPE_LINEAR_ATTENTION	Linear attention

RKNN3_PAGE_CACHE_TYPE

Page KV cache category. The enum values match the cache categories in the model schema.

Enum Value	Value	Description
RKNN3_PAGE_CACHE_GENERAL	11	General cache
RKNN3_PAGE_CACHE_FULL_ATTENTION	12	Full-attention cache
RKNN3_PAGE_CACHE_SLIDING_ATTENTION	13	Sliding-attention cache
RKNN3_PAGE_CACHE_LINEAR_ATTENTION	14	Linear-attention cache
RKNN3_PAGE_CACHE_CONV_STREAMING	15	Convolution streaming cache
RKNN3_PAGE_CACHE_FULL_RESIDUAL	17	Full-attention residual cache
RKNN3_PAGE_CACHE_SLIDING_RESIDUAL	18	Sliding-attention residual cache

6.3.4. LLM Enumerations

RKNN3_LLM_INPUT_TYPE

Defines the input types that can be provided to the LLM.

Enum Value	Description
RKNN3_LLM_INPUT_PROMPT	Input is a text prompt
RKNN3_LLM_INPUT_TOKEN	Input is a sequence of tokens
RKNN3_LLM_INPUT_EMBED	Input is embedding vectors
RKNN3_LLM_INPUT_MULTIMODAL	Multimodal input
RKNN3_LLM_INPUT_AUX	Other type of input
RKNN3_LLM_INPUT_MAX	Maximum value of the input type enumeration

LLMCALLSTATE

Describes the possible states of an LLM call.

Enum Value	Description
RKLLM_RUN_NORMAL	LLM call is in normal running state
RKLLM_RUN_WAITING	LLM call is waiting for a complete UTF-8 encoded character
RKLLM_RUN_FINISH	LLM call has finished execution

Enum Value	Description
RKLLM_RUN_STOP	User stopped the LLM call
RKLLM_RUN_MAX_NEW_TOKEN_REACHED	Maximum new token count has been reached
RKLLM_RUN_ERROR	An error occurred during the LLM call
RKLLM_RUN_PAUSE	LLM call has been paused
RKLLM_RUN_RESUME	LLM call has resumed from pause

RKNN3_LLM_TASK_TYPE

LLM task type enumeration.

Enum Value	Description
RKNN3_LLM_TASK_GENERATE	Generation task
RKNN3_LLM_TASK_EMBEDDING	Embedding task
RKNN3_LLM_TASK_RERANKER	Reranking task

LLMOUTPUTCALLBACKSTATE

Output callback stage enumeration.

Enum Value	Description
RKLLM_OUTPUT_CALLBACK_PREFILL_PROCESSING	Prefill stage callback processing
RKLLM_OUTPUT_CALLBACK_PREFILL_FINISHED	Prefill stage callback finished
RKLLM_OUTPUT_CALLBACK_DECODE_PROCESSING	Decode stage callback processing
RKLLM_OUTPUT_CALLBACK_DECODE_FINISHED	Decode stage callback finished

6.3.5. Image Processing Enumerations

RKNN3_IM_FMT

Image format enumeration, used for RKNN3 image-related interfaces.

Enum Value	Description
RKNN3_IM_FMT_RGB888	RGB888 format, commonly used for general image preprocessing
RKNN3_IM_FMT_BGR888	BGR888 format, commonly used for OpenCV image processing
RKNN3_IM_FMT_GRAY8	8-bit grayscale image format
RKNN3_IM_FMT_YCbCr_420_SP	YCbCr 4:2:0 semi-planar format, commonly used for video encoding/decoding
RKNN3_IM_FMT_YCrCb_420_SP	YCrCb 4:2:0 semi-planar format, commonly used for video encoding/decoding
RKNN3_IM_FMT_YCbCr_422_SP	YCbCr 4:2:2 semi-planar format, commonly used for video encoding/decoding
RKNN3_IM_FMT_YCrCb_422_SP	YCrCb 4:2:2 semi-planar format, commonly used for video encoding/decoding

Enum Value	Description
RKNN3_IM_FMT_UNKNOWN	Unknown or unsupported image format

RKNN3_IM_PROC_FLAG

Image processing flag bit enumeration.

Enum Value	Description
RKNN3_IM_PROC_FLAG_NONE	No processing
RKNN3_IM_PROC_FLAG_CROP	Crop
RKNN3_IM_PROC_FLAG_RESIZE	Resize
RKNN3_IM_PROC_FLAG_FILL	Fill
RKNN3_IM_PROC_FLAG_COLOR_SPACE_CONVERT	Color space conversion
RKNN3_IM_PROC_FLAG_DECODE	Decode
RKNN3_IM_PROC_FLAG_ENCODE	Encode

6.3.6. Custom Operator Enumerations

RKNN3_OP_TARGET_TYPE

Custom operator backend target enumeration.

Enum Value	Description
RKNN3_OP_TARGET_TYPE_CPU	Run on CPU
RKNN3_OP_TARGET_TYPE_MAX	Maximum value of the enumeration

RKNN3_OP_PLUGIN_TYPE

Custom operator plugin type enumeration.

Enum Value	Description
RKNN3_OP_PLUGIN_TYPE_POSTPROCESS	Post-processing plugin
RKNN3_OP_PLUGIN_TYPE_CUSTOM_OP	Custom operator plugin (currently not supported)
RKNN3_OP_PLUGIN_TYPE_MAX	Maximum value of the enumeration

6.3.7. Structured Output Enumerations

RKNN3_GRAMMAR_TYPE

Grammar type enumeration for structured output, used by `rknn3_grammar_set` / `rknn3_session_set_grammar`.

Enum Value	Description
RKNN3_GRAMMAR_JSON_SCHEMA	JSON Schema string
RKNN3_GRAMMAR_EBNF	EBNF grammar string

Enum Value	Description
RKNN3_GRAMMAR_BUILTIN_JSON	Built-in JSON grammar (any valid JSON)
RKNN3_GRAMMAR_REGEX	Regular expression
RKNN3_GRAMMAR_STRUCTURAL_TAG	Structural tag (JSON serialized)

6.4. General Structures

Tensor Related Structures

RKNN3_INPUT_OUTPUT_NUM

Information structure for RKNN3_QUERY_IN_OUT_NUM.

Member	Data Type	Description
n_input	uint32_t	Number of inputs
n_output	uint32_t	Number of outputs

RKNN3_QUANT_INFO

Quantization information structure.

Member	Data Type	Description
scale	float	Scale data
zero_point	int32_t	Zero point data

RKNN3_TENSOR_ATTR

Structure for storing RKNN3_QUERY_INPUT_ATTR/RKNN3_QUERY_OUTPUT_ATTR query results.

Member	Data Type	Description
index	uint32_t	Input parameter: index of the input/output tensor; the index value must be set before calling rknn3_query
name	char[RKNN3_MAX_NAME_LEN]	Name of the tensor
n_dims	uint32_t	Number of dimensions
shape	uint32_t[RKNN3_MAX_DIMS]	Valid dimension array
aligned_size	uint64_t	Tensor size of the aligned shape in bytes
n_stride	uint32_t	Number of strides
stride	uint64_t[RKNN3_MAX_STRIDE_DIMS]	Strides of the tensor; e.g., strides of a 16x16 tensor are [16*16, 16, 1]
n_elems	uint32_t	Number of elements in the tensor
dtype	rknn3_tensor_type	Data type of the tensor
layout	rknn3_tensor_layout	Data layout of the tensor
qnt_type	rknn3_tensor_qnt_type	Quantization type of the tensor

Member	Data Type	Description
qnt_info	rknn3_quant_info	Quantization information of the tensor
n_orig_dims	uint32_t	Number of original dimensions
orig_shape	uint32_t[RKNN3_MAX_DIMS]	Valid original dimension array
orig_layout	rknn3_tensor_layout	Data layout of the original tensor
core_id	int32_t	Core ID of the tensor buffer

RKNN3_TENSOR_MEM

Tensor memory information structure.

Member	Data Type	Description
virt_addr	void*	Virtual address of the tensor buffer
phys_addr	uint64_t	Physical address of the tensor buffer
fd	int32_t	File descriptor of the tensor buffer
buffer_id	uint64_t	Buffer ID of the tensor buffer, used for memory management
offset	uint64_t	Memory offset
size	uint64_t	Size of the tensor buffer
flags	uint64_t	Flags of the tensor buffer, reserved field
core_id	int32_t	NPU core ID
priv_data	void*	Private data of the tensor buffer
mem_type	rknn3_mem_type	Memory type of the tensor buffer

RKNN3_TENSOR

Structure representing an RKNN3 tensor containing memory and attribute information.

This structure contains the memory information and attributes of the tensor used in RKNN3 operations, and is the basic data structure for the RKNN3 runtime to handle tensors.

Member	Data Type	Description
mem	rknn3_tensor_mem*	Memory information of the tensor
attr	rknn3_tensor_attr*	Attributes of the tensor

Model and Configuration Structures

RKNN3_SDK_VERSION

SDK version information structure.

Member	Data Type	Description
api_version	char[256]	Version of the RKNN3 API
drv_version	char[256]	Version of the RKNN3 driver

RKNN3_CORE_MEM_SIZE

Structure for weight and internal memory size information for each core.

Member	Data Type	Description
core_id	int32_t	Physical NPU core ID to which the memory belongs
weight_size	uint64_t	Weight memory size in bytes
internal_size	uint64_t	Internal tensor memory size in bytes
reserved	uint8_t[32]	Reserved field

RKNN3_CUSTOM_STRING

Custom string information structure.

Member	Data Type	Description
string	char[1024]	Custom string, maximum length is 1024 bytes

RKNN3_CONFIG

Model loading control parameter structure.

Member	Data Type	Description
priority	int32_t	Run priority
run_timeout	uint32_t	Run timeout in milliseconds; 0 means no timeout
run_core_mask	uint32_t	Core mask for model execution
user_mem_weight	uint8_t	Whether weight memory is allocated by the user
user_mem_internal	uint8_t	Whether internal memory is allocated by the user
user_sram	uint8_t	Whether SRAM memory is allocated by the user
reserved	uint8_t[128]	Reserved field

RKNN3_SHAPE_INFO

RKNN3 model tensor shape information structure.

This structure contains shape information for RKNN3 model input and output tensors, including tensor attributes and shape configuration details.

Member	Data Type	Description
shape_id	int32_t	Unique ID for this shape combination
n_inputs	uint32_t	Number of input tensors
input_attrs	rknn3_tensor_attr*	Input tensor attribute array
n_outputs	uint32_t	Number of output tensors
output_attrs	rknn3_tensor_attr*	Output tensor attribute array
is_default	uint8_t	Whether this is the default shape
reserved	uint8_t[31]	Reserved field

RKNN3_SHAPE_CONFIG

Configuration structure for dynamic shape setting.

This structure contains information about shape combinations for dynamic shape inference in the RKNN3 model and the currently active shape.

Member	Data Type	Description
n_shapes	uint32_t	Number of available shape combinations
current_shape_id	int32_t	ID of the currently active shape configuration; -1 means no active shape

RKNN3_ALLOCATION_INFO

Memory allocation information structure for the RKNN3 model.

This structure provides detailed memory allocation information for different memory types (command, weight, internal, KV cache) and their distribution across NPU cores.

Member	Data Type	Description
core_id	int32_t	NPU core ID
command_mem	rknn3_tensor_mem	Command memory information
weight_mem	rknn3_tensor_mem	Weight memory information
internal_mem	rknn3_tensor_mem	Internal memory information
kvcache_mem	rknn3_tensor_mem	KV cache memory information
reserved	uint8_t[128]	Reserved field

Device Related Structures

RKNN3_INIT_EXTEND

Device-specific initialization information structure.

This structure is used to specify device-specific parameters during RKNN3 runtime context initialization, including device ID and reserved space for future use.

Member	Data Type	Description
device_id	char*	Input parameter indicating which device to select. Can be set to NULL if only one device is connected.
reserved	uint8_t[128]	Reserved field

RKNN3_NODE_MEM_INFO

RKNN3 device node memory information structure.

This structure provides detailed memory information for each node in an RKNN3 device, including total memory available for allocation and free memory.

Member	Data Type	Description
total	uint64_t	Total memory available for this node in bytes

Member	Data Type	Description
free	uint64_t	Free memory available for this node in bytes

RKNN3_DEV_MEM_INFO

RKNN3 device node memory information structure.

This structure provides detailed memory information for each node in an RKNN3 device, including total memory available for allocation and free memory.

Member	Data Type	Description
node_num	uint32_t	Number of nodes in the device
sys_total	uint64_t	Total system memory of the device in bytes
sys_free	uint64_t	Free system memory of the device in bytes
node_mem_info	rknn3_node_mem_info[RKNN3_MAX_NPU_NODE_NUM]	Memory information for each node

RKNN3_DEVICE

Structure representing an RKNN3 device.

This structure contains information about a specific RKNN3 device, including its ID, type, and memory information.

Member	Data Type	Description
id	char[RKNN3_MAX_DEV_LEN]	Device ID
type	char[RKNN3_MAX_DEV_LEN]	Device type
mem_info	rknn3_dev_mem_info	Memory information of the device

RKNN3_DEVICES

Structure containing RKNN3 device information.

This structure contains the number of available RKNN3 devices.

Member	Data Type	Description
n_devices	uint32_t	Number of devices
devices	rknn3_device[RKNN3_MAX_DEVS]	Device information

6.5. LLM and VLM Structures

RKNN3_LLM_CONFIG

LLM configuration structure.

This structure contains the basic configuration required to initialize and run an LLM.

Member	Data Type	Description
chat_template	char*	Chat template string
vocab_size	uint32_t	Vocabulary size
embedding_dim	uint32_t	Embedding dimension (typically equals hidden size)
max_ctx_len	uint32_t	Maximum context length
max_position_embeddings	uint32_t	Maximum position embedding length
kvcache_store_method	rknn3_kvcache_store_method	KV cache storage method
kvcache_dtype	rknn3_kvcache_dtype	KV cache data type
kvcache_group_size	uint32_t	KV cache group quantization group size
kvcache_residual_depth	uint32_t	KV cache residual depth
model_type	char*	Model type string
task_type	rknn3_llm_task_type	LLM task type
rope_cache_host_storage	uint8_t	Whether RoPE cache is stored on the host side
n_attention_kvcache_lens	uint32_t	Number of valid attention KV cache length configurations
attention_kvcache_lens	rknn3_attention_kvcache_lens [RKNN3_MAX_ATTENTION_TYPE_NUM]	KV cache length configuration for each attention type
reserved	uint8_t[127]	Reserved field

RKNN3_VOCAB_INFO

RKNN3 model vocabulary information structure.

This structure contains vocabulary information used in the RKNN3 model, including size and special token IDs.

Member	Data Type	Description
vocab_size	int	Vocabulary size
special_bos_id	int[RKNN3_MAX_SPECIAL_BOS_ID_NUM]	Special beginning-of-sequence (BOS) token IDs
special_eos_id	int[RKNN3_MAX_SPECIAL_EOS_ID_NUM]	Special end-of-sequence (EOS) token IDs
n_special_bos_id	int	Number of special BOS token IDs
n_special_eos_id	int	Number of special EOS token IDs
linefeed_id	int	Linefeed token ID
skip_special_token	bool	Whether to skip special tokens during generation
ignore_eos_token	bool	Whether to ignore EOS tokens during generation
reserved	uint8_t[64]	Reserved field for future expansion

RKNN3_LLM_EXTEND_PARAM

LLM extended parameter structure.

Member	Data Type	Description
reserved	uint8_t[128]	Reserved field

RKNN3_SAMPLING_PARAMS

Structure defining sampling parameters for an LLM instance.

Member	Data Type	Description
top_k	int32_t	Top-K sampling parameter for token generation
top_p	float	Top-P (nucleus) sampling parameter
temperature	float	Sampling temperature, affecting the randomness of token selection
repeat_penalty	float	Penalty for repeated tokens during generation
frequency_penalty	float	Penalty for frequent tokens during generation
presence_penalty	float	Penalty based on token presence in the input

RKNN3_LLM_PARAM

Structure defining parameters for configuring an LLM instance.

Member	Data Type	Description
logits_name	char*	Name of the output logits. Only needs to be explicitly set when the model has multiple outputs; otherwise can be NULL.
max_context_len	int32_t	Maximum number of tokens in the context
sampling_param	rknn3_sampling_params	Sampling parameters for token generation
vocab_info	rknn3_vocab_info	Vocabulary information
extend_param	rknn3_llm_extend_param	Extended parameters

RKNN3_LORA

Structure defining LoRA (Low-Rank Adaptation) parameters used in model fine-tuning.

Member	Data Type	Description
lora_name	char[RKNN3_MAX_NAME_LEN]	Name of the LoRA
scale	float	Scale factor applied to the LoRA

RKNN3_LLM_MULTIMODAL_TENSOR

Structure representing multimodal input (e.g., text, image, audio, and video). Top-level members

Member	Data Type	Description
name	const char*	Name of this tensor
prompt	const char*	Text prompt input
tokens	int32_t*	Token input. Mutually exclusive with prompt input; the user can provide either. Note: 1. If both inputs are provided, prompt input is used by default for inference. 2. When choosing token input, the user needs to parse text and multimodal tags into tokens according to the rules.
n_tokens	uint64_t	Number of tokens in the token input. Needs to be set when using token input.
enable_thinking	bool	Controls whether "thinking mode" is enabled
image	struct	Image substructure
audio	struct	Audio substructure
video	struct	Video substructure

image substructure members

Member	Data Type	Description
image_embed	float16*	Image embeddings (size: n_image x n_image_tokens x embedding_dim x sizeof(float16))
n_image_tokens	uint64_t	Number of tokens per image
n_image	uint64_t	Number of images
image_start	const char*	Start tag for images in multimodal input
image_end	const char*	End tag for images in multimodal input
image_content	const char*	Image content placeholder tag in multimodal input
image_width	uint64_t	Image width
image_height	uint64_t	Image height

audio substructure members

Member	Data Type	Description
audio_embed	float16*	Audio embeddings (size: n_audio x n_audio_tokens x embedding_dim x sizeof(float16))
n_audio_tokens	uint64_t	Number of tokens per audio segment
n_audio	uint64_t	Number of audio segments
audio_start	const char*	Start tag for audio in multimodal input

Member	Data Type	Description
audio_end	const char*	End tag for audio in multimodal input
audio_content	const char*	Audio content placeholder tag in multimodal input

video substructure members

Member	Data Type	Description
video_embed	float16*	Video embeddings (size: n_video x n_frame_per_video x n_frame_tokens x embedding_dim x sizeof(float16))
n_frame_tokens	uint64_t	Number of tokens per frame
n_frame_per_video	uint64_t	Number of frames per video
n_video	uint64_t	Number of videos
video_start	const char*	Start tag for video in multimodal input
video_end	const char*	End tag for video in multimodal input
video_content	const char*	Video content placeholder tag in multimodal input
frame_width	uint64_t	Video frame width
frame_height	uint64_t	Video frame height

RKNN3_LLM_TENSOR

Structure representing a tensor for large language model operations.

This structure contains the basic components needed to handle language model embeddings, including tensor name, embedding vectors, token IDs, and token count.

Member	Data Type	Description
name	const char*	Name of this tensor
prompt	const char*	Text prompt input if input_type is RKNN3_LLM_INPUT_PROMPT
embed	float16*	Pointer to embedding vectors if input_type is RKNN3_LLM_INPUT_EMBED (size: n_tokens * hidden_size)
tokens	int32_t*	Token ID array if input_type is RKNN3_LLM_INPUT_TOKEN
n_tokens	uint64_t	Number of tokens represented in the embedding
enable_thinking	bool	Controls whether "thinking mode" is enabled

RKNN3_LLM_INPUT

Structure representing different types of LLM input via a union.

Member	Data Type	Description
role	const char*	Message role: "user" (user input), "tool" (function result)
input_type	rknn3_llm_input_type	Specifies the type of input provided (e.g., token, embed, aux)
llm_input	rknn3_llm_tensor	Embedding vectors when input_type is RKNN3_LLM_INPUT_EMBED; token array when input_type is RKNN3_LLM_INPUT_TOKEN; uses the prompt field when input_type is RKNN3_LLM_INPUT_PROMPT
multimodal_input	rknn3_llm_multimodal_tensor	Multimodal input when input_type is RKNN3_LLM_INPUT_MULTIMODAL
aux_input	rknn3_aux_tensor	AUX input if input_type is RKNN3_LLM_INPUT_AUX; currently rknn3_aux_tensor and rknn3_tensor are the same type

RKNN3_LLM_INFER_PARAM

LLM inference parameter structure, defining parameters during the inference process.

Member	Data Type	Description
keep_history	int	History retention flag (1: keep history, 0: discard history)
max_new_tokens	int32_t	Maximum number of new tokens to generate
prefill_only	bool	Whether to only perform the prefill stage. When set to true, the model only performs prefill and skips the decode stage, but still samples the prefill output tokens. Defaults to false.
disable_sampling	bool	Whether to disable sampling. When set to true, the model does not perform sampling, suitable for scenarios where only prefill output is needed and prefill output tokens are not desired (e.g., embedding and reranker models). Typically used with `prefill_only` (setting `disable_sampling` alone is ineffective). Defaults to false.
keep_grammar	bool	Flag to keep the grammar matcher state across runs. When enabled, the per-request grammar matcher is NOT re-initialized at the start/end of each `rknn3_session_run`, so the grammar state is retained for continuous generation across multiple runs. Defaults to false.
reserved	uint8_t[127]	Reserved field for future expansion

RKLLMRESULT

Structure representing LLM inference results.

Member	Data Type	Description
token_ids	int*	Pointer to the tokens generated by the LLM

Member	Data Type	Description
num_tokens	int	Number of generated tokens

RKLLMRUNSTATE

The RKLLMRunState structure contains the state information of the LLM run.

Member	Data Type	Description
n_total_tokens	uint64_t	Total number of tokens processed so far
n_reuse_tokens	uint64_t	Number of tokens in the current input that hit the KV cache, i.e., the number of KV cache reused tokens
n_max_tokens	uint64_t	Maximum number of tokens that can be processed
n_prefill_tokens	uint64_t	Number of tokens processed in the prefill stage
n_decode_tokens	uint64_t	Number of tokens generated in the decode stage
n_input_tokens	uint64_t	Number of tokens input to the model (same as n_prefill_tokens)
n_output_tokens	uint64_t	Number of tokens output by the model (n_output_tokens = n_decode_tokens + 1, where 1 is the prefill output token)
input_tokens	const int32_t*	Pointer to the token array input to the model; the user should not free this pointer
output_tokens	const int32_t*	Pointer to the model's output token array; the user should not free this pointer
kvcache_policy	rknn3_kvcache_policy	KV cache policy
n_loras_enabled	int32_t	Number of enabled LoRAs
loras_enabled	rknn3_lora*	List of enabled LoRAs

6.6. KV Cache Structures

KV Cache Related Structures

RKNN3_KVCACHE_POLICY_PARAM

Structure defining parameters for KV cache policy. If the model contains a system prompt, n_keep and n_keep_aligned in the recurrent parameters are ignored.

Member	Data Type	Description
recurrent.n_keep	int64_t	Number of cache entries to keep under the recurrent policy; ignored if the model contains a system prompt
recurrent.n_keep_aligned	int64_t	Number of kept entries aligned to kvcache_group_size under the recurrent policy

Member	Data Type	Description
save_checkpoint.checkpoint_start_pos	int64_t	Starting position for checkpoint saving (e.g., 0); must be aligned to 128, automatically adjusted if not aligned (no checkpoint is saved at the starting position)
save_checkpoint.checkpoint_interval	int64_t	Token interval for saving checkpoints (e.g., every 128 tokens); must be aligned to 128, automatically adjusted if not aligned
save_checkpoint.max_checkpoint_count	int64_t	Maximum number of checkpoints to save from checkpoint_start_pos; must be less than (max_context_len - checkpoint_start_pos) / checkpoint_interval, otherwise automatically adjusted to the maximum available value
save_checkpoint.checkpoint_tail_overwrite	bool	When set to true, the tail/last checkpoint slot is used as a rolling overwrite slot. (max_checkpoint_count - 1) checkpoints are saved at fixed positions, and when the token position exceeds (checkpoint_start_pos + max_checkpoint_count * checkpoint_interval), subsequent checkpoints will overwrite the tail slot. For example: max_checkpoint_count=5 means 4 fixed checkpoints + 1 rolling checkpoint at the tail
reserved	uint8_t[64]	Reserved field

RKNN3_ATTENTION_KVCACHE_LENS

KV cache buffer length configuration structure, for one attention type.

Member	Data Type	Description
attention_type	rknn3_attention_type	Attention type
n_kvcache_buffer_lens	uint32_t	Number of valid KV cache buffer lengths
kvcache_buffer_lens	int32_t[RKNN3_MAX_KVCACHE_LEN_GROUPS]	KV cache buffer length array

RKNN3_KVCACHE_LEN_GROUP_INFO

KV cache length group information structure, for RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO query.

Usage: First get core_number via RKNN3_QUERY_CORE_NUMBER, then allocate `rknn3_kvcache_len_group_info info[core_number]`, and query with `sizeof(rknn3_kvcache_len_group_info) * core_number` as size.

Member	Data Type	Description
core_id	int32_t	Physical ID of the NPU core

Member	Data Type	Description
n_groups	uint32_t	Number of KV cache length groups (0 means single-group traditional mode)
active_group_id	int32_t	Currently active group ID (default 0)
kvcache_sizes	uint64_t[RKNN3_MAX_KVCACHE_LEN_GROUPS]	kvcache_sizes[group_id] is the KV cache size for that group on this core

RKNN3_ATTENTION_CONFIG

Attention configuration structure returned by `RKNN3_QUERY_ATTENTION_CONFIG`. The fields most relevant to paged KV cache allocation are `attention_type`, `kvcache_buffer_len`, `tokens_per_page`, and `rollback_tokens_align`.

Member	Data Type	Description
attention_type	rknn3_attention_type	Attention type
kvcache_buffer_len	int32_t	KV cache buffer length for this attention type
kvcache_buffer_lens	int32_t[RKNN3_MAX_KVCACHE_LEN_GROUPS]	KV cache buffer length for each length group
n_kvcache_buffer_lens	int32_t	Number of valid entries in `kvcache_buffer_lens`
tokens_per_page	uint32_t	Number of tokens in each page for FullAttention paging
rollback_tokens_align	uint32_t	Alignment granularity for KV cache rollback positions

RKNN3_CACHE_PAGE_ATTR

Page properties for one paged KV cache category, returned in `rknn3_kvcache_page_info.caches[]`.

Member	Data Type	Description
cache_type	int32_t	Page cache category; see `rknn3_page_cache_type`
page_size	uint64_t	Page size in bytes; pass this as the `size` argument to `rknn3_create_mem`
max_page_num	uint32_t	Compiled maximum page capacity; not changed by binding or growth
reserved	uint8_t[32]	Reserved field

RKNN3_KVCACHE_PAGE_INFO

Paged KV cache properties and concurrent-request information for one physical core. Set `core_id` before calling `rknn3_query`; legacy non-paged models return `n_cache_types == 0`.

Member	Data Type	Description
core_id	int32_t	Input field: physical NPU core ID to query
max_request_num	uint32_t	Maximum number of concurrent requests supported by continuous batching
n_cache_types	uint32_t	Number of valid cache categories
caches	rknn3_cache_page_attr[RKNN3_MAX_CACHE_PAGE_TYPES]	Page properties for each cache category
reserved	uint8_t[128]	Reserved field

RKNN3_KVCACHE_PAGE_BINDING

Page-table binding used by `rknn3_set_kvcache_pages`.

Member	Data Type	Description
core_id	int32_t	Physical NPU core ID
cache_type	int32_t	Cache category to bind
request_index	uint32_t	Concurrent-request index; should be less than the corresponding `max_request_num`
n_pages	uint32_t	Number of pages in the `pages` array; 0 unbinds this key
pages	const rknn3_tensor_mem *const*	Page memory array; each page is an independent `page_size`-byte allocation on the corresponding core
reserved	uint8_t[64]	Reserved field

The page memory referenced by `pages` remains owned by the caller. The runtime does not copy page contents or release these objects. After a successful binding, keep the page objects valid until a binding with `n_pages = 0` successfully unbinds them.

6.7. Custom Operator Structures

Custom Operator Structures

RKNN3_CUSTOM_OP_ATTR

Custom operator attribute descriptor structure, used for querying a single operator attribute via `get_param`.

Member	Data Type	Description
name	char[RKNN3_MAX_NAME_LEN]	Operator attribute name
dtype	rknn3_tensor_type	Data type of the operator attribute
n_elems	uint32_t	Number of array elements

Member	Data Type	Description
data	void*	Operator attribute array pointer (non-owning, only valid within the current callback scope; the caller should not cache or free it)

RKNN3_CUSTOM_OP_CONTEXT

Custom operator context structure, managed by the framework; users can store custom information via user_data.

Member	Data Type	Description
rknn_ctx	rknn3_context	RKNN3 context handle
priv_data	void*	Framework private data
user_data	void*	User-defined data
get_param	`int (*)(rknn3_custom_op_context*, const char*, rknn3_custom_op_attr*)`	Function pointer for querying operator parameters by attribute name; set by the framework before calling init/compute callbacks, users should not overwrite it. The returned `op_attr.data` after calling is non-owning, only valid within the current callback scope

RKNN3_CUSTOM_OP

Custom operator structure, containing operator metadata and callback function set.

Member	Data Type	Description
op_type	const char*	Custom operator type name
plugin_type	rknn3_op_plugin_type	Plugin type
target	rknn3_op_target_type	Backend execution target
version	uint32_t	Operator version number
author	const char*	Author information
description	const char*	Operator description
init	`int (*)(rknn3_custom_op_context*)`	Initialization callback (optional)
prepare	`int (*)(rknn3_custom_op_context*, rknn3_tensor*, uint32_t, rknn3_tensor*, uint32_t)`	Prepare callback (optional)
compute	`int (*)(rknn3_custom_op_context*, rknn3_tensor*, uint32_t, rknn3_tensor*, uint32_t)`	Compute callback (required)
deinit	`int (*)(rknn3_custom_op_context*)`	De-initialization callback (optional)
get_output_num	`int (*)(rknn3_custom_op_context*)`	Get output count callback (post-processing plugin, optional)
get_attrs	`int (*)(rknn3_custom_op_context*, rknn3_tensor_attr*, uint32_t, rknn3_tensor_attr*, uint32_t)`	Get input/output tensor attributes callback (post-processing plugin, optional)

6.8. Image Processing Structures

Image Processing Structures

RKNN3_IM_RECT

Image rectangle structure, used to describe a region of interest (ROI) in image processing.

Member	Data Type	Description
x	int	Top-left X coordinate
y	int	Top-left Y coordinate
width	int	Rectangle width in pixels
height	int	Rectangle height in pixels

RKNN3_IM_METADATA

Metadata structure associated with RKNN3 image memory, used for recording extra information between host and device.

Member	Data Type	Description
peer_im_mem_addr	uint64_t	Address of the peer (remote) image memory object

RKNN3_IM_MEM

Image memory information structure, describing an image buffer stored in device memory, used by image interfaces such as `rknn3_im_mem_create`, `rknn3_im_cvt_color`, etc.

Member	Data Type	Description
width	int	Image width in pixels
height	int	Image height in pixels
stride	int	Image buffer stride in bytes
format	rknn3_im_fmt	Image format; see rknn3_im_fmt
sync_to_host	bool	Whether image data needs to be written back to the host; default false
data_mem	rknn3_tensor_mem*	Pointer to the underlying tensor memory information
metadata	rknn3_im_metadata	Extra metadata shared between host and device

6.9. Callback Types

This section summarizes the callback function pointer types used in LLM inference and the callback container structure. For detailed descriptions of each callback type, parameter semantics, and examples, see 5.8 LLM Configuration and Callbacks.

LLMRESULTCALLBACK

LLM result callback function type. Handles LLM generation results. For details, see LLMResultCallback.

Function Pointer Type	LLMResultCallback
Signature	`int (*LLMResultCallback)(void* userdata, RKLLMResult* result, LLMCallState state)`
Return	int Status code: see 6.2. Status Codes

LLMSAMPLINGCALLBACK

LLM sampling callback function type. Handles sampling logits. For details, see LLMSamplingCallback.

Function Pointer Type	LLMSamplingCallback
Signature	`int (*LLMSamplingCallback)(void* userdata, float16* logits, char* logits_name)`
Return	int: Returns the selected token ID (>=0) on success, negative value for error code; see 6.2. Status Codes

LLMGETEMBEDCALLBACK

Callback function type for getting LLM embedding vectors. For details, see LLMGetEmbedCallback.

Function Pointer Type	LLMGetEmbedCallback
Signature	`int (*LLMGetEmbedCallback)(void* userdata, int32_t* tokens, uint64_t num_tokens, void* embed, uint64_t len)`
Return	int Status code: see 6.2. Status Codes

LLMTOKENIZERCALLBACK

LLM tokenizer callback function type. For details, see LLMTokenizerCallback.

Function Pointer Type	LLMTokenizerCallback
Signature	`int (*LLMTokenizerCallback)(void* userdata, const char* text, int32_t text_len, int32_t* tokens, int32_t n_tokens_max)`
Return	int: Returns the number of generated tokens (>=0) on success, negative value for error code; see 6.2. Status Codes

LLMOUTPUTCALLBACK

Callback function type for getting output tensors. For details, see LLMOutputCallback.

Function Pointer Type	LLMOutputCallback
Signature	`int (*LLMOutputCallback)(void* userdata, rknn3_tensor* output_tensors, uint32_t n_output_tensors, LLMOutputCallbackState state)`

Function Pointer Type	LLMOutputCallback
Return	int: Returns 0 on success, non-zero on error; see 6.2. Status Codes

LLMINPUTCALLBACK

Callback function type for providing custom input tensors. For details (including the `LLMInputCallbackParam` parameter structure), see LLMInputCallback.

Function Pointer Type	LLMInputCallback
Signature	`int (*LLMInputCallback)(void* userdata, rknn3_tensor* input_tensors, uint32_t n_input_tensors, LLMInputCallbackParam param)`
Return	int: Returns 0 on success, non-zero on error; see 6.2. Status Codes

RKLLMCALLBACK

The RKLLMCallback structure contains callback functions for LLM operations, registered via `rknn3_session_set_callback`.

Member	Data Type	Description
result_callback	LLMResultCallback	Callback function for LLM result return
result_userdata	void*	User data for LLMResultCallback
sampling_callback	LLMSamplingCallback	Optional: only used when custom sampling is needed
sampling_userdata	void*	User data for LLMSamplingCallback
tokenizer_callback	LLMTokenizerCallback	Optional: only used when a custom tokenizer is needed
tokenizer_userdata	void*	User data for LLMTokenizerCallback
embed_callback	LLMGetEmbedCallback	Optional: only used when custom embedding retrieval is needed
embed_userdata	void*	User data for LLMGetEmbedCallback
output_callback	LLMOutputCallback	Optional: only used when output tensors are needed
output_userdata	void*	User data for LLMOutputCallback
output_tensors	rknn3_tensor*	Output tensor array pointer
n_output_tensors	uint32_t	Number of output tensors
input_callback	LLMInputCallback	Optional: only used when custom input tensors need to be provided
input_userdata	void*	User data for LLMInputCallback
input_tensors_index	int32_t*	Index array of input tensors
n_input_tensors	uint32_t	Number of input tensors

6.10. Structured Output Structures

Structured Output Structures

RKNN3_GRAMMAR_CONFIG

Grammar engine initialization configuration structure (context level), carrying the encoded vocabulary (token_id -> byte string) built by the caller plus stop token IDs. The runtime uses this to build the XGrammar TokenizerInfo and GrammarCompiler in one step.

`encoded_vocab` layout: for each token i in [0, vocab_size), store `int32_t len_i`, followed by `len_i` bytes of the token's string representation.

Member	Data Type	Description
vocab_size	int32_t	Number of tokens in encoded_vocab
encoded_vocab	const uint8_t*	Serialized vocabulary data block (layout above)
encoded_vocab_len	int32_t	Total bytes in encoded_vocab
n_stop_tokens	int32_t	Number of stop (EOS) token IDs
stop_tokens	const int32_t*	Array of n_stop_tokens stop token IDs

RKNN3_GRAMMAR

Opaque session-level grammar handle type. Holds the grammar rule string/type and per-request GrammarMatcher state, sharing the context-level TokenizerInfo / GrammarCompiler. Created via `rknn3_grammar_create`, destroyed via `rknn3_grammar_destroy`.

RKNN3_GRAMMAR_SET_OPTIONS

Compile options for `rknn3_grammar_set` / `rknn3_session_set_grammar`. A value of -1 (int fields) or NULL (string fields) means "use the xgrammar default"; a NULL options pointer is equivalent to all defaults. Options are applied at set time (each set recompiles the grammar).

Member	Data Type	Description
any_whitespace	int	JSON Schema: -1=default(true), 0=false, 1=true
indent	int	JSON Schema: -1=default(None), >=0=indent spaces
strict_mode	int	JSON Schema: -1=default(true), 0=false, 1=true
max_whitespace_cnt	int	JSON Schema: -1=default(None), >=0=max whitespace between elements
any_order	int	JSON Schema: -1=default(false), 0=false, 1=true
item_separator	const char*	JSON Schema: NULL=default, else element separator (comma)
key_value_separator	const char*	JSON Schema: NULL=default, else key-value separator (colon)
root_rule_name	const char*	EBNF: NULL=default("root")

6.11. MatMul Types

These types are defined in `rknn3_matmul_api.h`. Zero-initialize input structures, including their `reserved` fields.

rknn3_matmul_context

An opaque MatMul handle; zero is invalid. Created by `rknn3_matmul_init` and released by `rknn3_matmul_destroy`, with a lifecycle managed separately from its parent `rknn3_context`.

rknn3_matmul_type

W denotes the bit width of B (weights), and A denotes the bit width of input A. The actual types are listed below; use the queried `dtype` as the reference.

Enumeration	A	B	C	set_quant Requirements
RKNN3_MATMUL_W16A16	FP16	FP16	FP16	None
RKNN3_MATMUL_W4A16	FP16	INT4	FP16	B only, group quantization
RKNN3_MATMUL_W6A16	FP16	INT6	FP16	B only, group quantization
RKNN3_MATMUL_W8A16	FP16	INT8	FP16	B only, group quantization
RKNN3_MATMUL_W8A8_PER_CHANNEL	INT8	INT8	INT8	A/C per tensor, B per output channel
RKNN3_MATMUL_W8A8_PER_LAYER	INT8	INT8	INT8	A/C per tensor, B per layer
RKNN3_MATMUL_W8A8O32	INT8	INT8	INT32	None; K must not exceed 4096

`RKNN3_MATMUL_TYPE_MAX` is the enumeration boundary and cannot be used as an execution type.

rknn3_matmul_desc

Field	Description
M / K / N	Matrix multiplication dimensions; all must be greater than zero.
A_batch / B_batch	A/B batch counts; both must be greater than zero. C_batch=max(A_batch,B_batch) must be divisible by both.
type	Computation type of this handle.
A_layout / C_layout	Currently both set to `RKNN3_TENSOR_NCHW`, describing logical input/output.
B_layout	Set NCHW for ordinary B; the queried layout is packed weight storage. Single-shape W16A16 may use NC1HWC2 for feature B.
group_size	32, 64, or 128 for W4A16/W6A16/W8A16; zero for other types.
core_mask	Mask of a single NPU core; combining multiple cores is unsupported. AUTO currently selects core 0.
reserved[64]	Reserved; all bytes must be zero.

Only M and A_batch may differ within one handle. The type, core_mask, B_batch, K, N, group_size, B_layout, and queried B size and layout must match. Feature B applies only to single-shape W16A16.

Canonical shapes and logical storage: Queried attributes use `n_dims=4`. The shape always uses the logical coordinates below, even when the physical NPU layout is NC1HWC2.

Tensor	shape	Contiguous NCHW Logical Index
A	[A_batch,K,1,M]	`(a_batch*K+k)*M+m`
B	[B_batch,K,1,N]	`(b_batch*K+k)*N+n`
C	[C_batch,N,1,M]	`(c_batch*N+n)*M+m`

For a single batch, `C[n*M+m] = sum_k A[k*M+m]*B[k*N+n]` (W8A8O32). Other types use their corresponding floating-point or quantization rules. Do not replace logical shape dimensions with physically padded dimensions.

rknn3_matmul_config

Contains only `uint8_t reserved[256]`. Pass NULL for default building, or a completely zero-initialized structure.

rknn3_matmul_tensor_attr

Field	Description
n_dims / shape	Canonical logical shape, as listed above.
n_strides / strides	Physical storage strides, in storage elements rather than bytes. NC1HWC2 has five entries ordered as [N,C1,H,W,C2]; packed weights and contiguous logical endpoints have zero entries.
size	Size in bytes. For NPU storage, includes padding and specifies minimum capacity; for logical storage, specifies contiguous data size.
alignment	Required byte alignment of the effective NPU start address; ignored for logical endpoints.
quant_info	Quantization requirements; queries do not include actual scale/zero point arrays.
layout / dtype	Storage layout and computation data type; logical INT4/INT6 use unpacked int8 storage.
core_id	Core ID used for NPU memory allocation.
reserved[32]	Reserved; all bytes must be zero.

Use NPU attributes exactly as queried. `rknn3_tensor_mem.size` is the capacity of the entire buffer and differs from the attribute's `size`. Ensure `offset <= size` and sufficient remaining capacity. Allocate on the queried `core_id` and ensure that the effective start address `phys_addr + offset` satisfies `alignment`. Increase allocation size if extra alignment space is needed. CPU and NPU access must use the same offset.

rknn3_matmul_info

Field	Description
shape_id	Registered shape index.
type / group_size	Computation type and group size for this shape.
A / B / C	`rknn3_matmul_tensor_attr` for the three caller-owned NPU buffers.
reserved[32]	Reserved fields.

The queried A/C layout is NC1HWC2. When the descriptor's B_layout is NCHW, the queried B layout is O1I1HWI2O2; when B_layout is NC1HWC2, the queried B layout is NC1HWC2. Allocate A/B/C according to the query; the runtime manages other execution resources.

rknn3_matmul_quant_type_t

Enumeration	Description
RKNN3_MATMUL_QUANT_NONE	No quantization parameters.
RKNN3_MATMUL_QUANT_PER_TENSOR	One parameter set per tensor; B values are provided separately for each batch.
RKNN3_MATMUL_QUANT_PER_CHANNEL	One parameter set per output channel of B.
RKNN3_MATMUL_QUANT_GROUP_PER_TOKEN	B is grouped along K, with one parameter set per batch, group, and output channel.

rknn3_matmul_quantization_t

Field	Description
type / group_size	Quantization mode and group size; must match the query.
scale_dtype / zero_point_dtype	Scale data type and the operator's quantized zero point type.
scales / scales_size	Scale array pointer and byte size; element count is num_quant_params.
zero_points / zero_points_size	Zero point array pointer and byte size; storage is always unpacked `int32_t`.
num_quant_params	Number of parameter sets; prepare arrays according to the query.
reserved[3]	Reserved; all entries must be zero.

Queries return NULL `scales` / `zero_points`. For submission, copy the corresponding `quant_info`, then set the array pointers and actual byte sizes. `scales_size = num_quant_params * scale element size in bytes`, and `zero_points_size = num_quant_params * sizeof(int32_t)`. Do not pack zero points according to the bit width of `zero_point_dtype`.

Type	A Parameter Count	B Parameter Count	C Parameter Count	Scale Storage
W4A16/W6A16/W8A16	None	B_batch×ceil(K/group_size)×N	None	FP16 bit patterns, 2 bytes each
W8A8 per-channel	1	B_batch×N	1	FP32, 4 bytes each
W8A8 per-layer	1	B_batch	1	FP32, 4 bytes each

Quantization arrays must cover all batches and the corresponding groups or output channels. Use the queried `num_quant_params` for the required count. W16A16/W8A8O32 have no quantization arrays.

STORAGE CONVENTIONS FOR SCALES AND ZERO_POINTS

`scales` stores scale factors and `zero_points` (abbreviated as zp below) stores zero points. A quantized value q represents the real value `scale * (q - zp)`. Each quantization unit is described by a scale/zp pair. The two arrays have the same element count and logical shape; `scales[i]` and `zero_points[i]` always describe the same unit.

The arrays are stored separately and contiguously, without interleaving. The shapes below describe the logical dimensions of caller-supplied arrays, with the rightmost dimension contiguous. Do not pad or reorder them into the NPU NC1HWC2 or O1I1HWI2O2 layout, or supply a packed device Param buffer.

Computation Type	Tensor	Logical Shape of Both Arrays	Elements in Each Array	Scale Element Storage	Zp Element Storage
W4A16/W6A16/W8A16	B	`[B_batch,G,N]`, `G=ceil(K/group_size)`	`B_batch*G*N`	FP16 bit patterns, 2 bytes	`int32_t`, 4 bytes
W8A8 per-channel	A and C separately	`[1]`	1 each	`float` (FP32), 4 bytes	`int32_t`, 4 bytes
W8A8 per-channel	B	`[B_batch,N]`	`B_batch*N`	`float` (FP32), 4 bytes	`int32_t`, 4 bytes
W8A8 per-layer	A and C separately	`[1]`	1 each	`float` (FP32), 4 bytes	`int32_t`, 4 bytes
W8A8 per-layer	B	`[B_batch]`	`B_batch`	`float` (FP32), 4 bytes	`int32_t`, 4 bytes
W16A16/W8A8O32	A/B/C	None	0	None	None

`scales_size` and `zero_points_size` are byte sizes, not element counts. Although both pointers are declared as `const uint8_t*`, prepare their elements using the storage types above. FP16 scales must be converted to FP16 bit patterns; casting an FP32 array pointer does not convert its elements to FP16. All zp arrays use `int32_t`; do not use `int8_t` arrays or INT4/INT6 bit packing.

All scales must be finite and positive. Even when every zero point is 0, provide a zero-filled zp array rather than NULL. Both scale and zp arrays are required for each quantized tensor. Actual values come from the caller; queries return only format, count, and size requirements.

W4A16/W6A16/W8A16: B GROUPED ALONG K

A and C are FP16 and need no scale/zp submission. Call `set_quant(matmul, NULL, &quant_B, NULL)`. B has logical shape `[B_batch,K,1,N]`, with a separate parameter pair for each batch, K group, and output channel n.

Let `G=ceil(K/group_size)`. For logical B coordinates `(batch,k,n)`:

```
group = k / group_size                 // Integer division
index = (batch * G + group) * N + n
scale = scales[index]
zp    = zero_points[index]
B_real(batch,k,n) = scale * (B_quant(batch,k,n) - zp)
```

All k values within a group share scale/zp for the same output channel. Arrays are ordered by batch, then group, with `n=0..N-1` contiguous within each group. `group_size` is 32/64/128. If K is not divisible by group_size, the final partial group still requires N parameter pairs for its valid k values; do not increase the count based on physically padded K.

For example, `B_batch=2,K=65,N=4,group_size=32` gives `G=3`. Both arrays have shape `[2,3,4]` and contain 24 elements each:

Array Indices	B Batch	Valid k Values Covered	n Order
0..3	0	0..31	0..3
4..7	0	32..63	0..3
8..11	0	64	0..3
12..15	1	0..31	0..3
16..19	1	32..63	0..3
20..23	1	64	0..3

Coordinates `(batch=1,k=64,n=3)` use `scales[23]` and `zero_points[23]`. Here `scales_size=24*2=48` bytes and `zero_points_size=24*4=96` bytes.

All three types use the same array shape and indexing; the B weight bit width and queried zero point type differ. The current interface reports `zero_point_dtype=INT4` for W4A16, with zp values in `[-8,7]`; W6A16/W8A16 report `zero_point_dtype=INT8`, with zp values in `[-128,127]`. These constrain the values; array storage is always `int32_t`.

One successful B parameter submission suffices for the entire handle. Switching M/A_batch does not change B quantization arrays.

W8A8 PER-CHANNEL: ONE PARAMETER PAIR PER B OUTPUT CHANNEL

A, B, and C are INT8. Call `set_quant(matmul, &quant_A, &quant_B, &quant_C)`. A and C each use one scalar parameter pair covering all elements and all batches of that tensor. Their parameter counts do not increase with M, A_batch, or C_batch.

Both B arrays have shape `[B_batch,N]`. For `(batch,k,n)`, use `index=batch*N+n`; all k values of that output channel share `scales[index]` and `zero_points[index]`. Channel refers to B output channel n, not K or A row m.

For example, with `B_batch=2,N=3`, B array indices correspond to `(batch,n)=(0,0),(0,1),(0,2),(1,0),(1,1),(1,2)`. Each B array contains 6 elements and occupies 24 bytes. Each A/C scale or zp array contains 1 element and occupies 4 bytes.

The scale/zero point mappings are:

```
A_real = A_scale[0] * (A_quant - A_zp[0])
B_real(batch,k,n) = B_scale[batch*N+n] * (B_quant(batch,k,n) - B_zp[batch*N+n])
C_real = C_scale[0] * (C_quant - C_zp[0])
```

C parameters describe the quantization mapping of the INT8 output. All A/B/C scales are FP32; all zp values are stored as `int32_t` and must be in `[-128,127]`. Submit A/B/C once per shape. B array values must match across shapes, while the A/C scalar values may differ.

W8A8 PER-LAYER: ONE PARAMETER PAIR PER B BATCH

A/C parameter shapes, storage types, and submission are the same as per-channel. Both B arrays have shape `[B_batch]`; all K×N weights in each batch share one parameter pair:

```
index = batch
B_real(batch,k,n) = B_scale[batch] * (B_quant(batch,k,n) - B_zp[batch])
```

For example, `B_batch=2` requires 2 scale/zp elements in each B array, with each array occupying 8 bytes. A single batch requires 1 element and 4 bytes in each array. The parameter count does not increase with K or N. Scales are FP32; zp values use `int32_t` storage and must be in `[-128,127]`. Multiple-shape submission rules are the same as per-channel.

W16A16/W8A8O32: NO SCALE/ZP SUBMISSION

Do not call `set_quant()` for either type. W16A16 uses FP16 inputs directly. W8A8O32 uses INT8 inputs and produces INT32 accumulation results, without accepting additional A/B/C scale/zp arrays.

7. RKNN3 C API Change Log

v0.3.0 -> v0.4.0 Session API Main Changes (2025-11-03)

> Reference demo1: rknn3-runtime/examples/rknn3_session_test_demo
>
> Reference demo2: rknn3_model_zoo/examples/Qwen2_5_VL

1. Added multimodal `audio/video` input types, supporting multiple modalities in arbitrary order and combination.
2. When inputting a prompt, explicitly insert modality tags, e.g.: `You need to do 3 things: 1.<audio>Convert this speech to text; 2.<image>Describe the content of this image; 3.<video>Describe the content of this video`.
3. rknn3_llm_multimodal_tensor structure optimized: e.g., changed input_tensor.n_image members to input_tensor.image.n_image substructure representation.
4. Added `thinking mode` toggle.
5. Enable thinking mode by setting enable_thinking = true (default is false).
6. Added support for configuring multiple `eos tokens` to control inference termination.
7. params.vocab_info.special_eos_id initialization updated to list-based assignment.
8. Added `rknn3_dump_features` interface for running the model layer by layer and exporting intermediate tensors in `.npy` format. Supports auto-allocating output tensors when not provided.

v0.4.0b0 -> v0.5.0 Main Changes (2025-11-29)

1. Added post-processing related query commands and custom operator plugin mechanism.
2. Added `RKNN3_QUERY_POSTPROCESS_*` query items.
3. Added `rknn3_register_custom_ops_plugins` and related structures and enumerations.
4. LLM callback and configuration enhancements.
5. Added `rknn3_session_set_function_tools` interface.
6. Replaced "last hidden layer callback" with `LLMOutputCallback` + `LLMOutputCallbackState`.
7. `rknn3_llm_config` added `model_type` and `task_type` fields.
8. `rknn3_kvcache_policy_param` added recurrent parameters.
9. Image/YUV related interface improvements.
10. `rknn3_im_mem_create` parameters expanded, supporting size/core_id/flags.
11. Added `rknn3_im_proc_flag` enumeration for describing image processing flows.

v0.5.0 -> v1.0.0 Main Changes (2025-12-22)

1. Added performance and memory analysis interfaces.

2. Added `rknn3_profile_ops`, `rknn3_profile_mem`.

v1.0.0 -> v1.0.4 Main Changes (2026-05-09)

1. Added streaming weight loading interface.
2. Added `rknn3_load_weight_chunk` interface, supporting segment-by-segment upload of weight data to NPU devices in streaming mode.
3. `rknn3_load_model_from_path` and `rknn3_load_model_from_data` support passing NULL weight parameter to enter streaming weight loading mode.
4. `rknn3_model_init` in streaming weight loading mode only completes model configuration; weight data must be uploaded in segments via `rknn3_load_weight_chunk`.
5. Added `rknn3_create_mem_from_fd` interface, supporting creating tensor memory objects from file descriptors.
6. API header file optimization.
7. `rknn3_api.h` removed dependency on non-essential data types.

v1.0.4 -> v1.1.0 Main Changes (2026-08-03)

1. Added KV cache grouping mechanism: `rknn3_attention_type` enumeration, `rknn3_attention_kvcache_lens` / `rknn3_kvcache_len_group_info` structures, `RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO` query command, and attention KV cache length configuration fields in `rknn3_llm_config`.
2. Added interfaces: `rknn3_set_kvcache_group`, `rknn3_set_weight_mem`, `rknn3_session_pause`, `rknn3_session_resume`, `rknn3_mem_sync_range`.
3. Completed custom operator related definitions:
4. Added `rknn3_custom_op_attr` structure documentation.
5. `rknn3_custom_op_context` structure completed `get_param` function pointer field description.
6. Completed structure fields: `RKLLMCallback` (input_callback, etc.), `RKLLMRunState` (n_input_tokens, etc.), `rknn3_kvcache_policy_param` (checkpoint_tail_overwrite), `rknn3_llm_infer_param` (prefill_only/disable_sampling); `rknn3_sdk_version` field length corrected from 64 to 256; `RKNN3_MAX_LORA_NUM` corrected from 128 to 32.
7. Completed `RKNN3_QUERY_CUSTOM_STRING` query command enum value.
8. Fixed `rknn3_session_save_kvcache` interface parameter type: `char*` corrected to `const char*`.
9. `rknn3_mem_sync` interface added `MEM_SYNC_CHUNK_SIZE` environment variable description (FROM_DEVICE mode block transfer, default 2MiB).
10. Synchronized with the `rknn3_api.h` header file, correcting the following interface and data structure definition inconsistencies:
11. `rknn3_tensor_attr` structure: added the missing `n_orig_dims`, `orig_shape[RKNN3_MAX_DIMS]`, and `orig_layout` fields.
12. `rknn3_im_mem_create` interface: parameter types `width` / `height` / `size` / `core_id` corrected from `int` to `int32_t`.

13. Array-type fields in multiple structures: added missing array dimensions. The `reserved` field of `rknn3_core_mem_size` / `rknn3_config` / `rknn3_shape_info` / `rknn3_allocation_info` / `rknn3_init_extend` / `rknn3_llm_config` / `rknn3_vocab_info` / `rknn3_llm_extend_param` / `rknn3_llm_infer_param` corrected to `uint8_t[N]`; `rknn3_dev_mem_info`'s `node_mem_info` corrected to `rknn3_node_mem_info[RKNN3_MAX_NPU_NODE_NUM]`; `rknn3_devices`'s `devices` corrected to `rknn3_device[RKNN3_MAX_DEVS]`.

v1.1.0 -> v1.2.0 Main Changes (2026-10-22)

1. Added PagedAttention (paged KV cache) support:
2. Added the page-table binding interface `rknn3_set_kvcache_pages`, which builds page bindings by `(core_id, cache_type, request_index)`; page memory is managed by the caller through `rknn3_create_mem` / `rknn3_destroy_mem`.
3. Added the query command `RKNN3_QUERY_KVCACHE_PAGE_INFO` and data structures including `rknn3_kvcache_page_info`, `rknn3_kvcache_page_binding`, `rknn3_cache_page_attr`, and the `rknn3_page_cache_type` enumeration.
4. See Section 4.4.4 (Paged KV Cache Memory) for the usage flow.
5. Added structured output (grammar) interfaces based on xgrammar for JSON Schema/EBNF/regex output constraints:
6. Context-level engine initialization: `rknn3_init_grammar`.
7. Grammar handle management: `rknn3_grammar_create`, `rknn3_grammar_destroy`.
8. Grammar rule set/clear/reset: `rknn3_grammar_set`, `rknn3_grammar_clear`, `rknn3_grammar_reset`.
9. Sampling constraint: `rknn3_grammar_apply_mask`, `rknn3_grammar_accept_token`.
10. Session-level convenience interfaces: `rknn3_session_set_grammar`, `rknn3_session_clear_grammar`.
11. Added the `rknn3_grammar_type` enumeration, the `rknn3_grammar_config` / `rknn3_grammar` / `rknn3_grammar_set_options` structures, and the `RKNN3_GRAMMAR_SET_OPTIONS_INIT` initialization macro.
12. Completed enum values: `rknn3_tensor_type` added `RKNN3_TENSOR_INT6`; `rknn3_kvcache_clear_policy` completed with `RKNN3_KVCACHE_KEEP_N_TOKENS`.
13. Added the `keep_grammar` field to `rknn3_llm_infer_param` (the `reserved` array was reduced from 128 to 127 bytes): when enabled, the per-request grammar matcher is not re-initialized at the start/end of each `rknn3_session_run`, retaining the grammar state for continuous generation across multiple runs.
14. LoRA interfaces updated: `rknn3_lora_init` / `rknn3_lora_init_from_data` support repeated calls to register multiple weight files/buffers (adapter names must be unique across all registered files); `rknn3_lora_enable` and `rknn3_session_enable_lora` clarified that different adapters mapped to the same pattern slot cannot be enabled together (different sessions may reuse that slot; the runtime reapplies each session's LoRA binding before inference).
15. Added the `rknn3_load_model_from_device` interface, supporting loading RKNN3 models from the device-side filesystem (emmc). `model_path` / `weight_path` refer to paths on the

device (e.g., coprocessor side) filesystem, which are read and parsed locally by the device instead of the host.